

Assignment B: Closing the Loop

从语料到一条被服务的请求

回顾式讲义 · 基于 LLM-system-project 与它到 llm-serving 的接缝

Version 0.1.0 · 2026-07-29

1 讲义总览

这份讲义回顾一次为期几天的冲刺：把两个原本互不相干的仓库接成一条能从头跑通的链路——真实语料混合 → 从零预训练一个 0.19B 模型 → 指令微调 ablation → 增量 KV cache → 通过适配层接进 serving engine → 跨仓库 benchmark。

它和姊妹篇 Assignment A 的分工是：A 讲 serving engine 本身的设计，B 讲这条链路怎么闭合，以及闭合过程中测出来的东西怎么和预期不一样。

后半句是这份讲义真正的主题。这次冲刺里被推翻的预期比被证实的多，而被推翻的那些更值得写下来。

1.1 这条闭环闭上了什么，没闭上什么

闭上的：

1. 按 LLaMA 思路配比的混合语料（web / wikipedia / instruction），带一份记录实际达成比例的 manifest
2. 0.19B 模型从零预训练到约 492M token，2×RTX 3090，约 7 小时，干净收尾
3. 六个 arm 的指令微调 ablation（3 个学习率 × 有无 replay），配对评测
4. 增量 KV cache，附带一道正确性门禁——缓存解码必须与重算解码逐位相同
5. 自训权重穿过 serving engine 的完整请求链路，除 tokenizer 外链路上没有任何 HuggingFace 模型
6. 跨仓库 serving benchmark：TTFT、decode 分位延迟、吞吐，recompute 与 cache 两种模式直接对比

没闭上的，讲义会在对应位置反复说明：

1. 微调没有 loss masking，所以严格说这是 domain-adaptive continued pretraining，不是教科书意义上的 SFT (§2)
2. 2.8 token/param，离 Chinchilla 区间还很远——这是 wall-clock 决定的，不是配置失误 (§3)
3. 验证集有一处真实污染，指令那 10% 成分不干净 (§2)
4. 训练时那条 val loss 曲线并不是混合验证集上的 loss (§3)
5. DDP 性能比较的第一版结论被撤回；复测后的实验从设计上就不可能达到 $p < 0.05$ (§6)
6. 语料没有被自己爬下来，用的是现成的 HuggingFace 数据集

1.2 你可以亲手跑什么

按算力分三档。第一档不需要 GPU，而且覆盖了这份讲义里最值得亲手验证的东西。

无 GPU：

```
uv run pytest -q tests/test_kv_cache.py # 6 passed
uv run python cs336_systems/experiments/kv_cache_gate.py # 小随机模型上的正确性门禁
uv run python cs336_systems/data_harvest/build_mixed_corpus.py --dry_run
```

Resource requirements: CPU 即可，各几秒到一两分钟。--dry_run 只产出约 2 MB——配比逻辑和 manifest 都能验，只是数字没有统计意义。

单卡：

```
uv run python cs336_systems/experiments/eval_checkpoint.py --stream web=...
uv run python cs336_systems/experiments/sample_from_checkpoint.py
PYTHONPATH=<llm-serving> uv run python cs336_systems/experiments/bench_serving.py \
    --modes recompute cache
```

两卡：

```
uv run python cs336_systems/experiments/run_pipeline.py \
    --config cs336_systems/experiments/real_pretrain_config.json \
    --attention_kernel flash_attention_triton --ddp_wrapper flashddp
```

1.3 代码长什么样

新增的部分集中在两处：cs336_systems/data_harvest/（语料构建，三个脚本）和 cs336_systems/experiments/（评测、采样、benchmark、报告）。KV cache 是加在 cs336-basics/ 里的三个新方法，forward 一个字节没动。适配层在姊妹仓库里。

真正该先读的不是目录树，而是 cs336_systems/experiments/real_pretrain_config.json。那份 config 里有八个 _NOTE_* 字段，把每一次失败的根因直接写进了配置文件本身——改模型尺寸、batch size 或学习率之前应该先读它们。

字段	它记录的失败	讲义位置
_NOTE_ffn_init_bug	反复 NaN 发散的真根因，第二次发散之后才找到	§3
_NOTE_divergence_fix	第一次标定就发散，以及当时那个错误的诊断	§3
_NOTE_approx_param_count	参数量算错——head 是独立矩阵，不共享 embedding	§3
_NOTE_tokenizer_swap	自训 BPE 在真实语料上要 20 小时，换 tiktoken	§3
_NOTE_batch_size_oom	漏算 logits 张量，vocab 涨 5 倍这一项就翻 5 倍	§3
_NOTE_wandb_artifact_hang	2.3 GB checkpoint 上传把 rank 0 卡在 barrier 后面	§3
_NOTE_max_iters	用实测吞吐反推步数预算，而不是拍脑袋	§3
_NOTE_eval_frequency	预训练期 eval 为什么故意稀疏	§3

Table 1: real_pretrain_config.json 的八个 _NOTE_* 字段。把失败写进配置文件本身，是这次冲刺里最值得直接抄走的一个做法。

1.4 怎么用这份讲义

橙色 **Checkpoint** 框的用法和姊妹篇一样：先合上代码答，再看下半格的参考答案和代码位置。

两条 B 专属的约定：

- 每张结果表都带出处——哪个 JSON、多少 batch、什么 seed。这份讲义有一整章在讲「不要报没有出处的数字」，如果它自己的表格没有出处就自相矛盾。看到没有出处的数字，就当它不存在。
- 本讲义含被撤回的结论。深红色的「撤回」框装的是曾经报出去、后来自己推翻的结论，原样保留是故意的：§6 整章的主题就是「怎么才有资格说一个性能结论」，删掉被撤回的那一版就没得讲了。

1.5 前置：假定你读过姊妹篇的哪几节

这份讲义不重复 serving engine 的设计论证。以下几处会直接指过去：

- Assignment A §3.4 〈四个方法逐行读〉——§5 讲适配层「只补四样东西」时的前提
- Assignment A §3.2 〈KV cache 为什么合法〉——为什么可以缓存。B 只讲怎么写、怎么测、值不值

- Assignment A §5.1 〈差分测试：让参照实现来定义“正确”〉——用一个可信实现当 oracle。B 里这个手法用了两次
- Assignment A §3.1 〈为什么不用 generate()〉——为什么 LMEngine 手写驱动模型，这是适配层能存在的原因

没读过姊妹篇也能读这份，但 §5 会少掉一半的「为什么是这样」。

2 数据：一个混合语料，以及它在验证集上骗了我

这一章要讲的东西写起来只有三百行 Python，但它是整条链路上唯一一个我们自己造出来、没有任何外部基准可以对照的环节。模型结构抄的是标准 pre-norm Transformer；训练循环早就写好了；serving 那一端还有 HuggingFace 的 generate() 可以做差分测试。只有语料是自己拼的——拼错了，下游每一个数字都跟着错，而且不会有任何东西报错。

先把痛点摆在前面。这次冲刺的数据部分，一开始能讲的只有一句话：

常见误解：我用了 LLaMA 的配比：C4 70%、Wikipedia 20%、instruction 10%。

这句话在没有分域验证集之前是零证据的。它描述的是一个输入——我们往语料里塞了多少字节——而不是一个结果。模型有没有因为那 20% 的 Wikipedia 而在 Wikipedia 上更好？有没有因为那 10% 的指令数据而学会 `### Response:` 这个格式？只报配比，这两个问题一个都没回答。真正让这句话变成可检验命题的，是 §2.5 那三条分域验证流；而在把它们造出来的过程中，我们发现混合验证集本身是错的。

本章路线：配比怎么定 (§2.1)、小域怎么凑够占比 (§2.2)、一个没有测过的问题 (§2.3)，然后是重点——一次从「加权平均对不上」查到「自己的数据脚本有 bug」的追查 (§2.4)；最后是修法 (§2.5) 和微调语料 (§2.6)。

2.1 配比从哪来：70 / 20 / 10

三个源，全部走 HuggingFace 的 streaming 接口，按原始字符预算切片：

- allenai/c4 (en), web text, 目标 70%，预算 546,000,000 字符；
- wikimedia/wikipedia (20231101.en), 目标 20%，预算 156,000,000 字符；
- tatsu-lab/alpaca, instruction / response, 目标 10%，预算 78,000,000 字符。

合计 780M 字符的 train，外加 20M 字符的 valid (14M / 4M / 2M，同样 70/20/10)。两组预算是硬编码常量，见 `cs336\_systems/data\_harvest/build\_mixed\_corpus.py:37` 与 `cs336\_systems/data\_harvest/build\_mixed\_corpus.py:38`。

为什么这是「受启发」而不是「复现」。这里要说得硬气一点，因为「用了 LLaMA 的配比」是一句很容易被追问的话。LLaMA 原论文的混合表有 7 个域 (CommonCrawl、C4、GitHub、Wikipedia、Books、ArXiv、StackExchange)。我们只留了 3 个，而且：

- 70/20/10 这三个数不是把原表按剩余域重新归一化得到的。把 CommonCrawl 和 C4 合起来算「web」大约是 82%，Wikipedia 只有 4.5%，都对不上 70/20。这三个数是我们自己拍的，只保留了「web 主导」这个定性特征。
- instruction 这个域 LLaMA 论文里根本没有。它是为了让后面的指令微调有一个「格式已经见过」的起点才加进来的。
- 真正借来的是机制而不是数字：配比不靠按字节平均切，而是靠给小而精的域跑多个 epoch。这一点写在 `cs336\_systems/data\_harvest/build\_mixed\_corpus.py:7` 的模块 docstring 里，下一节展开。

所以正确的说法是：一个受 LLaMA 数据混合思路启发的三域配比，不是 LLaMA 配比的复现。这个区别在简历上值两句话的篇幅，别省。

源（目标占比）	split	实际字符	文档数	passes	oversample
allenai/c4 (70%)	train	546,000,616	251,371	1.00	否
	valid	14,003,074	6,560	1.00	否
wikimedia/wikipedia (20%)	train	156,001,590	25,167	1.00	否
	valid	4,009,029	829	1.00	否
tatsu-lab/alpaca (10%)	train	78,000,388	192,699	3.90	是
	valid	2,000,066	4,956	1.91	是（错的）

Table 2: manifest.json 记录的实际结果（2026-07-27 的正式构建，转录自 run_logs/master_pipeline.log）。train 三域的 achieved_ratio 精确落在 0.7000 / 0.2000 / 0.1000，valid 是 0.6997 / 0.2003 / 0.0999。最后一行的「错的」是 §2.4 的主题。

2.2 小域怎么凑占比：cycle 不是 hack

Alpaca 一共 52,002 条样本。脚本先按 5% 切出 held-out 池：train 池 49,402 条，valid 池 2,600 条（cs336_systems/data_harvest/build_mixed_corpus.py:105）。每条格式化成

```
### Instruction:
{instruction}
### Input:      # 只有 input 非空时才有这一段
{input}
### Response:
{output}
```

平均一条约 405 字符。49,402 条跑满一遍大约 20M 字符，而 instruction 这个域的预算是 78M——差 3.9 倍。于是 build_instruction() 直接循环这个池子直到填满预算：

```
137     written = docs = 0
138     n = len(pool)
139     idx = 0
140     while written < char_budget:
141         written += _write_doc(fh, pool[idx % n])
142         docs += 1
143         idx += 1
144     return {
145         "source": "tatsu-lab/alpaca",
146         "chars": written,
147         "docs": docs,
148         "pool_size": n,
149         "passes": round(idx / n, 2),
150     }
```

idx % n 这一行就是全部的 oversample 实现，passes 字段把倍数记进 manifest。

这里值得停一下：这不是一个为了凑数字的 hack，它就是 LLaMA 论文里那个做法本身。一个只有 20M 字符的高质量域，如果坚持「按字节切、每个域只看一遍」，它在 780M 的混合里最多只能占 2.6%；想让它占 10%，唯一的办法就是让模型在它上面多走几遍。LLaMA 对 Wikipedia 和 Books 也是这么处理的——配比表里那个百分比描述的是采样权重，不是磁盘上的字节数。

Checkpoint 2.1 (oversample_is_the_recipe): 为什么「重复采样小域」不是作弊

不看代码回答：

- (a) 一个只有 20M 字符的域，在 780M 字符的混合语料里，如果只允许出现一遍，最多能占多少？想让它占 10%，你有哪两种办法？

- (b) `passes=3.90` 和 `passes=1.91` 这两个数分别出现在哪个 split 上? 其中一个是对的、一个是错的——凭什么这么说?
- (c) 如果把 instruction 的预算从 78M 提到 156M (占比 20%), `passes` 会变成多少? 这时模型第一次和第八次看到同一条 Alpaca 样本, 学到的东西一样多吗?

Hint 「配比」这个词描述的是采样权重, 不是磁盘字节数。一旦接受这一点, (a) 的第二种办法就很显然了。

参考答案: (a) 最多 $20/780 \approx 2.6\%$ 。两种办法: 要么把其他域砍到只剩 180M 字符 (贵, 等于扔掉 77% 的 web 数据), 要么把小域重复采样 3.9 遍 (便宜, 这是 LLaMA 的做法, 也是这里的做法)。

(b) 3.90 在 train split 上, 1.91 在 valid split 上。train 上重复是对的, 因为训练集的作用就是按配比给梯度加权; valid 上重复是错的, 因为验证集的作用是估计模型在未见数据上的表现, 重复不增加任何未见数据, 只是把同一批样本在平均里数了两遍——而且会让它和别的流不再可比 (§2.4)。

(c) $156/20 \approx 7.8$ 遍。不一样: 小语料上跑多个 epoch 的边际收益递减, 同时过拟合风险上升。LLaMA 那个「小域多跑几遍」的做法在 1T token 尺度上是安全的; 在我们这个 174M unique token 的语料上, 整体已经跑了 2.8 遍 (§3), instruction 这个域实际被看到的次数是 $2.8 \times 3.9 \approx 11$ 遍。这个数我们没有做消融, 不知道是不是太多了。

代码位置: `cs336/_systems/data/_harvest/build/_mixed/_corpus.py:140`

Downscaling Tip: 先用 `--dry_run` 把配比逻辑跑通, 再去烧带宽

`build_mixed_corpus.py--dry_run` 换用一组小 20 倍的预算 (1.4M/0.4M/0.2M 字符, `cs336/_systems/data/_harvest/build/_mixed/_corpus.py:41`), 产出约 2MB 的 `train.txt + valid.txt` 和一份完整的 `manifest.json`。

它能验的: 三个 streaming 源都连得上、配比归一化算得对、`passes` 字段写得对、下游 tokenize 能吃这个文件格式。它不能验的: 任何一个 loss 数字。2MB 语料上训出来的东西没有意义, 别去看它的 val loss。

Resource requirements: CPU 即可, 主要开销是网络; HuggingFace Hub 若限流 (429) 需要配 `HF_TOKEN`。

2.3 一个没有测过的开放问题: 字符配比不等于 token 配比

`manifest.json` 里的 `achieved_ratio` 是字符比例。但模型看到的是 token, 而每个域的「字符/token」换算率并不一样: web 文本里有大量常见英文词 (一个 token 吃掉四五个字符), Alpaca 里有 ###、换行、密集标点这类结构标记 (一个 token 常常只吃一两个字符)。

训练语料每个域各自的 token 数, 我们没有测过。原因很实际: `data/mixed_corpus/` 在本地是空目录 (可以自己 `ls` 确认), 全量语料留在那台已经 stop 掉的 vast.ai 实例的磁盘上, 重建一次要 2-3 小时。

能确定的只有总量: 780,002,594 字符编码成 173,972,065 个 token (tiktoken 的 gpt2 编码, 见 §3.2), 平均 4.48 字符 / token。

能估计的: 分域验证流用了一个每域相等的 1.2M 字符预算, 产出 267K / 281K / 226K token (web / wiki / instruction), 换算率分别约 4.5 / 4.3 / 5.3 字符 / token。把这三个率套回训练语料的字符预算:

$$\underbrace{\frac{546}{4.5}}_{121 \text{ M}} : \underbrace{\frac{156}{4.3}}_{36 \text{ M}} : \underbrace{\frac{78}{5.3}}_{15 \text{ M}} \implies 70.4\% / 21.1\% / 8.5\%$$

mixed_valid 区段	权重	pretrained	lr1e5_replay	delta	加权贡献
web	0.70	3.7717	3.7854	+0.0137	+0.0096
wikipedia	0.20	3.5247	3.5515	+0.0268	+0.0054
instruction	0.10	2.3863	2.4787	+0.0924	+0.0092
加权合计					+0.0242
整条 mixed_valid 上直接实测		3.5866	3.6104	+0.0238	

Table 3: 切开之后，算术闭合。数字取自 artifacts/eval/ 下的 slice_pretrained.json、slice_lr1e5_replay.json、pretrained.json、sft_lr1e5_replay.json，均为 200 batch、固定 seed 的配对打分。

三项加起来 172M，和实测的 174M 差 0.7%，说明这个换算方向大致对。但结论只能写成：token 层面的 instruction 占比可能不是 10% 而是 8%–9%，我们没有直接测过。要测很便宜——分域各 tokenize 一次，比较 `len(arr)`——只是当时没做，现在数据不在手边。

这个数字重要吗？对「配比起没起作用」这个定性结论不重要（8.5% 和 10% 不会翻转任何排序）。对「我复现了 LLaMA 的 70/20/10」这句话重要：如果 LLaMA 的百分比指的是 token，那我们报的就不是同一个量。

2.4 验证集污染：从一个对不上的加权平均查起

这一节是本章的重点，也是整个冲刺里最值得复述的一次 debug。它的形状很典型：一个算术恒等式不成立，顺着查下去发现是自己造数据的脚本有 bug。按发生顺序走，五步。

第一步：症状——微调 delta 的符号反了

指令微调有六个 arm，每个 arm 都要在两类流上打分：general（混合验证集 mixed_valid.npy）和三条分域流。winner 那个 arm（lr1e5_replay）的分域结果是 web -0.0093、wiki +0.0138、instruction -0.3263（负号 = 更好）。按 70/20/10 加权应该得到

$$0.70 \times (-0.0093) + 0.20 \times (+0.0138) + 0.10 \times (-0.3263) = -0.0364,$$

也就是 general 应该变好 0.036。实测是 +0.0238——变差了。符号相反，不是幅度不对。

第二步：先排除采样噪声

eval_checkpoint.py 每次打分是从流里随机抽 200 个 batch（固定 seed 的 shuffle，cs336/systems/experiments/eval_checkpoint.py:37），所以第一反应必须是「会不会只是抽样抽歪了」。换两个 seed 重打，三次的 general delta：

$$+0.0238, \quad +0.0240, \quad +0.0244$$

散布 ± 0.0003 ，比 $0.0364 + 0.0238 = 0.0602$ 这个待解释的差距小两个数量级。排除了采样噪声，说明是结构性问题。

这一步花了大约十分钟，但它把假设空间从「测量误差 / 数据不同 / 代码 bug」砍成了后两个。跳过它的代价是：你会开始猜，然后猜错。

第三步：把 mixed_valid 按域边界切开直接打分

第二步说明「后造的那三条分域流」和「mixed_valid 里对应的区段」大概不是同一批数据。验证方法很直接：不用后造的流，直接把 mixed_valid.npy 按域边界切成三段，再打一次分。

+0.0242 对 +0.0238，残差 0.0004，落在第二步量出的 ± 0.0003 噪声带边上。算术完全闭合——从来就没有测量错误。真正的结论是：mixed_valid 的 instruction 区段上，微调让模型变差

了 0.0924；而在后造的干净 instruction 流上，同一个 checkpoint 变好了 0.3263。同一批 2,600 条 Alpaca 样本，两条流，符号相反。

Example 2.1 (chasing_the_weighted_average): 三步把一个 0.06 的差距追成一个 bug

每一步只做一件事，并且明确记下它排除了什么。

第 0 步（观察） 分域 delta 加权 = -0.0364 ，直接实测 = $+0.0238$ ，差 0.0602 且符号相反。待排除的假设：{采样噪声、两条流不是同一批数据、打分脚本有 bug、加权用错了权重}。

第 1 步（换 seed 重打三次） $+0.0238 / +0.0240 / +0.0244$ ，散布 ± 0.0003 。排除了：采样噪声。差距比噪声大 200 倍，不可能是抽样抽出来的。代价：三次打分，约 3 分钟 GPU。

第 2 步（不用后造的流，直接切 mixed_valid） 见表 3: $0.70(+0.0137) + 0.20(+0.0268) + 0.10(+0.0924) = +0.0242$ ，对实测 $+0.0238$ 。

排除了：打分脚本有 bug、权重用错。加权公式是对的，只要喂给它的是同一批数据，它就闭合。留下：唯一还活着的假设是「后造的三条流不等于 mixed_valid 的三个区段」。代价：写一个按域边界切 .numpy 的十行脚本，加六次打分。

第 3 步（去读造数据的代码） 到这一步已经不需要再跑实验了，只要回答一个问题：这两条流是怎么造出来的，哪里不一样。答案在 build_instruction() 上，见下文。

这个例子里最该学的不是结论，是第 1 步的顺序。「先量噪声、再谈差异」这条纪律在本项目里还救过另外几次场——反过来，§6 里 DDP 三种策略之间那几个百分点的比较，正是因为没有先量噪声而被整段撤回。

第四步：根因——train split 上正确的做法被用到了 valid split 上

build_instruction() 在 main() 里被调用了两次：一次填 train 预算 (cs336_systems/data_harvest/build_mixed_corpus.py:184)，一次填 valid 预算 (cs336_systems/data_harvest/build_mixed_corpus.py:198)。它内部无条件 cycle。

对 train split, cycle 是正确的——那就是 §2.2 讲的整个设计。对 valid split, 它把 2,600 条 held-out 样本重复了 1.91 遍，只为了让验证集的字符配比也是 70/20/10。

而验证集根本没有理由去匹配训练集的混合配比。验证集要回答的是「模型在没见过的数据上表现如何」，重复一遍不会增加任何没见过的数据。

代价是实测出来的：预训练模型在 mixed_valid 的 instruction 区段上是 2.3863，在同样这 2,600 条样本、不重复的那条流上是 2.7345——差 0.348 nats。而且它把微调 delta 的符号弄反了 ($+0.0924$ 对 -0.3263)。

结论和修法已经写回代码里了，就在 build_instruction() 的 docstring (cs336_systems/data_harvest/build_mixed_corpus.py:117) ——把结论写在最容易被下一次调用者读到的地方，比写在某个 markdown 里有用。

第四步补：写这份讲义时才发现，那 0.348 nats 大概率不是「重复」造成的

docstring 里给的机制解释是「一个 1024-token 窗口里可能装着同一条样本两次，第二遍靠 in-context 复制就能预测」。复盘时把数字代进去，这个机制站不住：

- 一条 Alpaca 样本约 87 个 token，2,600 条跑一遍是约 2.3×10^5 个 token。同一条样本的两次出现相隔约 20 万 token，而上下文只有 1024。一个窗口里不可能装下同一条样本两次。
- 更根本的一点：对一个权重已经冻结的模型，把同一批文档重复一遍，平均每 token 的 loss 期望不变——那还是同一批文档，只是在平均里被数了两次。重复本身不可能凭空产生 0.348 nats。

脚本	文档之间写什么	代码位置
build_mixed_corpus.py (预训练语料)	doc+"\textbackslash"+EOT+"\textbackslash"	build_mixed_corpus.py:51
build_domain_valid.py (分域验证流)	EOT.join(docs), 无换行	build_domain_valid.py:35
build_sft_corpus.py (微调语料)	doc + EOT, 无换行	build_sft_corpus.py:43

Table 4: 三种写法，三个文件（都在 `cs336_systems/data_harvest/`）。预训练模型只见过第一种，所以在后两种上每遇到一个文档边界就要吃一次意外。

域	每文档 token (估)	边界密度	两条流 loss 差	每边界摊到 (nats)
instruction	87	1.15×10^{-2}	0.3482	30.3
web	475	2.11×10^{-3}	0.0752	35.7
wiki	1,133	8.8×10^{-4}	0.0271	30.7

Table 5: 文档长度跨了 13 倍，「每个文档边界摊到多少额外 nats」却落在 30–36 这个窄区间里。这是复盘时用已有数字算的，没有跑新实验。

那 0.348 nats 从哪来？最可能的是文档分隔符不一致。三个脚本写文档边界的方式各不相同：

如果这个解释对，那么两条流的 loss 差应该正比于文档边界的密度，也就是反比于平均文档长度。用手头已有的数字验算：

三个域的文档长度差 13 倍，摊到每个边界的额外代价却几乎是常数。这对「分隔符不一致」是很强的支持，对「重复导致 in-context 复制」则没有解释力——web 和 wiki 两条流都没有重复，却表现出同一个规律。

必须说清楚这个结论的不确定性：web 和 wiki 的文档数是我用混合语料里的平均文档长度估出来的（分域流的 .npy 不在本地）；打分本身有 ± 0.01 的 seed 间散布，wiki 那一行的 0.0271 只比噪声大两三倍。这是一个未经测试的假设，不是结论。要证伪它很便宜：把 build_domain_valid.py 的 _encode 改成和 _write_doc 一样的分隔符，重新编码，再打一次分——如果 0.348 大幅缩小，假设成立。

撤回 (retracted): 重复过的验证数据是假的容易，白送了 0.35 nats。

这句话现在写在 build_instruction() 的 docstring 和 real_pretrain_config.json 的 note 里，两处都保留原样，因为它是当时能写出的最好解释。

站不住的地方：重复距离约 2×10^5 token，上下文只有 1024，窗口内复制不可能发生；而且对冻结的模型，重复同一批文档不改变期望 loss。

不受影响的部分（这一点同样重要，别一起丢掉）：2.3863 和 2.7345 两个数是实测的；表 3 的算术闭合是成立的；「valid split 不要 cycle」这条修法仍然是对的——只是它对的理由是「验证集不需要匹配训练配比、重复不产生新信息」，而不是「重复让验证变容易」。

第五步：影响范围，以及什么没有被影响

范围必须说准：说宽了等于把所有结论都作废，说窄了等于没承认。

- 受影响：预训练那条 val 曲线，以及 general 这个指标——它 10% 的成分 (instruction 区段) 来自一条构造方式和其他流不一致的数据。任何拿 general 的绝对值说事的地方都要打折扣。
- 不受影响：arm 之间的排序。六个 arm 打的是同一批数据、同一个 seed、同一组 batch，是配对比较；一个所有 arm 共享的偏置不改变谁比谁好。这也是为什么最后没有推翻指令微调的任何结论 (§4)。

- 干净的数：分域那三条流。它们都没有 oversample，而且各自独立编码。

修法写进了 docstring：valid split 不 cycle；验证集不需要匹配训练混合配比；让每个域贡献它 held-out 池子实际的量，然后分域分别报。

Checkpoint 2.2 (sign_flip): 同一批样本，两条流，delta 符号相反

不看代码回答：

- 微调后的 lr1e5_replay 在 mixed_valid 的 instruction 区段上是 +0.0924（变差），在 domain_valid_instruction 上是 -0.3263（大幅变好）。这两条流装的是同一批 2,600 条 held-out Alpaca 样本。什么样的差异能让同一批文本给出相反的符号？
- 如果只看得到 general 这一个数（+0.0238），你会得出关于这次微调的什么结论？它错在哪？

Hint 微调语料 alpaca_train.txt 是用 build_sft_corpus.py 的 _write_pool 写的，不是用 build_mixed_corpus.py 的 _write_doc。去比较这两个函数在文档之间写了什么。

参考答案：(a) 两条流的文档分隔符不同（表 4）：mixed_valid 用的是预训练见过的写法（换行加 EOT 再加换行），domain_valid 和微调语料用的是裸 EOT。微调之后模型适应了裸 EOT 那种写法，于是在用同样写法的流上大幅变好、在预训练写法的流上变差。同一批文本、不同边界约定，就足够让符号翻转。注意这是复盘时提出的解释，没有跑实验验证（见上面的 retraction）。

(b) 只看 general 会得出「微调让模型全面变差了一点点」。方向上不算错（灾难性遗忘确实存在），但幅度和归因都不对：变差的那 +0.0238 里有接近四成来自 instruction 区段（加权贡献 +0.0092，见表 3），而 instruction 恰恰是这次微调最该变好的域。一个混合指标可以把两个方向相反的效应压成一个看起来温和的数字。这就是要分域报的理由。

代码位置：cs336_systems/data_harvest/build_mixed_corpus.py:51 与 cs336_systems/data_harvest/build_sft_corpus.py:43

Checkpoint 2.3 (arithmetic_closure): 为什么「算术闭合」比「找到 bug」更早成为结论

不看代码回答：

- 表 3 里加权得到 +0.0242、直接实测 +0.0238。这个 0.0004 的残差应该和哪个已经测过的量比较，才能判断它算不算「闭合」？
- 闭合这个事实排除了哪几个假设？它没有排除哪个？
- 如果闭合失败（比如加权得到 +0.024、实测 -0.010），下一步该查什么？

Hint 加权公式本身是个恒等式：只要三段的权重就是它们在流里的 token 占比，加权平均必然等于整体平均。所以闭合失败只能有一个来源。

参考答案：(a) 和第二步量出的 seed 间散布 ± 0.0003 比。0.0004 落在这个量级上，所以是闭合而不是「差一点」。没有先量噪声就没法回答这一问——这就是第二步的价值。

(b) 排除了：打分脚本算错、权重用错、模型加载错。没有排除的是「这两条流装的不是同一批数据」——恰恰相反，闭合把这个假设变成了唯一还活着的一个。

(c) 查权重：三段在 mixed_valid 里的实际 token 占比是不是真的 70/20/10。字符配比是 70/20/10，token 配比可能不是 (§2.3)——如果 instruction 实际只占 8.5%，加权结果会偏。

这次没踩到，是因为那个偏差比残差还小。

代码位置: `cs336\_systems/experiments/eval\_checkpoint.py:30`

Checkpoint 2.4 (valid_contamination): 一行 `idx % n`，两个 `split`，一对一错

不看代码回答：

- `build_instruction()` 被调用了两次。同一段代码，为什么一次对、一次错？把「对」和「错」的判据分别写成一句话。
- 假设只能改一处，让这个函数在两个 `split` 上都正确。改哪里最省？不止一种答案，说出你选的那种的代价。
- 这个 bug 在 `manifest.json` 里其实留下了记录。是哪个字段？为什么当时没人看它？

Hint 判据不在代码里，在两个 `split` 各自的用途里。

参考答案: (a) `train split` 的用途是按配比给梯度加权，重复采样正是实现配比的手段，所以对。`valid split` 的用途是估计未见数据上的表现，重复不产生未见数据，只会让这条流和别的流不再可比，所以错。

(b) 三种: (1) 给 `build_instruction` 加一个 `allow_cycle` 参数，`valid` 调用传 `False`——最小改动，但把决定权留给了调用方，下次还可能传错；(2) `valid split` 干脆不设字符预算，直接把 `held-out` 池整个写进去——这是 `docstring` 里给的修法，代价是 `valid` 的三域配比不再是 70/20/10，而这恰恰是想要的；(3) 在函数里断言预算不超过池子的总量，越界就报错——最防御，但对 `train` 是误报。选了 (2)。

(c) `passes: 1.91`。它一直写在 `manifest.json` 的 `valid` 段里，构建时还打印到了 `stdout` (`cs336\_systems/data\_harvest/build\_mixed\_corpus.py:199`)。没人看，是因为当时关注的是「配比对不对」(`achieved_ratio` 是 0.0999，完美)，而 `passes` 被当成了只有 `train split` 才需要关心的字段。一个字段在一个 `split` 上是特性、在另一个 `split` 上是缺陷，这种字段最容易被漏看。

代码位置: `cs336\_systems/data\_harvest/build\_mixed\_corpus.py:116`

2.5 分域验证流：三条独立、等大、都不 **oversample**

`build_domain_valid.py` 是上面那次追查的直接产物。它写三个独立的 `.npz`：

- `domain_valid_web.npz`: C4 的真 `validation split`；
- `domain_valid_wiki.npz`: Wikipedia 没有 `validation split`，所以跳过训练语料消费掉的那一段再取；
- `domain_valid_instruction.npz`: Alpaca 的 `held-out` 池，全部 2,600 条，只跑一遍。

三条都用同一个每域相等的字符预算 `--target_chars`，默认 1,200,000 (`cs336\_systems/data\_harvest/build\_domain\_valid.py:41`)。

为什么要等大。三个理由，按重要性排：

- 验证集不该复刻训练配比。这正是 §2.4 那个 bug 的来源——「`valid` 也要是 70/20/10」这个念头本身就是错的。一旦放弃它，每个域取多少就变成一个纯粹的工程问题，而不是一个要匹配的约束。

- (2) 每个域的数字要带同样大小的误差棒。这三条流的用途是横向比较七个 checkpoint 在三个域上的 delta。如果 instruction 只有 web 的 1/7 大，它那一列的抽样方差就会大得多，而 instruction 恰恰是 delta 最大的一列——很容易把「噪声大」误读成「效应强」。
- (3) 便宜。每条几十万 token、200 个 batch，一个 checkpoint 三条流打完不到一分钟。六个 arm 加一个预训练基线一共七次，这个开销小到可以每次都跑，而不是「等有空再补」。

两处如实的保留。第一，「等大」是字符等大，不是 token 等大：三条流实际是 267K / 281K / 226K token，最大和最小差 24%。第二，`build_domain_valid.py` 的 docstring 说它取的是「`build_mixed_corpus.py` 用于 valid split 的同一批 held-out 切片」，这句话对 web 和 instruction 成立，对 wiki 不成立：WIKI_VALID_SKIP = 200_000 (`cs336\systems\data\harvest\build\_domain\_valid.py:31`)，而训练语料实际只消费了 25,167 篇（表 2）。跳得更远只会更安全（肯定不重叠），但两条 wiki 流确实不是同一段文本——实测也对得上：预训练模型在 `domain_valid_wiki` 上是 3.5518，在 `mixed_valid` 的 wiki 区段上是 3.5247。

Checkpoint 2.5 (why_equal_size_domain_valid): 三条验证流为什么要等大，又为什么不必等太准

不看代码回答：

- (a) 如果照搬训练配比，把三条验证流做成 70/20/10 大小，哪一列的结论最容易被误读？为什么？
- (b) 「等大」在这里指的是等字符还是等 token？实际差了多少？这个差影响 (a) 的论证吗？
- (c) 这三条流是用来报绝对质量的，还是用来做比较的？这个定位决定了它们可以小到什么程度？

Hint 想想 instruction 那一列的 delta (-0.3263) 和 wiki 那一列 ($+0.0138$) 分别有多大，再想它们各自的误差棒有多大。

参考答案: (a) instruction。它只占 10%，样本量最小、抽样方差最大，同时 delta 的绝对值又最大（比 web/wiki 大一个数量级）——最容易把大方差读成强效应，或者反过来把真实效应淹进噪声里。

(b) 等字符（`--target_chars` 默认 1.2M）。实际 token 数 267K / 281K / 226K，最大最小差 24%。不影响 (a) 的论证：24% 的样本量差异对应约 12% 的标准误差，远小于 70/20/10 带来的 $\sqrt{7} \approx 2.6$ 倍差异。

(c) 比较。`eval_checkpoint.py` 固定 seed，所以每个 checkpoint 打的是同一批 batch，是配对比较，方差大部分抵消（`cs336\systems\experiments\eval\_checkpoint.py:37`）。这就是为什么几十万 token 就够——真正让 0.05 量级的差异可分辨的是配对，不是样本量。反过来说，这三个 loss 的绝对值不该被当成模型质量的权威数字。

代码位置: `cs336\systems\data\harvest\build\_domain\_valid.py:13`

2.6 微调语料与 replay 变体

`build_sft_corpus.py` 造两个 arm 的训练流。这一节只讲语料怎么造；哪个 arm 赢、赢多少，归 §4。

格式复用，不重写。它直接从 `build_mixed_corpus` 导入 `_load_alpaca_pools`（`cs336\systems\data\harvest\build\_sft\_corpus.py:31`），所以微调看到的 `### Instruction: / ### Response:` 结构和预训练那 10% 完全一致，而且 train/valid 的切分也是同一个 `seed=0` 的 shuffle——不会出现「微调集里混进了预训练验证集的样本」这种泄漏。

需要补一句：这个复用只保证了文档内部的格式一致，文档之间的分隔符并不一致（表 4）。模块 docstring 里 byte-identical prompt structure 这个说法在边界处站不住。

replay 变体的配比公式。目标是让预训练数据占最终 token 数的 $f = 25\%$ 。设 Alpaca 的 token 数为 n_a ，要拼进去的预训练 token 数 n_r 满足

$$\frac{n_r}{n_a + n_r} = f \implies n_r = n_a \cdot \frac{f}{1 - f},$$

$f = 0.25$ 时系数是 $1/3$ 。代码就是这一行：

```

81 # Replay arm: alpaca tokens + a contiguous slice of the pretraining stream, sized so
82 # pretraining data is `replay_frac` of the result. Interleaving happens for free --
83 # TokenStreamDataset draws sliding windows and DistributedSampler shuffles them, so
84 # concatenating is enough; no need to shuffle documents on disk.
85 alpaca_train = encoded["train"]
86 n_alpaca = len(alpaca_train)
87 n_replay = int(n_alpaca * args.replay_frac / (1 - args.replay_frac))
88 pretrain = np.load(str(PRETRAIN_TRAIN_NPY), mmap_mode="r")
89 if n_replay > len(pretrain):
90     raise SystemExit(f"replay slice {n_replay:,} exceeds pretraining stream {len(
91 pretrain):,}")
92 replay = np.concatenate([alpaca_train, np.asarray(pretrain[:n_replay], dtype=np.int32
93 )])
94 out = OUT_DIR / "alpaca_replay_train.npy"
95 np.save(str(out), replay)
96 print(f"[sft] replay arm: {n_alpaca:,} alpaca + {n_replay:,} pretrain "
97       f"= {len(replay):,} tokens ({n_replay / len(replay):.1%} pretrain) -> {out}")

```

注意第 82–84 行注释里的取舍：不需要在磁盘上打散。TokenStreamDataset 是滑窗索引，DistributedSampler 又会 shuffle 窗口，所以简单 concatenate 出来的流在训练时已经是交错的。少写一个 shuffle，也就少一处可能写错的地方。

一条必须自己说出来的声明。cs336_basics 的 cross_entropy 没有 loss masking——它没有 ignore_index，也不接受 mask，直接对整个张量求平均（cs336-basics/cs336_basics/train/loss.py:47）。也就是说 loss 是在整篇 `###Instruction:...###Response:...` 文档上算的，包含 instruction 和 input 那部分 token，而不是只在 response 上算。`

严格说，这不是教科书意义上的 SFT，而是在指令数据上做 domain-adaptive continued pre-training。这句话写在 build_sft_corpus.py 的模块 docstring 里（cs336_systems/data_harvest/build_sft_corpus.py:18），不是事后补的免责声明。

Checkpoint 2.6 (not_really_sft): 为什么这不能叫 SFT

不看代码回答：

- 教科书 SFT 在算 loss 时会对哪些 token 做 mask？不做这个 mask，模型实际在学什么额外的东西？
- 这个差别会让 instruction 验证集上的 loss 偏高还是偏低？为什么它不影响 arm 之间的排序？
- 要在这个 codebase 里补上 loss masking，最小的改动是什么？为什么这次没做？

Hint 去看 cross_entropy 的签名，数一数它接受几个参数。

参考答案: (a) 教科书 SFT 只在 response 的 token 上算 loss, 把 prompt (instruction 加 input) 那部分 mask 掉。不 mask 的话, 模型同时在学「怎么生成一条 Alpaca 风格的指令」——而那不是我们想要的能力, 推理时指令是用户给的, 不需要模型自己编。

(b) 偏低: instruction 那部分文本模式性很强 (### *Instruction:* 这类标记几乎是确定的), 拉低了平均。不影响排序, 因为六个 arm 用的是同一个 loss 定义、同一批验证数据、同一组 batch, 是配对比较; 一个共享的偏置会整体平移所有 arm 的数字, 不改变谁比谁好。

(c) 最小改动: 在 build_sft_corpus.py 里除了 token 流再写一条等长的 mask 数组, 标出 ### *Response:* 之后的位置; 然后给 cross_entropy 加一个可选 mask 参数, 默认 None 时行为不变, 测试基线不动。没做的原因是它会改到 cs336_basics 的公共 loss 函数——那是所有测试和整个训练路径共用的, 在一个只剩几小时 GPU 预算的窗口里改它, 风险大于收益。改成如实声明, 比改代码便宜, 而且诚实。

代码位置: cs336-basics/cs336_basics/train/loss.py:17

这一章造出来的东西, 下一章会被拿去烧七个小时的 GPU。在那之前值得记住的只有一句: 语料构造脚本里每一个「为了让数字好看」的动作, 都会在下游变成一个你解释不了的数字。

real_pretrain_config.json 字段	一句话	本章
_NOTE_approx_param_count	untied head + SwiGLU 三矩阵，参数量算错两次	§3.1
_NOTE_tokenizer_swap	自训 BPE 要 20 小时，换 tiktoken 花 92 秒	§3.2
_NOTE_batch_size_oom	漏算 logits 张量的 6.6GB	§3.3
_NOTE_divergence_fix	第一次诊断：缺 LR warmup（诊断错了）	§3.4
_NOTE_ffn_init_bug	真根因：FFN 三个矩阵按 std=1.0 初始化	§3.4
_NOTE_eval_frequency	全程只做约 5 次 eval，以及那条曲线的问题	§3.6
_NOTE_wandb_artifact_hang	2.3GB artifact 上传撞上 NCCL 600 秒看门狗	§3.7
_NOTE_max_iters	30000 步怎么定出来的，以及 token/param 的取舍	§3.8

Table 6: 配置文件里的八条笔记与本章小节的对应。把失败写回下一个要改这个文件的人一定会读到的地方，比写进某个 markdown 有用。

3 预训练：0.19B / 492M token，以及一次诊断错误

难的不是写训练循环。训练循环三个月前就写好了，Triton FlashAttention kernel 和自研 bucketed DDP 也早就跑通过。这一章真正的题目是另一个：

在一个固定 7 小时、两张 3090、无人值守的预算里，怎么保证这 7 小时不是在乘 NaN。

这个约束会把很多平时无所谓的东西变成阻塞项。一个只在第 250 步之后才发作的初始化 bug，平时是「跑一次重跑一次」；在这里它是「烧掉半个预算」。一个只写进 W&B 而不打到 stdout 的 loss，平时是风格问题；在这里它意味着你 SSH 上去 tail 日志，看到的只有一根 tqdm 进度条，而模型其实已经发散了 300 步。

real_pretrain_config.json 里有八个 _NOTE_* 字段，每一条都是一次这样的教训被写回配置文件。本章按主题把它们展开：

以下 FlashDDP_runner.py 一律指 cs336_systems/Parallelization/FlashDDP/FlashDDP_runner.py。

3.1 模型定型：两次把参数量算错

用户给的区间是 50–200M 参数，要求「把 24GB 显存吃满」。定型过程反复了两轮，两次都是被真实日志打脸的。

第一次按「embedding 和 output head 共享权重 (tied) + FFN 两个矩阵」估，选了 num_layers=12, d_model=1024, d_ff=3584。启动之后日志直接打出 modelinitializedwith0.29Bparameters——比计划的 190M 多了一半。两个假设都错：

- (1) TransformerLM.in_embedding (cs336-basics/cs336_basics/lm.py:152) 和 TransformerLM.head(cs336-basics/cs336_basics/lm.py:180) 是两个独立矩阵，没有 tie。词表 50257、 $d = 768$ 时，这一项就是 $2Vd = 77.2\text{M}$ ——占最终模型的 40%。
- (2) FFN 是 SwiGLU，三个矩阵 (w_1/w_3 升维， w_2 降维，cs336-basics/cs336_basics/transformer/pointwise_ffn.py:22)，不是普通 FFN 的两个。

正确的公式：

$$N = L(4d^2 + 3dd_{\text{ff}}) + 2Vd + (2L + 1)d,$$

其中最后一项是 RMSNorm 的 gain 向量 (每个 block 两个、最后一个，cs336-basics/cs336_basics/transformer/transformer.py:72)。重算之后选了 $L = 16$ 、 $d = 768$ 、12 头、 $d_{\text{ff}} = 2048$ (正好是 SwiGLU 常用的 $8d/3$)，日志确认 0.19B。

参数组	公式	数量
每层 attention ($W_Q/W_K/W_V/W_O$)	$4d^2$	2,359,296
每层 SwiGLU FFN ($W_1/W_2/W_3$)	$3 d d_{ff}$	4,718,592
每层小计		7,077,888
16 层	$16 \times 7,077,888$	113,246,208
in_embedding	Vd	38,597,376
head (untied, 独立矩阵)	Vd	38,597,376
RMSNorm gains	$(2 \cdot 16 + 1) \times 768$	25,344
合计		190,466,304

Table 7: $L = 16$ 、 $d = 768$ 、 $d_{ff} = 2048$ 、 $V = 50257$ 、 $context_length=1024$ 下的参数量拆解。日志打印的是 0.19B。配置文件的 `_NOTE_approx_param_count` 给的公式漏掉了 RMSNorm 那一项，得 190,440,960，差 0.013%——可以忽略，但要知道它被忽略了。

Checkpoint 3.1 (param_count_formula): 参数量公式里最容易漏的两项

不看代码回答：

- 写出这个模型参数量的公式。哪两个结构假设一旦搞错，会让估计偏离 50% 以上？
- V 从 10000 涨到 50257，模型总参数量涨多少？涨的那部分全部集中在哪里？
- 如果把 embedding 和 head tie 起来，能省多少？为什么这个项目没有 tie？

Hint 不要从论文里回忆公式，去数 TransformerLM.__init__ 里创建了几个 Parameter。

参考答案：(a) $N = L(4d^2 + 3 d d_{ff}) + 2Vd + (2L + 1)d$ 。两个致命假设：head 是否与 embedding 共享权重（决定 $2Vd$ 还是 Vd ），以及 FFN 是两矩阵还是 SwiGLU 三矩阵（决定 $2 d d_{ff}$ 还是 $3 d d_{ff}$ ）。这次两个都猜错了，日志报 0.29B 而不是计划的 190M。

(b) $2\Delta Vd = 2 \times 40257 \times 768 \approx 61.8M$ ，全部集中在 embedding 和 head 两个矩阵上，Transformer 层一个参数都没变。这也解释了为什么这个模型 40% 的参数在做「查表和投影」而不是在做计算。

(c) 省 $Vd \approx 38.6M$ （约 20%）。没有 tie，因为 lm.py 里这两个就是分开声明的两个模块（Embedding 和 Linear），tie 需要改模型结构；而这次冲刺的原则是不动已经通过测试的模型代码。代价是记住它没 tie，别在估参数量时按 tied 算——这正是踩过的坑。

代码位置：cs336-basics/cs336_basics/lm.py:152 与 cs336-basics/cs336_basics/lm.py:180

3.2 tokenizer 换掉：BPE merge 的单步成本由什么决定

项目里有一个自己写的 BPE tokenizer，在 TinyStories 上训 vocab=10000 大约半小时跑完。换成真实混合语料（web + wiki + instruction）之后，merge 阶段的速度掉到了完全不可接受的程度。

日志里的实测（run_logs/master_pipeline.log，全量语料）：

```
Merging BPE pairs: 1%| | 100/9743 [14:48<21:57:35, 8.20s/merge]
Merging BPE pairs: 1%| | 104/9743 [15:19<20:43:16, 7.74s/merge]
```

约 7.7–8.7 秒一个 merge，tqdm 给出的 ETA 在 21–23 小时之间。配置文件的 `_NOTE_tokenizer_swap` 记的是「约 7.5 s/merge、约 20 小时」——方向对，但比日志略乐观，以日志为准。9743 次 merge（目标 vocab 10000 减去初始的字节级基元和特殊 token）会把整个时间预算全砸在训 tokenizer 上。

第一反应是「语料太大，采样一部分」。按 4% 比例采样重跑，日志里是 (run_logs/tokenizer_sample_train.log):

```
Merging BPE pairs: 0% | | 1/9743 [00:01<3:59:57, 1.48s/merge]
Merging BPE pairs: 1% | | 67/9743 [01:14<2:40:33, 1.00merge/s]
```

约 1.0–1.5 秒一个 merge，ETA 仍有 2.6–4.0 小时。语料缩到 1/25，单步成本只降到 1/6。这个不成比例的关系就是这一节真正的知识点：

BPE 每一步 merge 的成本，由词汇多样性 (distinct pair 的数量、pretoken 表的规模) 决定，不是由原始字节数决定。

TinyStories 是刻意用受限词表写的儿童故事，distinct pair 的数量小得多；换成 C4 加 Wikipedia，同样的 merge 循环要在一个大出一两个数量级的 pair 表上找最大值并更新。按比例采样只能线性地削掉「重复出现的常见 pair」的计数工作量，削不掉 pair 表本身的多样性。

解法不是优化那个循环，是不做这件事：换 tiktoken.get_encoding("gpt2") (50257 词表，本来就是项目依赖)。实测 (run_logs/tiktoken_encode.log):

```
[encode] train: 780,002,594 chars -> 173,972,065 tokens in 86.0s
[encode] valid: 20,012,169 chars -> 4,516,054 tokens in 5.9s
```

92 秒，对 20 小时。代价是 vocab.pkl / merges.pkl 现在是占位文件 (flashddp 训练路径不读它们的内容，只有 stage_train_tokenizer 的 exists-check 会看)，以及 cs336_basics/text_gen.py 从此用不了——它要 Tokenizer.from_files(vocab.pkl, merges.pkl)，所以另写了 sample_from_checkpoint.py。换来的是把省下的 20 小时全部花在真正的训练上。

Checkpoint 3.2 (bpe_merge_cost): 为什么采样到 4% 只快了 6 倍

不看代码回答：

- 同一份 BPE 实现，TinyStories 上 vocab=10000 半小时跑完，真实混合语料上一个 merge 就要 8 秒。语料大小之外，还有什么变了？
- 按 4% 采样，语料缩到 1/25，单步成本只降到约 1/6。这个不成比例说明成本主要花在什么上？
- 什么情况下「训一个自己的 tokenizer」仍然值得？（提示：想想这次换成 gpt2 词表之后，§3.3 里那个 OOM 是怎么来的。）

Hint 一次 merge 要做的事是：在 pair 计数表里找最大值、合并、更新受影响的 pretoken。哪一步和语料字节数成正比，哪一步和 distinct pair 数量成正比？

参考答案：(a) 词汇多样性。TinyStories 是受限词表的儿童故事，distinct pretoken 和 distinct pair 都少一两个数量级；C4 加 Wikipedia 有拼写变体、专名、数字、多语言残留、URL 片段。每一步 merge 都要扫这张表。

(b) 主要花在维护和扫描 pair 表上，而不是扫语料字节。按比例采样削掉的是常见 pair 的计数（和字节数成正比的那部分），削不掉表的基数——罕见 pair 大多在 4% 的样本里也还留着至少一个实例。

(c) 目标语域和现成 tokenizer 差很远的时候：非英语、代码、化学式、DNA。这次不值得，因为语料本来就是英文 web 文本，gpt2 词表就是在这个分布上训的。代价也是真的：50257 的词表把 2v4 撑到 77M 参数，而且直接引出了 §3.3 的 OOM——换掉 tokenizer 不是一个局部决定。

代 码 位 置： cs336_systems/experiments/real_pretrain_config.json:65
(_NOTE_tokenizer_swap)

3.3 OOM: 漏算的那个 logits 张量

tr_batch_size=32 是从 vocab=10000 那版计划里带过来的。换到 50257 词表之后，两张卡同时炸：

```
torch.OutOfMemoryError: CUDA out of memory. Tried to allocate 96.00 MiB.
GPU 1 has a total capacity of 23.56 GiB of which 25.00 MiB is free.
```

漏算的是前向最后那个 logits 张量：

$$\underbrace{32}_{\text{batch}} \times \underbrace{1024}_{\text{ctx}} \times \underbrace{50257}_V \times \underbrace{4}_{\text{fp32}} = 6,587,285,504 \text{ B} \approx 6.6 \text{ GB},$$

这还没算 cross-entropy 里 softmax 和反向的中间 buffer。关键在于这一项和 V 成正比：词表从 10000 涨到 50257，别的都没动，光这一项就翻了 5 倍。在一张 24GB 卡上，6.6GB 只是 logits 一个张量，模型权重加 AdamW 一二阶矩 (fp32, 190M 参数约 2.3GB) 还要单算。

降到 8 之后稳定运行，配置里保守地停在这里没往上试 (_NOTE_batch_size_oom 说 16 大概也能跑，但无人值守的多小时任务不值得为这点吞吐冒险)。

Checkpoint 3.3 (logits_oom): 显存预算里最容易漏的一项

不看代码回答：

- (a) 一个 L 层、 d 维、词表 V 、batch B 、上下文 T 的模型，前向过程中单个最大的激活张量通常是哪个？写出它的大小。
- (b) 这一项和 L 有关吗？和 d 有关吗？这意味着「加层」和「扩词表」哪个对显存更危险？
- (c) 这次把 batch 从 32 降到 8 解决了问题。还有哪两种不降 batch 的解法？它们各自的代价是什么？

Hint 别去数 attention 的 BHT^2 ——这个模型用的是 Triton FlashAttention，那一项根本没有物化。

参考答案：(a) 输出 logits: $BTV \times 4$ 字节 (fp32)。这里是 $32 \times 1024 \times 50257 \times 4 \approx 6.6 \text{ GB}$ 。(b) 和 L 、 d 都无关。所以扩词表更危险：加一层只增加约 7.1M 参数和对应的激活，而词表翻 5 倍会让这个张量直接翻 5 倍，且不受任何 checkpointing 策略保护（它是最后一层的输出，必须留着算 loss）。(c) 其一，把 head 和 loss 融合 (chunked / fused cross-entropy)，不物化完整 logits，只按块算——省显存但要改 loss 实现；其二，混合精度 (bf16 激活) 把这一项减半——但这次全程 fp32，改动会波及数值稳定性，而当时正在追一个 NaN (§3.4)，不能同时动两个变量。

代码位置： cs336_systems/experiments/real_pretrain_config.json:69 (_NOTE_batch_size_oom)

3.4 NaN，以及一次错误的诊断

这一节是本章的重点。它值得讲的不是「最后修好了」，而是中间那个看起来非常合理、证据也齐全、但就是错的假设，是被哪一个具体事实推翻的。六步，顺序不能倒。

第一步：症状

第一次正式标定跑，lr 从 step 0 起固定 6×10^{-4} ：

```
val loss 6.90 (step 100) → 6.60 (step 200) → nan (step 300+)
```

前两个点在正常下降，第三个点直接 nan。

第二步：错误假设——缺 LR warmup

当时的判定是「没有 LR warmup」。这个假设在当时看起来非常成立，因为有三条独立的证据同时指向它：

- (1) 代码里确实缺。cs336_basics/train/optimizer.py 里的 lr_scheduler (linear warmup + cosine decay) 一直存在，但只接在单卡 local_train 路径上；flashddp 这条路径从来没调用过它，确实是 step 0 就直接上满 lr。
- (2) lr 的取值确实不对。 6×10^{-4} 是 GPT-2 small 在约 0.5M tokens/batch 下的取值，而这次是 2 卡 $\times$ batch 8 $\times$ 1024 ctx = 16K tokens/batch，小了约 30 倍。同一个 lr 配一个小 30 倍的 batch，梯度噪声大得多。
- (3) 症状形状对得上。「训几百步、loss 正常下降、然后突然 NaN」是学习率过大导致发散的教科书形状。

于是接上了 warmup、把峰值 lr 降到 3×10^{-4} 。这两个改动本身都是对的，也都保留了下来 (grad_clip=1.0 全程开着，同样没能救回来)。

常见误解：降低学习率并加上 warmup 之后还是炸，说明学习率降得还不够。

这是当时最容易走上的那条岔路，也是最贵的一条——每试一个更低的 lr 就要烧掉几十分钟 GPU，而且每次失败都会强化「再降一点就好了」的直觉。真正终结这条路的不是又试了一个 lr，是去看第二次是在什么 lr 上炸的。

第三步：否认

改完之后重跑，NaN 从 step 约 250 推迟到了 step 约 820。看起来「有改善」。但把当时的 lr 算出来之后，这个「改善」变成了否认：

第一次正式 run 的 warmup_iters=963、峰值 3×10^{-4} ，采用 linear warmup，所以 step 820 时的学习率是

$$3 \times 10^{-4} \times \frac{820}{963} \approx 2.55 \times 10^{-4},$$

还在爬坡段，从没到过峰值。也就是说：

第一次在 6×10^{-4} 上炸，第二次在 2.55×10^{-4} 上炸。学习率降了 2.4 倍，模型还是炸了。

一个真正由学习率过大引起的发散，不会在一个更小的学习率上重现。这一个事实就足够把「LR 假说」判死——不需要再试第三个 lr，不需要 loss 曲线，不需要梯度范数。假设空间从 {LR、初始化、数据、数值实现} 塌缩到剩下几个，而其中只有初始化能解释「和 lr 无关地在几百步后爆炸」。

Example 3.1 (refuting_the_lr_hypothesis): 把假设空间从四个砍到一个

假设空间（第一次 NaN 之后）

- H1. 学习率过大 / 缺 warmup
- H2. 权重初始化有问题
- H3. 数据里有异常值（超长 token id、越界索引、编码错误）
- H4. 数值实现问题（Triton kernel、混合精度、loss 里的 logsumexp）

证据 1：三条独立线索都支持 H1（调度器没接上、lr 取自 30 倍大的 batch 规模、症状形状

匹配)。行动：接 warmup, $\text{lr } 6 \times 10^{-4} \rightarrow 3 \times 10^{-4}$ 。

证据 2 (决定性的那一条): NaN 从 step 250 移到 step 820, 而 step 820 时 lr 只有 2.55×10^{-4} 。推理：若 H1 为真, 在更小的 lr 下发散应当更晚或不发生, 而不是仍然发生。观察到的是「在更低的 lr 上照炸」, 因此 H1 被否定。注意这里不是「H1 解释力不够」, 是「H1 预测了一个没发生的事」。代价：一次约 12 分钟的 run, 加上算一次 warmup 的 lr。

排除 H3: 如果数据里有异常值, 发散时刻应当和数据顺序绑定 (换 seed 会换发散步数), 而且 step 0 的 loss 不会恰好等于 $\ln V$ 。两次发散的 step 数差了三倍多, 且初始 loss 一直是 10.82, 不像数据问题。

排除 H4: Triton kernel 和 loss 实现都有测试, 而且第一次 NaN 之前 loss 正常下降了两百多步——一个数值实现 bug 通常在第一步就露馅, 不会等到第 250 步。

剩下 **H2**, 于是不再猜, 直接去读初始化代码。从「决定去读代码」到「找到 bug」花了几分钟——真正贵的是前面那两次 run, 而不是最后那次阅读。

第四步：真根因

去读初始化代码, 问题在一个默认参数上。工具函数的签名是：

```

6 def trunc_normal_para_init(dim_out, dim_in, mu=0, var=1, trunc_magnitude=3, device="cpu",
7 dtype=torch.float16):
8     """
9     Return the initialized parameters value using truncated normal.
10    """
11    device = device if device is not None else "cpu"
12    dtype = dtype if dtype is not None else torch.float16
13    tensor = torch.empty(dim_out, dim_in, dtype = dtype, device = device)
14    weight_mat: Float[tensor.Tensor, "dim_out, dim_in"] = tensor
15    nn.init.trunc_normal_(
16        weight_mat, mean=mu, std=math.sqrt(var),
17        a=mu-trunc_magnitude, b=mu+trunc_magnitude,
18    )
19    return weight_mat

```

var 的默认值是 1。而 SwiGLU FFN 里那三次调用都没有传 var, 于是 $w_1/w_2/w_3$ 全部按 $\sigma = 1.0$ 初始化。模型里其他所有矩阵 (Linear、 $w_Q/w_K/w_V/w_O$) 都老老实实传了 Xavier 的 $2/(d_{\text{out}} + d_{\text{in}})$ 。对 FFN 该有的值是

$$\sigma_{\text{FFN}} = \sqrt{\frac{2}{d_{\text{ff}} + d}} = \sqrt{\frac{2}{2048 + 768}} \approx 0.0266,$$

也就是说实际用的标准差比应该用的大了约 37 倍。修复就是把这个方差显式传进去：

```

13     # Xavier variance, matching Linear (linear_module.py) and the attention
14     projections
15     # (multiheads_attention.py). Passing it explicitly matters:
16     trunc_normal_para_init's
17     # `var` defaults to 1, and these three calls used to omit it, so every FFN matrix
18     was
19     # initialized at std=1.0 instead of ~0.027. Pre-norm hides that at step 0 (
20     RMSNorm
21     # renormalizes, so initial loss is still exactly ln(vocab_size)), but the FFN
22     branch
23     # writes values with RMS ~2.5e4 into a residual stream whose branches should be
24     # contributing ~0.4 -- a ~65,000x overshoot that reliably diverges to NaN a few
25     # hundred steps in, once the LR is high enough to actually move those weights.
26     ffn_var = 2 / (dim_ff + dim_model)

```

```

22     self.W1: Float[torch.Tensor, "dim_ff, d_model"] = torch.nn.Parameter(
        trunct_normal_para_init(dim_ff, dim_model, 0, ffn_var, 3, device, dtype))
23     self.W3: Float[torch.Tensor, "dim_ff, d_model"] = torch.nn.Parameter(
        trunct_normal_para_init(dim_ff, dim_model, 0, ffn_var, 3, device, dtype))
24     self.W2: Float[torch.Tensor, "d_model, dim_ff"] = torch.nn.Parameter(
        trunct_normal_para_init(dim_model, dim_ff, 0, ffn_var, 3, device, dtype))

```

第五步：为什么一直没被发现

这才是这个 bug 真正有意思的地方：pre-norm 架构把它藏住了。

每个 block 的输入先过 RMSNorm 再进分支，所以不管 FFN 的权重多大，下一个 block 拿到的东西都会被重新归一化；最后一层之后还有一个 self.norm，然后才进 head。于是 step 0 的 loss 恰好还是均匀分布的交叉熵：

$$\ln(50257) = 10.8248,$$

日志里第 20 步打出来的是 train loss 10.7412——完全正常。前几百步 loss 也在正常下降。日志上什么都看不出来。

真正被破坏的是残差流的尺度。实测（拿纯 torch 复算，见下）FFN 分支往残差流里写的值 RMS 约 2.5×10^4 ，而正常情况下各分支应该贡献约 0.37——约 6.5 万倍的超调。RMSNorm 会把这个尺度在下一层的入口重新压回来，所以前向数值不会立刻溢出；但只要 lr 大到能真正推动这些权重，AdamW 的更新就会在这个被放大了四个数量级的分支上迅速失控。这解释了两件事：为什么「降 lr + 加 warmup」只能推迟而不能阻止，以及为什么第二次是在更低的 lr 上炸的（warmup 让权重被推动得更慢，所以晚了几百步，仅此而已）。

Checkpoint 3.4 (prenorm_hides_init_bug): pre-norm 帮了什么倒忙

不看代码回答：

- (a) 三个 FFN 矩阵按 $\sigma = 1.0$ 初始化（该是 0.0266），为什么 step 0 的 loss 仍然精确等于 $\ln V$ ？
- (b) 如果这是一个 post-norm 模型，这个 bug 会在第几步暴露？为什么？
- (c) 「初始 loss 等于 $\ln V$ 」这个健康检查，能证明什么、不能证明什么？

Hint 顺着一个 token 的向量从 embedding 走到 head，数一数中间经过几次 RMSNorm，以及最后一次 RMSNorm 在 head 之前还是之后。

参考答案：(a) 因为 head 之前有一个 self.norm (cs336-basics/cs336_basics/lm.py:177)。不管残差流被 FFN 撑到多大，进 head 之前都会被归一化回单位尺度；而 head 自己的权重是正确的 Xavier 初始化，所以输出 logits 近似零均值小方差，softmax 近似均匀分布，loss 就是 $\ln V$ 。这个检查测的是最后一层归一化之后的东西，对之前发生了什么免疫。

(b) post-norm 里归一化在残差加法之后，FFN 分支的输出会直接进入下一层的主干而不是先被压回去，尺度会逐层累乘——很可能第一次前向就 inf 或者 loss 明显异常。也就是说 post-norm 会更早报错，而 pre-norm 的稳定性在这里反过来变成了掩盖能力。

(c) 能证明：embedding 和 head 的初始化大致合理、loss 函数没写反、词表大小和数据对得上。不能证明：中间任何一层的初始化、残差流的尺度、梯度的量级。要覆盖后者，得直接量每类矩阵的 std 和各分支写进残差流的 RMS——这正是下面那个 Tip 要做的事。

代码位置：cs336-basics/cs336_basics/transfromer/pointwise_ffn.py:13

Checkpoint 3.5 (rms_overshoot): 那个 65000 倍是怎么来的

不看代码回答：

- (a) FFN 分支往残差流里写的 RMS 约 2.5×10^4 ，正常应该约 0.37。这两个数是推导出来的还是量出来的？这个区别为什么要在讲义里写明？
- (b) 既然前向数值没有溢出（loss 一直是有限的），为什么这个超调最终一定会导致 NaN？
- (c)（开放）你能不能解析地估出 2.5×10^4 这个量级？

Hint (b) 想想 AdamW 的更新量和梯度大小的关系，以及一个被放大了四个数量级的分支在反向里会产生多大的梯度。

参考答案: (a) 量出来的——用纯 torch 复算：按两种方差各初始化一次 FFN，喂一个 RMSNorm 之后的输入，直接读输出张量的 RMS。不是解析推导。写明这一点是因为「 2.5×10^4 」看起来很像一个从公式里落出来的数，而它不是；一个读者如果照着某个式子复现不出这个数，应该怀疑那个式子而不是怀疑自己。（顺带一提：pointwise_ffn.py 的注释把正常值写成约 0.4，配置文件的 note 写的是 0.37，两处是同一个实测量的不同取整。）

(b) 因为 pre-norm 只在前向把尺度压回去，不改变权重本身有多大。反向传播经过这个分支时梯度同样被放大，AdamW 的分母（二阶矩）虽然会做归一化，但一旦这些权重开始移动，被放大四个数量级的分支输出会以同样的比例变化，残差流的相对结构被瞬间破坏。grad_clip=1.0 只限制全局范数，不能修复「某一个分支的尺度本身就错了四个数量级」这件事——实测它确实没救回来。

(c) 留作开放小问，我们没有做这个推导。要做的话，需要的量都在手边：RMSNorm 之后输入每个分量的尺度、 $\sigma = 1$ 时 $W_1 x$ 的 fan-in 放大、SiLU 在大输入上的近似行为、门控那一步是两个都被放大过的量相乘（这一步最容易被低估），以及 W_2 再做一次投影。如果你推出来了，拿它和实测的 2.5×10^4 比一下：差在一个数量级以内就说明你的心智模型是对的；差得更多，多半是漏掉了门控那一次相乘。

代码位置： cs336_systems/experiments/real_pretrain_config.json:63
(_NOTE_ffn_init_bug)

Checkpoint 3.6 (lr_hypothesis_refuted): 哪一个具体事实否掉了 LR 假说

不看代码回答：

- (a) 「缺 warmup」这个假设当时有三条证据支持。加上 warmup 并把 lr 降到 3×10^{-4} 之后，NaN 从 step 250 推迟到 step 820。哪一个具体事实让你判定这个假设是错的？请给出一个数。
- (b) 为什么「NaN 推迟了」不能算作假设成立的证据？
- (c) 如果当时 warmup_iters 设成 100（而不是 963），step 820 时 lr 已经过了峰值并开始 cosine 衰减。这会不会让这次否认失效？

Hint 去算 step 820 那一刻的学习率，然后和第一次发散时的 6×10^{-4} 比。

参考答案: (a) 第二次发散发生在 2.55×10^{-4} 这个学习率上（ $3 \times 10^{-4} \times 820/963$ ，warmup 还没爬到峰值）。第一次是在 6×10^{-4} 上炸的。学习率降了 2.4 倍，发散照样发生——一个由学习率过大引起的现象不会在更小的学习率下重现。这一条是决定性的；「推迟了」不是。

(b) 因为「推迟」和两个假设都相容。若根因是初始化，降低 lr 会让那些坏权重被推动得更慢，发散自然更晚——推迟恰恰是初始化假设也预测的。一个不能区分两个假设的观察，对判定这两个假设中的哪一个为真没有信息量。真正有信息量的是「在更低的 lr 上仍然炸」这个方向性事实。

(c) 会削弱。如果 step 820 时 lr 已经衰减到比 6×10^{-4} 更高或相当的值，就没法说「在更低的 lr 上炸」。这次否认之所以干净，是因为 warmup 长（963 步）恰好保证了发散点落在爬坡段，lr 明确低于第一次。这是运气，不是设计——一个更稳的做法是发散时把当时的 lr 一起打进异常信息里，这正是第六步 fail-fast 做的事。

代码位置: cs336_systems/experiments/real_pretrain_config.json:66
(_NOTE_divergence_fix)

第六步：修复、验证，以及两道防止重演的闸

修复本身是三行。验证做了三件事：FNN.W1 的实测 std 对上 Xavier 目标（差 0.4%）、初始 loss 等于 $\ln V$ 、global grad norm 有限（2.15），前反向全部有限。另外确认了 tests/ 里没有任何测试依赖 FFN 的随机初始化（adapter 都是喂固定权重进去的），所以这个改动不动测试基线。

比修复更重要的是两道闸，因为它们保证下一次不会再瞒这么久：

```

507         # Fail fast on divergence. Without this the job happily burns its whole
508         # wall-clock budget multiplying NaNs, and nothing in stdout says so until the
509         # next eval -- which can be thousands of steps away.
510         if not math.isfinite(loss_value):
511             raise RuntimeError(
512                 f"Rank {rank}: training DIVERGED -- non-finite loss ({loss_value}) at
513                 "
514                 f"global_step {global_step} (lr={current_lr if use_lr_schedule else
515                 peak_lr:.3e}). "
516                 f"Aborting instead of burning the rest of the budget."
517             )
518         if log_every_n_steps is not None and global_step % log_every_n_steps == 0:
519             # Also to stdout, not just W&B: the run log is the only thing available
520             when
521             # reconnecting over SSH, and previously it showed nothing but a tqdm bar.
522             lr_str = f" | lr {current_lr:.3e}" if use_lr_schedule else ""
523             print(f"Rank {rank} | step {global_step:06d} | train loss {loss_value:.4f}
524             |{lr_str}", flush=True)

```

第一道是 `math.isfinite(loss_value):` 一旦非有限立刻 `raise`，并把当时的 `global_step` 和 `lr` 一起打出来——而不是继续拿 NaN 乘 7 小时。

第二道是第 521 行那个 `print(..., flush=True):` 训练 loss 从此同时打到 stdout，不只进 W&B。这条看起来最不起眼的改动，恰恰是整件事的关键——第二次发散能瞒到 step 1100 才被发现，就是因为当时 loss 只进 W&B，SSH 上 tail 日志只看得到一根 tqdm 进度条，最后是去查 W&B API 才发现的。一个无人值守的任务，它的健康状态必须出现在你最可能去看的那个地方。

Downscaling Tip: 在笔记本上验初始化，不需要 GPU

这个 bug 完全可以在 Day 0 用一段不到二十行的 CPU 代码抓出来，代价约一分钟，能省下 7 小时 GPU 和两次发散。做法：

1. 用小配置构造一个 TransformerLM (`d_model=128` 之类，结构和大模型一样就行)；
2. 遍历 `named_parameters()`，对每个二维矩阵打印它的 `std()`，以及它和 $\sqrt{2/(\text{fan_out} + \text{fan_in})}$ 的比值；

3. 看那一列比值。所有正常矩阵应该在 1.0 附近；这个 bug 下 $w_1/w_2/w_3$ 会是 30–40。

打比值而不是打 std 本身很关键：不同矩阵的 fan-in/fan-out 不同，std 的绝对值本来就该不一样，一列参差不齐的数字看不出问题；一列本该全是 1.0 的数字里混进三个 37，一眼就能看见。

顺带再打一次前向的 loss（应该等于 $\ln V$ ）和一次反向的 global grad norm，就覆盖了本章两个最贵的失败模式。

Resource requirements: CPU 或 MPS 即可，约一分钟。注意这条路径以前是走不通的——cs336_basics 的 @nvtx.range 注解在非 CUDA build 上一调用就炸，正是它当时挡住了这个本地检查；修法见 §3.7。

3.5 _orig_mod. 前缀：影响面比看起来大得多

训练跑到 step 7500 出第一个中途 checkpoint 时，顺手拿采样脚本做了个冒烟测试，当场炸出来：

```
RuntimeError: Error(s) in loading state_dict for TransformerLM:
  Missing key(s) in state_dict: "in_embedding.embed_mat", ...
  Unexpected key(s) in state_dict: "_orig_mod.in_embedding.embed_mat", ...
```

根因：compile: true 时 torch.compile 把模块包成 OptimizedModule，它的 state_dict() 会给每个 key 加上 _orig_mod. 前缀；而保存端只 unwrap 了 DDP 那一层 (.module)，没 unwrap compile 这一层。

影响面。这里最该学的不是修法，是正确评估影响面。「采样脚本跑不了」听起来像一个可以放到最后处理的小麻烦，实际上同一个前缀会让：

- load_ddp_checkpoint（断点续训）一样失败；
- Day 3–4 的 serving adapter 加载这个 checkpoint 时必然踩到 (§5)；
- eval_checkpoint.py 对所有 arm 打分时全部失败。

也就是说：这次训练的产出，对下游全是不可用的。一个「采样脚本的小问题」实际上等价于「7 小时白训」。

修法（两端都改）。保存端 unwrap，让以后的 checkpoint 直接是干净的 plain key：

```
115     if rank == 0:
116         os.makedirs(checkpoint_dir, exist_ok=True)
117         ckpt_path = os.path.join(checkpoint_dir, f"checkpoint_{tag}_{epoch:04d}.pt")
118         raw_model = model.module if hasattr(model, "module") else model
119         # Unwrap torch.compile too, so checkpoints are written with plain module keys and
120         # stay loadable by anything that builds a bare TransformerLM (the sampler, the
121         # serving adapter). Otherwise every key carries an "_orig_mod." prefix.
122         raw_model = getattr(raw_model, "_orig_mod", raw_model)
```

加载端统一做一次 k.removeprefix("_orig_mod."), 对没有前缀的 checkpoint 是 no-op——三处都改了：cs336_systems/Parallelization/FlashDDP/FlashDDP_runner.py:175（续训）、cs336_systems/Parallelization/FlashDDP/FlashDDP_runner.py:351（init_from）、cs336_systems/experiments/eval_checkpoint.py:73（打分）。

正因为加载端做了兼容，正在跑的那个 run 没有为此重启——已经写出去的带前缀 checkpoint 靠加载端读得回来。这是「两端都改」相对「只改保存端」的全部价值。

Checkpoint 3.7 (orig_mod_prefix): 哪一层 wrapper 没被 unwrap

不看代码回答：

- (a) 保存时模型外面套了几层？分别是什么？save_ddp_checkpoint 原来只 unwrap 了哪一层？
- (b) 修法为什么要两端都改？只改保存端会有什么后果、只改加载端呢？
- (c) removeprefix 对一个没有前缀的 checkpoint 做了什么？这个性质为什么重要？

Hint 数 parallel_train 里对 model 做过几次赋值。

参考答案：(a) 两层：外面是 DDPOverlapBucketed (通过 .module 取内层)，里面是 torch.compile 产生的 OptimizedModule (通过 ._orig_mod 取内层, cs336_systems/Parallelization/FlashDDP/FlashDDP_runner.py:368)。原来只 unwrap 了 DDP 那一层。

(b) 只改保存端：已经写出去的那些 checkpoint (包括当时训练已经产出的) 永远读不回来，等于必须重启训练。只改加载端：checkpoint 文件本身一直带着一个和训练配置耦合的前缀，任何不知道这件事的下游 (别人的脚本、serving adapter) 都会踩；而且 compile: false 重跑时又不带前缀，schema 变得不稳定。两端都改才既救了旧文件又让新文件干净。

(c) no-op——str.removeprefix 在前缀不存在时原样返回。正因为是 no-op，加载端可以无条件调用，不需要先判断 checkpoint 是哪种；一个不需要 if 的兼容层，出错面比一个需要判断的小。

代码位置：cs336_systems/Parallelization/FlashDDP/FlashDDP_runner.py:122

Checkpoint 3.8 (why_verify_early): 为什么只花了 2 分钟就发现

不看代码回答：

- (a) 这个 bug 是在训练跑到 step 7500 (全程 25%) 时发现的，不是训练结束时。是什么安排让它这么早暴露？
- (b) 如果等到 step 30000 训完再去验证加载路径，代价是什么？把它折算成小时。
- (c) 列举另外两个「产物在中途就该验一次」的例子。(提示：这次冲刺里至少还有两个。)

Hint checkpoint_every_n_steps=7500 这个配置项存在的理由不止「防断电」一个。

参考 答 案： (a) checkpoint_every_n_steps=7500 (cs336_systems/experiments/real_pretrain_config.json:49) 让训练在 25% 处就产出一个真 checkpoint，然后特意拿采样脚本对它跑了一次加载。关键动作是「产出第一个中途产物之后立刻验证下游能不能吃它」，而不是「等全部跑完再说」。

(b) 6.5 小时训练全部作废 (产物对下游不可用)，加上重跑的 6.5 小时——约 13 小时，而实际代价是 2 分钟的发现加上几分钟的修复，而且不需要重启正在跑的 run。

(c) 其一，标定阶段的 NaN 门禁：500 步标定跑完必须有两个有限的 val loss，否则 FATAL 中止，宁可跑也不拿没验证过的配置烧 7 小时。其二，KV cache 的正确性门禁必须拿最终权重再过一次——拿别的权重过的门禁不算这批权重的门禁 (§5)。共同的形状是：在还有便宜的退出机会时验证，而不是在唯一的退出机会已经过去之后。

代码位置：cs336_systems/experiments/real_pretrain_config.json:49

step	val loss
6000	4.4579
12000	4.2015
18000	4.0937
24000	4.0288
30000	4.0057

Table 8: 训练内的五次 eval (`run_logs/full_pretrain_run.log`)。不要把 4.0057 说成「混合验证集上的 val loss」——它测的是 `valid.txt` 开头约 1800 个 token，全部来自 C4，从头到尾没碰过 wiki 和 instruction。理由见正文。

3.6 那条 val loss 曲线，其实只测了 web 域头部约 1800 个 token

训练跑完之后才发现的，必须如实说。

三件本身都合理的事叠在一起，产生了一个不合理的结果：

- (1) `build_mixed_corpus.py` 是按域顺序写 `valid.txt` 的：先写完全部 web，再写 wiki，最后写 instruction (`cs336\_systems/data\_harvest/build\_mixed\_corpus.py:187`)，不是打散的。
- (2) `val_sampler=DistributedSampler(..., shuffle=False)` (`cs336\_systems/Parallelization/FlashDDP/FlashDDP\_runner.py:324`)，所以每个 rank 拿到的是按顺序交错的窗口下标。
- (3) `_run_eval` 用 `itertools.islice(val_iter, val_batch_num)` 取前 50 个 batch (`cs336\_systems/Parallelization/FlashDDP/FlashDDP\_runner.py:417`)。

算一下覆盖范围：`val_batch_size=8`、50 个 batch，每个 rank 取 400 个窗口；`DistributedSampler` 在 `shuffle=False` 下按 `indices[rank::world_size]` 切，所以两个 rank 合起来覆盖的窗口下标是 0 到 799；`TokenStreamDataset` 是滑窗索引，下标 i 的窗口是 token $[i, i + 1024)$ 。合起来：

每次 eval 覆盖的 token = $[0, 799 + 1024) \approx 1800$ 个 token，

也就是 `valid.txt` 最开头那几篇 C4 网页文档，每次 eval 都是同一批。

撤回 (retracted): 预训练的 val loss 曲线 4.4579 → 4.0057 是模型在混合验证集上的表现。

这条曲线仍然有意义，别一起丢掉：每次 eval 打的是完全相同的一批样本，等于天然配对，step 之间比较的方差极低，「loss 在下降」这个结论是成立的，而且比一个每次重新随机采样的曲线更干净。

不成立的是它的标签。它不是「混合验证集上的 val loss」，是「`valid.txt` 开头约 1800 个 web token 上的 val loss」。跑完之后用 `eval_checkpoint.py` (200 个随机 batch、固定 seed) 测出来的才是正经的数：general 3.5866 / web 3.8469。

没有中途去改：改成 `shuffle=True` 会让曲线前后不可比，而真正用来排序微调 arm 的本来就是 `eval_checkpoint.py` 那套方法。留给下一次 run 修：`val_sampler` 打开 `shuffle`，或者在 `_run_eval` 里对 `val_batch_num` 做随机采样而不是取前 N 个。

三个数字的大小关系完全自洽，这本身就是这个解释成立的一个旁证：

顺带：为什么全程只做 5 次 eval。`eval_every_n_steps=6000` 是故意设稀的 (`_NOTE_eval_frequency`)。每次 eval 本身就是验证流上的一次采样估计，自带误差棒；把频率提高十倍买到的是噪声不是信号，只是白烧 GPU。标定阶段例外，那时在 step 250 加一次，纯粹是为了有两个 val 数可以做 NaN 门禁。

测的是什么	规模	loss
训练内 val (valid.txt 开头约 1800 token, 全是 C4)	约 1.8K token	4.0057
domain_valid_web (C4 validation, 200 个随机 batch)	267K token	3.8469
mixed_valid (三域混合, 200 个随机 batch)	4.52M token	3.5866

Table 9: 窄 web 切片 > 完整 web > 三域混合。窄切片最难（几篇特定文档，运气成分大）；混合最容易（20% 的 Wikipedia 和 10% 的 instruction 都比 web 好预测，把平均拉低了）。

这里有个值得记的对照：微调那边反而把 eval 从每 300 步加密到每 100 步，因为微调的收益几乎全在前 100 步、之后单调过拟合，3 个点根本看不出那个拐点。同一个「eval 频率」参数，在两个阶段的正确方向相反——决定它的是你想看的现象的时间尺度，不是什么通用最佳实践。

Checkpoint 3.9 (val_curve_is_not_what_it_says): 三个合理的决定叠出一个不合理的结果

不看代码回答：

- (a) 三件事各自都有正当理由：按域顺序写 valid.txt、shuffle=False、islice 取前 50 个 batch。分别说出它们的理由，再说出叠加之后发生了什么。
- (b) 算出每次 eval 实际覆盖了多少 token。(需要的量：val_batch_size=8、val_batch_num=50、world_size=2、context_length=1024，以及 TokenStreamDataset 是滑窗。)
- (c) 这条曲线还剩下什么可以说的结论？哪一句话必须收回？

Hint (b) 的陷阱在滑窗：400 个窗口不等于 400×1024 个 token。

参考答案：(a) 按域顺序写：最简单、可复现、manifest 里的分域统计也好算。shuffle=False：验证集不需要 shuffle，而且关掉它能让不同 step 的 eval 可比。islice 取前 50：验证集有 28 万个 batch/rank，不设上限一次 eval 要约 12 小时。叠加之后：每次 eval 都落在同一段、同一个域、同几篇文档上。每一条单独看都是对的，这正是这类 bug 难发现的原因。

(b) 每 rank 400 个窗口，2 个 rank 交错覆盖下标 0..799；下标 i 的窗口是 token $[i, i + 1024)$ ，所以并集是 $[0, 1823)$ ，约 1800 个 token。

(c) 还能说的：loss 在单调下降，而且因为每次是同一批样本，这个下降的信噪比很高。必须收回的：「这是混合验证集上的 val loss」。正经的混合数字是 eval_checkpoint.py 打出来的 3.5866。

代码位置：cs336_systems/Parallelization/FlashDDP/FlashDDP_runner.py:407

3.7 其他几道工程闸门

四个都不复杂，但每一个都能单独毁掉一次无人值守的运行。

(1) NCCL watchdog 把整个 job SIGABRT 掉。 标定 run 在写完最后一个 checkpoint 之后被杀。根因是 save_ddp_checkpoint 原来走 save_checkpoint_and_log，会把每个约 2.3GB 的 checkpoint (fp32 权重 + AdamW 一二阶矩) 当 W&B artifact 上传：rank 0 卡在阻塞式上传里，rank 1 在紧跟其后的 dist.barrier() 里干等，超过 NCCL 默认的 600 秒超时，watchdog 抛异常，整个进程组 abort。

修了四处：checkpoint 只存本地（靠 rsync 取回，cs336_systems/Parallelization/FlashDDP/FlashDDP_runner.py:123）；process group timeout 提到 60 分钟（cs336_systems/Parallelization/FlashDDP/FlashDDP_runner.py:56）；最后的 dist.barrier() 挪到 wandb.finish() 之前（cs336_

_systems/Parallelization/FlashDDP/FlashDDP_runner.py:609); 新增 checkpoint 轮转只保留最近 2 个 (实例根分区只有约 10GB 可用, 4 个 checkpoint 就能撑爆, cs336_systems/Parallelization/FlashDDP/FlashDDP_runner.py:69)。

背后是一条通用的东西: 任何 rank-0-only 的慢操作, 都不能夹在两个集合通信之间。

(2) **if i == 100: break**。训练循环里曾经有一行 `ifi==100;break#TODO:Removeit`。后果是: 不管 config 里 `max_iters` 填多少, `flashddp` 这条路径每个 epoch 硬编码只跑 100 个 batch; 而 `run_pipeline.py` 的 `stage_train_ddp` 也从来没把 `max_iters` 传给这个函数 (只传了 `epochs`)。Day 0 的 smoke test 日志里其实已经露过馅 (进度条打的是 Epoch 1: 99/121803), 当时只关注「跑没跑通」, 没深究那个数字。

修法不是删掉那一行了事, 而是补上一个真的步数预算: 跨 epoch 累计的 `global_step`, 到点跳出 (`cs336\_systems/Parallelization/FlashDDP/FlashDDP\_runner.py:539`)。因为原来的 checkpoint / eval / train-loss 记录全部是按 epoch 粒度的, 而真实训练一个 epoch 都跑不完 (`max_iters` 会在 epoch 1 中途截断), 所以还额外加了 `checkpoint_every_n_steps` / `eval_every_n_steps` / `log_every_n_steps` 三个按步数触发的版本, 并把 `grad_clip` (原来只有单卡路径在用) 也接到了这条路径上。

(3) 静默训练 0 步的「成功」续训。save_checkpoint 的 schema 只有一个 `iter` 字段, 而 `save_ddp_checkpoint` 在 `tag="step"` 时传的是 `global_step`、`tag="epoch"` 时传的是 `epoch`——同一个字段两种语义。加载端把它当 epoch 喂给 `range(start_epoch, epochs)`:

```
>>> list(range(15000, 100))    # step checkpoint 被当成 epoch
[]
>>> list(range(100, 100))      # epoch checkpoint, 已经是最后一个 epoch
[]
```

两种都是空 range。于是「续训」会正常启动、正常打印、一步不训直接退出, 而且不报错。

修法: `global_step` 和 `epoch` 分开存; resume 时恢复 `global_step` (否则就算循环跑起来, LR 也会从头 warmup); 旧 checkpoint 因字段有歧义明确报错并提示改用 `init_from`:

```
179     # Prefer the unambiguous fields. Older checkpoints only have "iter", which was
180     # written
181     # with a *step* count for tag="step" files and an *epoch* count for tag="epoch" files
182     ;
183     # feeding either into `range(start_epoch, epochs)` produced an EMPTY range
184     # (range(15000, 100) and range(100, 100) alike), so resuming silently trained for
185     # zero
186     # steps and exited looking like a successful run. global_step was never restored at
187     # all, so even a non-empty range would have restarted the LR warmup from scratch.
188     epoch = checkpoint.get("epoch")
189     global_step = checkpoint.get("global_step")
190     if epoch is None or global_step is None:
191         raise RuntimeError(
192             f"{resume_from} predates the checkpoint schema fix and records only "
193             f"iter={checkpoint.get('iter')}, which is ambiguous between a step count and
194             "
195             f"an epoch count -- resuming from it cannot be done correctly. Use --
196             init_from "
197             f"to start a fresh run from its weights instead."
198         )
```

(4) **NVTX**: 探测一次, 而不是猜。cs336_basics 有 8 个文件用 `@nvtx.range(...)` 标注 forward。torch.cuda.nvtx 在非 CUDA build 上导入正常、一调用就炸 (`RuntimeError: NVTXfunctionsnotinstalled`), 意味着 TransformerLM 在 CPU/MPS 机器上根本跑不起来——上面那个「在笔记本上验初始化」的 Tip, 当初就是被它挡住的。

nvtx_compat.py 的做法是 import 时探测一次：

```

32 def _nvtx_is_usable() -> bool:
33     """Probe rather than infer. ``torch.cuda.is_available()`` is the wrong question --
34     it asks about a *device*, while NVTX is about whether the *build* has the symbols,
35     and the two come apart (a CUDA build on a machine with no GPU still has NVTX)."""
36     try:
37         _torch_nvtx.range_push("_nvtx_probe")
38         _torch_nvtx.range_pop()
39         return True
40     except Exception:
41         return False
42
43
44 NVTX_AVAILABLE = _nvtx_is_usable()

```

这里的教学点在那三行注释上：不能靠 `torch.cuda.is_available()` 猜。那个函数问的是「这台机器上有没有可用的 GPU 设备」，而 NVTX 问的是「这个 torch build 里有没有编进 NVTX 符号」——两者会脱钩（一个 CUDA build 装在没有 GPU 的机器上，NVTX 照样可用）。当一个能力可以被直接试一次时，不要用一个相关但不等价的条件去推断它。

Checkpoint 3.10 (silent_zero_step_resume): 为什么续训跑了 0 步却不报错

不看代码回答：

- (a) `range(15000, 100)` 和 `range(100, 100)` 分别对应什么场景？为什么两种都是空的？
- (b) 空 `range` 会让整个训练脚本走完哪些步骤？为什么没有任何一处会抛异常？
- (c) 就算把循环修好、让 `range` 非空，还有一个东西会被静默重置。是什么？会造成什么后果？

Hint (b) 想想一个「一步不训」的 run 和一个「正常训完」的 run，在退出码、日志结构、产物文件上分别有什么区别。

参考答案：(a) 第一个是从一个 `tag="step"` 的 checkpoint 续训：`iter=15000` 是步数，被当成 epoch 喂进去，而 `epochs=100`，所以 `15000 > 100`，空。第二个是从 `tag="epoch"` 的 checkpoint 续训：`iter=100` 正好等于 `epochs`，`range(100, 100)` 也空。一个字段两种语义，两种语义各自都能产生空 `range`。

(b) 走完全部：初始化进程组、建模型、加载权重和优化器、打印「resuming at epoch ...」、跳过空的 epoch 循环、跑最后那次 `_run_eval`、保存一个最终 checkpoint、`wandb.finish()`、干净退出，`exit code 0`。没有一处抛异常，因为「空的 for 循环」在 Python 里是完全合法的——沉默失败的本质是「没有任何一处在断言我期望发生的事真的发生了」。这也是为什么修法要主动 `raise` 而不是继续跑。

(c) `global_step`。它原来在 `resume` 之后被无条件重置为 0，所以就算循环跑起来，LR schedule 也会从头重新 warmup——在一个已经训了一半的模型上重跑 warmup，等于把学习率曲线拦腰接错，训练动力学和从头训完全不同。现在 `global_step` 和 `epoch` 分开存、分开恢复。

代码位置：`cs336/systems/Parallelization/FlashDDP/FlashDDP_runner.py:179`

Checkpoint 3.11 (nvtx_probe_not_guess): 探测一次，而不是靠一个相关条件去猜

不看代码回答：

- (a) `torch.cuda.is_available()` 和「NVTX 能不能用」问的分别是什么？给出一个两者

不一致的具体场景。

- (b) 为什么这个探测放在模块 `import` 时只做一次，而不是每次调用 `range()` 时都 `try/except`？
- (c) 退化版的 `range` 用了 `contextlib.contextmanager`。如果直接写成一个返回 `None` 的普通函数，会在什么用法上炸？

Hint (c) 去看 `@nvtx.range(...)` 是怎么用的，再看 `with nvtx.range(...)`：是怎么用的。

参考答案：(a) 前者问设备：这台机器上有没有可见且可用的 CUDA GPU。后者问 `build`：这份 `torch` 二进制里有没有编进 NVTX 符号。不一致的场景：一台装了 CUDA 版 `torch` 但没插 GPU（或 `CUDA_VISIBLE_DEVICES=""`）的机器，`is_available()` 是 `False`，而 NVTX 调用完全正常。反过来也有：某些精简 `build` 有 CUDA runtime 但没有 NVTX。

(b) 因为答案在进程生命周期内不会变，而 `@nvtx.range` 装饰的是 `forward`——每步前向都要走一遍，每层都有。在热路径上放 `try/except` 是白付代价。探测一次、把结果固化成两套函数定义，调用点零开销。

(c) 炸在装饰器用法上。`@nvtx.range(...)` 要求 `range(...)` 的返回值是一个可调用的装饰器；而 `with nvtx.range(...)`：要求它是一个上下文管理器。`contextlib.contextmanager` 产生的是 `ContextDecorator`，同时满足两种——这正是 `torch` 自己的实现方式，所以退化版和真版行为一致。返回 `None` 的普通函数在 `with` 上就会 `AttributeError`。

代码位置：`cs336-basics/cs336\_basics/nvtx\_compat.py:32`

3.8 训练预算：2.6 token/param，是 wall-clock 决定的

最后这一节讲的是一个取舍，不是一个失败，但它必须被主动说出来，否则看到数字的人会以是配置失误。

`max_iters` 是这么定出来的：先跑 500 步标定，从训练进度条解析真实吞吐（稳态约 1.30 batch/s），再按目标 wall-clock 反推步数。最终 `max_iters=30000`、`warmup_iters=900`，对应约 6.5 小时。折算：

$$30000 \text{ step} \times \underbrace{2}_{\text{rank}} \times \underbrace{8}_{\text{batch}} \times \underbrace{1024}_{\text{ctx}} = 491,520,000 \text{ token} \approx 492\text{M}.$$

对 190.4M 参数，这是 **2.58 token/param**。Chinchilla 建议的 20 倍需要约 3.8B token，也就是要训约 51 小时——差了 7.7 倍。

三个必须一起说清楚的数。这三个数容易被混起来，分开写：

- 2.58 token/param：491.5M/190.4M，衡量的是模型训得够不够。
- 2.83 遍语料：491.5M/174.0M (unique token)，衡量的是数据被重复了几次。
- 2.8 token/param：这是标定阶段按 `max_iters=32119` (526M token) 算的数，最终 `run` 缩到 30000 步之后不再适用。冲刺日志里两个数都出现过，容易看混——以 2.58 为准。

这是取舍，不是失误。在「固定 7 小时、2 张 3090」这个预算下，只有两个选项：训一个明显欠训、但曲线干净、产物完整可用的 190M；或者训一个 Chinchilla-optimal 但只能跑到 1/8 进度就没时间的模型。选了前者，因为这次冲刺要的是一条打通的链路（语料 → 训练 → 微调 → serving），不是一个 SOTA 数字。面试里要主动把这句话说出来，别等人问。

下一次的瓶颈不是步数，是数据。语料只有 174M unique token，已经训了 492M (2.83 遍)。继续加步数只是在同一批数据上多转几圈。正确的方向是把 `build_mixed_corpus.py` 的字符预算从 780MB 提到约 3GB (C4 和 Wikipedia 都是流式的，取之不尽)，得到约 700M token，再训到约 2B token (10.5 token/param)。估计成本：建语料 2-3 小时，训练约 12 小时。这个 loss 预测 (3.59 → 约 3.2) 是外推，没有做过。

这一章的产物是一个 checkpoint 和一堆数字。下一章要做的是回答「这些数字里哪些是真的」——§4 拿它做指令微调 ablation，§5 把它送进 serving 链路。

4 SFT ablation: 把灾难性遗忘变成一张表

每个人都“知道”指令微调会造成灾难性遗忘。这是教科书上的一句话，代价小到可以随口说出来。

但“知道”和“你的模型、在你选的那个 lr 上、在四条验证流上分别掉了多少 nats”是两件完全不同的事。前者不需要任何 GPU，后者需要一次预训练、六次微调、二十八次打分，以及一个能让 0.05 量级的差距不被采样噪声吞掉的评测协议。这一章讲的就是后面那件事：我们怎么把一句定性的常识，变成一张可以逐行读、可以拿去反驳自己的表。

这一章的结论有三条，其中两条比我事先预期的强，一条比预期弱。它们都写在 §4.4 和 §4.5；而 §4.6 专门用来写这张表不能支持的东西——这一章正在讲“不要报没有出处的数字”，如果它自己的表没有出处，就自相矛盾了。所以下面每一张表的 caption 里都写清楚：数据来自哪个 JSON、多少个 batch、什么 seed。

4.1 实验设计：6 个 arm，一次回答两个问题

微调这一步只有一个窗口：预训练在 UTC 20:12 跑完，用户睡觉，GPU 空着大约四小时。在这个预算里能做的不是“调出最好的微调模型”，而是“用最少的 arm 把两个自变量的效应分开”。

于是设计成 3×2 ：三个学习率 $\{1e-5, 3e-5, 1e-4\}$ ，乘以“纯 Alpaca”与“Alpaca + 25% 预训练数据回放”两种语料。六个 arm 之外的一切都锁死——同一个起点 checkpoint、同样 900 步、同样 90 步 warmup、同样 seed=0、同样 grad_clip=1.0、同样的 flash_attention_triton + flashddp 组合、同样 25MB 的 bucket。六份 config 逐字段 diff 之后只有两处不同：training.lr，和 tokenized_train 指向 alpaca_train.npy 还是 alpaca_replay_train.npy。

	lr=1e-5	lr=3e-5	lr=1e-4
纯 Alpaca	lr1e5_plain	lr3e5_plain	lr1e4_plain
+25% 回放	lr1e5_replay	lr3e5_replay	lr1e4_replay

Table 10: 六个 arm 的命名。每个 arm 900 步、warmup_iters=90、seed=0、每卡 batch 8、context 1024，都用 init_from 指向同一个预训练 checkpoint checkpoints_pretrain_v1/checkpoint_epoch_0100.pt。出处：configs_sft/*.json 六份，逐字段比对只有 lr 和 tokenized_train 两处不同。

900 步是怎么来的：2 卡 $\times$ batch 8 $\times$ 1024 context = 16,384 token/step，900 步就是约 14.7M token。Alpaca 语料是 4.31M token，于是纯 Alpaca 的 arm 大约看了 3.4 遍。

这里有一个我们当晚没算、写这份讲义时才算出来的混淆项。回放流的拼法见 §2.6：replay_frac=0.25 时拼进去的预训练 token 是 Alpaca 的 1/3，所以整条流是 Alpaca 长度的 4/3。于是在同样 900 步下，replay arm 对 Alpaca 的曝光是 $3.4 \times 0.75 = 2.6$ 遍，比 plain arm 少 25%。这不是设计缺陷，但它是真实的混淆：后面 §4.5 会看到 replay arm 的过拟合漂移小 2.6 倍，其中有多少来自“回放起到了正则作用”、有多少只是“Alpaca 少看了 0.85 遍”，这个实验分不开。

Checkpoint 4.1 (init_from_vs_resume_from): init_from 和 resume_from 差在哪

不看代码回答：

- (a) 微调应该用哪一个？为什么另一个会出问题？
- (b) 这个论证在什么情况下不成立？
- (c) 旧 checkpoint 为什么被显式拒绝，而不是尽力恢复？

Hint 预训练跑完的那一刻，cosine schedule 已经衰到底了。Adam 的一阶二阶矩是在“lr 已经很小”这个 regime 下累出来的。

参考答案: (a) 微调用 `init_from`: 只加载权重, AdamW 的矩和 LR schedule 全部重来。用 `resume_from` 会把两样东西一起继承过来——一个已经衰到最小值的 cosine (微调的前 90 步 warmup 会和它打架), 以及一组适配了那个 regime 的动量估计。微调有它自己的、短得多的 schedule, 继承任何一个都是把预训练末期的状态强加到一个新任务上。

(b) 这个论证只对微调成立, 对同任务续训不成立。一次被中断的预训练要继续跑, 就应该带上 optimizer state 和 `global_step` 恢复——那时 schedule 是同一条, 矩估计也是同一个分布上的, 丢掉它们才是错的。这一点在项目里被自己纠正过一次: 最初的说法是“继承 optimizer state 会和 schedule 打架”, 说得太宽了。

(c) 因为旧 checkpoint 只有一个 `iter` 字段, 而写入端在 `tag="step"` 时存的是 `global step`、`tag="epoch"` 时存的是 `epoch`——同一个字段两种语义。加载端把它当 `epoch` 喂给 `range(start_epoch, epochs)`, `range(15000, 100)` 和 `range(100, 100)` 都是空的: 续训会正常启动、正常打印、一步不训直接退出, 而且不报错。这种失败模式比崩溃危险得多, 所以现在遇到有歧义的旧 checkpoint 直接抛异常并提示改用 `init_from`。

代码位置: `cs336\systems\Parallelization\FlashDDP\FlashDDP\_runner.py:343` (`init_from` 分支)、`cs336\systems\Parallelization\FlashDDP\FlashDDP\_runner.py:179` (拒绝旧 checkpoint 的注释, 异常在 188 行)。

4.2 `init_from` 在代码里长什么样

```

343 # init_from vs resume_from: resume_from continues an interrupted run (weights +
344 # optimizer moments + step counter). init_from starts a NEW run from pretrained
345 # weights only -- fresh AdamW state and a fresh LR schedule -- which is what
346 # fine-tuning wants: the pretraining run ended with its cosine decayed to the
347 # minimum and its moments tuned to that regime, and inheriting either would fight
348 # the fine-tune's own (much smaller) schedule.
349 if init_from is not None:
350     ckpt = torch.load(init_from, map_location=device)
351     state = {k.removeprefix("_orig_mod."): v for k, v in ckpt["model"].items()}
352     model.load_state_dict(state)
353     print(f"Rank {rank} initialized weights from {init_from} "
354           f"(pretraining iter {ckpt.get('iter')}); optimizer state NOT restored.")

```

注意最后那句 `print` 里的 `optimizerstateNOTrestored`。它不是装饰: 无人值守跑六个 arm 的时候, 日志是唯一能证明“这一 run 确实是从预训练权重冷启动的”的东西。一个只在代码注释里写清楚、日志里看不出来的约定, 等于没有。

4.3 配对评测才是 arm 可分辨的原因

先看要分辨的差距有多大。表 13 里 `lr1e5_replay` 和 `lr1e5_plain` 在 general 上差 0.047 nats, 在 instruction 上差 0.007 nats。这是一个很容易被“换一批验证 batch”这件事本身盖掉的量级。

夜里为此做了两件事, 效果完全不同:

- 把最终打分的 batch 数从 100 提到 200;
- 让 `eval_checkpoint.py` 用固定 seed 构造 DataLoader 的 shuffle generator, 于是所有 checkpoint 打的是同一批 batch。

真正让 arm 可分辨的是第二件, 不是第一件。样本量翻倍只把标准误降到 $1/\sqrt{2}$; 而配对之后, “这一批 batch 恰好偏难”这个共同分量在做差时直接抵消掉了。

```

30 @torch.no_grad()
31 def eval_stream(model, path: str, context_length: int, batch_size: int, n_batches: int,
32                 device, seed: int) -> tuple[float, int]:

```

```

33 ds = TokenStreamDataset(path, context_length)
34 # Fixed-seed shuffle: the streams have very different lengths, so scoring the first
35 # N windows of each would compare "start of C4" against "start of alpaca" rather
36 # than a representative sample of either.
37 g = torch.Generator().manual_seed(seed)
38 loader = DataLoader(ds, batch_size=batch_size, shuffle=True, generator=g, num_workers
39                     =2)
40 total, seen = 0.0, 0
41 for x, y in loader:
42     x, y = x.to(device, non_blocking=True), y.to(device, non_blocking=True)
43     total += cross_entropy(model(x), y).item()
44     seen += 1
45     if seen >= n_batches:
46         break
47 return total / max(seen, 1), seen

```

这个说法当晚被验过一次，而且是被迫验的：另一条线索（见 §4.6）显示 general 上的 delta 符号和分域加权反推出来的对不上，第一反应是“会不会只是采样噪声”。于是换了两个 seed 各重打一次分。

	pretrained	lr1e5_replay	delta
主 eval (seed=0)	3.5866	3.6104	+0.0238
seedcheck s1	3.5853	3.6092	+0.0240
seedcheck s2	3.5669	3.5913	+0.0244

Table 11: 同一对 checkpoint，三个不同 seed 的 general 流打分。绝对值漂了 0.020 (3.5669 到 3.5866)，配对差值只漂了 0.0006。出处：artifacts/eval/pretrained.json、sft_lr1e5_replay.json、seedcheck_{pretrained,lr1e5_replay}_s{1,2}.json，每格 200 个 batch、data/tokenized/mixed_valid.npy。delta 由这两列现算。同一组数字在 §2.4 被用于另一个目的（排除采样噪声）；这里用它度量的是配对本身的方差抵消效果。

这张表本身就是配对为什么有效的证据：换 seed 之后单个模型的绝对 loss 动了 0.020，是我们分辨的 arm 间差距 (0.047) 的一半；而两个模型的差只动了 0.0006，小两个数量级。

Checkpoint 4.2 (paired_eval): 为什么 200 个 batch 能分辨 0.05 的差

不看代码回答：

- 如果每个 checkpoint 各自随机采 200 个 batch（不固定 seed），结论会怎样变？
- eval_stream 里为什么要 shuffle=True 而不是直接取前 200 个窗口？
- 配对能抵消什么，不能抵消什么？

Hint 表 11 里那两个数量级的差距，就是答案。

参考答案：(a) 每个 checkpoint 的 loss 上会各自叠一个约 ± 0.01 量级的采样波动（表 11 实测：换 seed 后绝对值动了 0.020），做差之后这两个波动不相消而是相加。0.047 的真实差距会被埋进去，六个 arm 的排序会开始随 seed 抖动。

(b) 因为三条流长度差别很大，而 TokenStreamDataset 是滑窗索引、语料是按域顺序写的。取前 N 个窗口等于拿“C4 的开头几篇网页”去比“Alpaca 的开头几条”，比的是文档而不是模型。固定 seed 的 shuffle 同时拿到了两样：代表性采样，以及跨 checkpoint 的可配对性。

(c) 抵消的是“这一批数据本身偏难/偏易”这个所有 checkpoint 共享的分量。不能抵消的是流本身的构造问题——如果这批数据里混了重复文本，配对只保证所有模型在同样被污染

的数据上被比较，不保证这个数字能被解释成“通用能力”。§4.6 就栽在这一点上。

代码位置: `cs336\_systems/experiments/eval\_checkpoint.py:37` (固定 seed 的 generator)、`cs336\_systems/experiments/eval\_checkpoint.py:34` (注释里写明为什么不能取前 N 个)。

4.4 读那张表

model	general	web	wiki	instruction
pretrained	3.5866	3.8469	3.5518	2.7345
sft_lr1e5_replay	3.6104	3.8375	3.5656	2.4083
sft_lr1e5_plain	3.6572	3.8859	3.5954	2.4150
sft_lr3e5_replay	3.6424	3.8650	3.6044	2.4531
sft_lr3e5_plain	3.7375	3.9637	3.6695	2.4793
sft_lr1e4_replay	3.7698	3.9914	3.7542	2.5685
sft_lr1e4_plain	3.9988	4.2319	3.9224	2.6589

Table 12: 七个 checkpoint $\times$ 四条验证流的绝对 cross-entropy loss (nats/token)。出处: `artifacts/eval/{pretrained,sft_*}.json` 共七个文件，每格 200 个 batch、batch size 8、context 1024、seed=0，流分别是 `mixed_valid.npy` / `domain_valid_web.npy` / `domain_valid_wiki.npy` / `domain_valid_instruction.npy`。所有 checkpoint 打的是同一批 batch。

arm	instruction	general	web	wiki	效率
lr1e5_replay	-0.3263	+0.0238	-0.0093	+0.0138	13.7 $\times$
lr1e5_plain	-0.3195	+0.0706	+0.0390	+0.0437	4.5 $\times$
lr3e5_replay	-0.2814	+0.0557	+0.0181	+0.0526	5.0 $\times$
lr3e5_plain	-0.2553	+0.1509	+0.1168	+0.1177	1.7 $\times$
lr1e4_replay	-0.1661	+0.1832	+0.1445	+0.2024	0.91 $\times$
lr1e4_plain	-0.0756	+0.4122	+0.3850	+0.3706	0.18 $\times$

Table 13: 相对预训练模型的变化，负号 = 变好。”效率” = $|\Delta_{\text{instruction}}|/\Delta_{\text{general}}$ ，即每付出 1 nat 的通用能力代价换回多少 nat 的指令能力。出处: 表 12 逐格相减; 与 `artifacts/NIGHT_REPORT.md` 第 3 节的 delta 表一致到最后一位。效率一列是写这份讲义时用 JSON 里的全精度值现算的，`NIGHT_REPORT.md` 没有这一列。

Example 4.1 (reading_the_ablation_table): 逐行读表 13

一、这个数在说什么。lr1e5_replay 那行的 -0.3263 是: 微调后的模型在 `domain_valid_instruction.npy` 上的 cross-entropy, 比预训练模型低 0.3263 nats。换成 perplexity 是 $15.40 \rightarrow 11.11$ (表 12 对应 JSON 里的 `ppl` 字段), 降了 28%。它不是“指令跟随能力”, 是“在这批 held-out 指令文本上的下一 token 预测更准了”。两者相关但不是一回事——语料里没有任何一条评估是让模型真的执行指令再由人打分的。

二、哪两行可以配对比较。同一列里只改一个自变量的两行:

- lr1e5_replay vs lr1e5_plain: 只差有没有回放。可比。
- lr1e5_plain vs lr3e5_plain: 只差 lr。可比。

这两组各自都是干净的单变量对照，因为除此之外的 config 字段逐字段相同 (表 10 的 caption)。

三、哪两行不能。13.7 $\times$ (lr1e5_replay) 和 0.18 $\times$ (lr1e4_plain) 之间那个 75 $\times$ 的比值不能被叫做“lr 升高造成的效率损失”——这两行同时改了 lr 和回放。同变量的诚实版本是:

- plain 一路： $4.5\times \rightarrow 1.7\times \rightarrow 0.18\times$ ， $1e-5$ 到 $1e-4$ 掉 24.7 倍。

- replay 一路： $13.7\times \rightarrow 5.0\times \rightarrow 0.91\times$ ，同样区间掉 15.1 倍。

两条路都单调、都跨越一个数量级，但数值不同，而且回放把塌陷放缓了。把 $75\times$ 写进结论是把两个效应乘在了一起。

常见误解：回放是拿一点指令能力去换通用能力，是个权衡。

表 13 说不是。同样 $lr=1e-5$ ，replay arm 在 instruction 上是 -0.3263 、plain 是 -0.3195 ——replay 在指令上也更好，同时 general 的代价低 3 倍。四个域全面占优，没有任何一格是 replay 更差。 $lr=3e-5$ 上同样成立（四个域全占优）， $lr=1e-4$ 上也成立。这是严格的 Pareto 改进，不是权衡。

Checkpoint 4.3 (pareto_not_tradeoff): Pareto 改进为什么是可能的

不看代码回答：

(a) 直觉上，“少喂 25% 的指令数据”应该让指令能力变差才对。为什么没有？

(b) 这个结论在三个 lr 上都成立，还是只在小 lr 上成立？

Hint 回想 §4.1 最后那段：在固定 900 步下，replay arm 看到的 Alpaca token 比 plain arm 少 25%。

参考答案：(a) 因为在这个规模上，多看 Alpaca 的边际收益早就转负了。§4.5 会看到所有 arm 的 instruction 验证 loss 在 step 100–300 就触底，之后一路变差——900 步里大部分时间是在过拟合。少喂 25% 等于少过拟合 25%，于是最终 checkpoint 反而更靠近最优点。这个解释只在“步数定得太长”的前提下成立；如果 900 步落在欠拟合区，同样的操作大概率会让指令能力变差。

(b) 三个 lr 上都成立，而且是四个域全占优（表 13 逐格比较 replay 行与同 lr 的 plain 行）。但注意 $lr=1e-4$ 那一对：replay 的 instruction 是 -0.1661 、plain 是 -0.0756 ，两个都远差于 $lr=1e-5$ 的 -0.32 。“replay 更好”和“这个 lr 本身是坏的”可以同时成立。

代码位置：数据在 `artifacts/eval/sft\_lr1e5\_replay.json:1` 等六个文件；回放比例的定义在 `cs336\_systems/data\_harvest/build\_sft\_corpus.py:87`。

Checkpoint 4.4 (why_replay_beats_pretrained_on_web): 为什么 replay arm 在 web 上比预训练模型还好

不看代码回答：

(a) `lr1e5_replay` 在 web 上是 -0.0093 （比预训练模型还低），在 wiki 上却是 $+0.0138$ （变差了）。为什么两个域方向相反？

(b) “25% 预训练数据回放”这个说法准确吗？

Hint 去看 `build_sft_corpus.py` 取的是 `mixed_train.npy` 的哪一段，再去看 `build_mixed_corpus.py` 是按什么顺序往 `train.txt` 里写的。

参考答案: (a) 因为回放的那 25% 全部是 web 文本, 一个 wiki token 都没有。build_sft_corpus.py 取的是 pretrain[:n_replay]——预训练流的一个连续前缀; 而 build_mixed_corpus.py 是按域顺序写文件的 (先 web, 再 wikipedia, 再 instruction), web 独占 546MB 字符预算中的 546MB 里的头部。语料总共 173,972,065 token (tiktoken_encode.log), web 段大致占前 1.2 亿 token; 而 n_replay=n_alpaca/3≈1.44M, 深深地落在 web 段内部。所以 replay arm 实际做的是”指令微调 + 继续在 C4 上预训练”, web 变好、wiki 照样遗忘, 方向完全对得上。(b) 不准确。夜报和 PROGRESS.md 里都写成”25% 预训练数据回放”, 听起来像是按 70/20/10 配比回放; 实测的构造是 100% web 回放。这不影响”replay 是 Pareto 改进”这条结论 (表 13 里 wiki 列 replay 也全面占优, 只是幅度小), 但它确实改变了这条结论的机制解释, 也说明如果想要一个真正按配比的回放, build_sft_corpus.py 得改成分层采样而不是切前缀。

代码位置: cs336_systems/data_harvest/build_sft_corpus.py:91 (pretrain[:n_replay])、cs336_systems/data_harvest/build_mixed_corpus.py:174 (train split 的写入顺序)。

Checkpoint 4.5 (forgetting_efficiency): 定义一个”遗忘效率”指标, 然后说出它的毛病

不看代码回答:

- (a) 表 13 最后一列是怎么算的?
- (b) 它有哪些三个已知缺陷?

Hint 分母是一个可以趋近于零的数。

参考答案: (a) 效率 = $|\Delta_{\text{instruction}}|/\Delta_{\text{general}}$, 两个 delta 都取自表 13, 用 JSON 全精度值算。读法是”每付出 1 nat 通用能力, 换回多少 nat 指令能力”。

(b) 三个毛病, 按严重程度排:

- (1) 分母趋零时它会爆炸。一个几乎不动权重的 arm (比如 lr=1e-7) 会拿到一个巨大的效率值, 而它其实什么都没学到。所以这个指标只能在指令增益可比的 arm 之间用——比较 13.7× 和 4.5× 是合法的 (两者 instruction 增益差 2%), 比较 13.7× 和 0.18× 就已经不太合法了 (instruction 增益差了 4 倍)。
- (2) 它把两个不同单位的东西当成同一个单位相除。分子是 Alpaca held-out 上的 nats, 分母是混合验证流上的 nats。两者可以相除, 但商没有物理意义, 只是一个排序用的比值。
- (3) 分母那条流本身有 10% 不干净 (§4.6)。排序不受影响, 但绝对值不该被引用。

所以这一列在讲义里保留, 是因为它把六个 arm 排出了一个和四列 delta 完全一致的序; 它不是一个可以拿去跨项目比较的指标。

4.5 曲线: 收益几乎全在前 100 步

六条曲线的形状是一样的: 在 step 100 或 300 触底, 然后基本单调变差 (有 0.002 量级的抖动, 例如 lr1e5_plain 在 300→400 微降、lr1e4_replay 在 400→500 微降)。把最低点和第 900 步的差叫做漂移:

漂移随 lr 单调上升 (0.0241 → 0.0878 → 0.2637, plain 一路), 回放把它压小 (同 lr 下 0.0241 → 0.0094, 小 2.6 倍)。但请记住 §4.1 末尾那个混淆项: replay arm 在同样步数下对 Alpaca 的曝光少 25%, 这 2.6 倍里有多少是”回放正则化”、有多少是”少看了 0.85 遍”, 本实验分不开。

arm	100	200	300	400	500	600	700	800	900
lr1e5_plain	2.4390	2.4465	2.4488	2.4486	2.4566	2.4585	2.4618	2.4620	2.4631
lr1e5_replay	2.4490	2.4475	2.4472	2.4515	2.4493	2.4546	2.4531	2.4553	2.4566
lr3e5_plain	2.4430	2.4618	2.4820	2.4854	2.4994	2.5209	2.5294	2.5296	2.5308
lr3e5_replay	2.4475	2.4520	2.4658	2.4898	2.4849	2.5028	2.5012	2.5072	2.5097
lr1e4_plain	2.4542	2.4955	2.5544	2.5703	2.5854	2.6689	2.6894	2.7010	2.7179
lr1e4_replay	2.4571	2.4639	2.5203	2.5727	2.5606	2.6095	2.6215	2.6303	2.6310

Table 14: 训练内验证 loss，每 100 步一次，粗体是各 arm 的最低点。出处：run_logs/sft_*.log 里的 Rank0|stepNNNNNN|Valloss：行，每次 eval 是 alpaca_valid.npy 上 val_bat_num=50 个 batch。注意这不是表 12 的 instruction 列——那一列是 domain_valid_instruction.npy 上 200 个随机 batch，两条流不同、采样也不同。

arm	best val	best 出现在	final val (step 900)	漂移
lr1e5_replay	2.4472	step 300	2.4566	+0.0094
lr1e5_plain	2.4390	step 100	2.4631	+0.0241
lr3e5_replay	2.4475	step 100	2.5097	+0.0622
lr3e5_plain	2.4430	step 100	2.5308	+0.0878
lr1e4_replay	2.4571	step 100	2.6310	+0.1739
lr1e4_plain	2.4542	step 100	2.7179	+0.2637

Table 15: best val 与 final val。出处：表 14 的九个点取最小值与末值，与 artifacts/NIGHT_REPORT.md 第 2 节的 best/final 两列逐位一致。“best 出现在”和“漂移”两列是本讲义现算的，NIGHT_REPORT.md 只有 best 和 final。

Checkpoint 4.6 (eval_density): 9% 的开销换一条什么样的曲线

不看代码回答：

- eval 从每 300 步加密到每 100 步，代价是多少？
- 如果保持每 300 步，表 14 会变成什么样？哪条结论会丢？
- 预训练那条曲线为什么反而故意做得很稀疏（全程约 5 次）？

Hint 一个 arm 的墙钟是多少，一次 eval 是多少。

参考答案: (a) 一次 eval 约 7 秒（50 个 batch），一个 arm 约 700 秒（六个 arm 的启动时间戳间隔约 15 分钟，见 run_logs/post_train_window.log）。9 次 eval $\approx$ 63 秒 $\approx$ 9%。

(b) 只剩 step 300 / 600 / 900 三个点。以 lr1e5_plain 为例就是 2.4488/2.4585/2.4631 ——单调上升，看得出“越训越差”，但看不出拐点在哪，因为最低点（step 100）根本不在采样里。”收益几乎全在前 100 步、合理步数是 100–200”这条结论会丢，剩下的只是“900 步太长”这个模糊印象。更糟的是 lr1e5_replay：三个点是 2.4472/2.4546/2.4566，最低点恰好落在 step 300 上，会让人以为拐点就在那儿。

(c) 因为两者问的问题不同。预训练那条曲线要回答的是“loss 在不在降、有没有 NaN”，一次 eval 本来就是验证集上一次带误差棒的随机采样估计，把频率提高十倍买到的是噪声不是信号，而 7 小时的训练里每次 eval 都在挤真正的训练步数。微调这边要回答的是“拐点在哪”，那是一个关于形状的问题，形状需要采样密度。同一个旋钮在两个场景下往相反方向拧，是因为要回答的问题不同，不是前后矛盾。

代码位置: configs\sft\lr1e5_replay.json:1 (eval_every_n_steps: 100); 预训练那边的理由写在 real_pretrain_config.json 的 _NOTE_eval_frequency 字段里。

Downscaling Tip: 3 arm 版本

没有四小时空闲 GPU 也能得到前两条结论。把网格砍成三个 arm: lr1e5_plain / lr1e5_replay / lr1e4_plain。

- Pareto 改进照样能得：它只需要 lr1e5 那一对（表 13 前两行）。
- 收益全在前 100 步照样能得：任何一个 arm 的九点曲线都够（表 14）。
- 遗忘临界点会退化两个点（ $4.5\times$ 和 $0.18\times$ ），只能说“效率在 $1e-5$ 到 $1e-4$ 之间掉了 24.7 倍”，不能说这条曲线长什么样。中间那个 $3e-5$ 的点（ $1.7\times$ ）就是买“单调性”这个性质的钱。

Resource requirements: 三个 arm 各 900 步。夜里六个 arm 在 2×3090 上跑了约 90 分钟（21:46–23:15 UTC，见 run_logs/post_train_window.log 的 arm 时间戳），所以三个 arm 约 45 分钟。评测部分（eval_checkpoint.py，四条流 $\times$ 200 batch）在 CPU 上也能跑，只是慢。

4.6 这一章不敢报的东西**Checkpoint 4.7 (last_not_best): 被打分的 checkpoint 不是最好的那个**

不看代码回答：

- 表 12 里每个 arm 被打分的是哪一步的权重？为什么这仍然是公平的？
- 为什么没有补一次“在最优步数停下”的 run？
- “best val”和“表 12 的 instruction 列”是不是同一个数？

Hint 表 15 的“漂移”列，以及表 14 的 caption。

参考答案：(a) 都是 step 900 的权重（checkpoint_epoch_0100.pt，见每个 eval JSON 的 checkpoint 字段）。checkpointing 存的是最后一步，不是最好那一步。这仍然公平，是因为六个 arm 待遇完全一致——都被扣掉了各自的漂移，而且排序问题问的是相对关系。但要注意这个扣减不是等量的：lr1e4_plain 被扣掉 0.2637，lr1e5_replay 只被扣掉 0.0094。也就是说这张表系统性地低估了大 lr arm，真实的“最优早停 lr1e4”没有表里看起来那么糟。这一条对结论方向没有影响（它只会让 lr1e4 更差看），但如果反过来——如果 winner 是漂移最大的那个 arm——这张表就不能用了。

(b) 因为 winner (lr1e5_replay) 的漂移只有 0.0094，它的最终 checkpoint 基本就在最优点上。为 0.009 nats 再租一次 GPU、重跑一遍 900 步，是做样子不是做实验。

(c) 不是。三处不同：best val 来自 alpaca_valid.npy、训练循环内的 50 个 batch、而且是 DistributedSampler(shuffle=False) 取前 50 个窗口（所以是该流开头的一个窄切片）；表 12 的 instruction 列来自 domain_valid_instruction.npy、200 个随机 batch、固定 seed。lr1e5_plain 的两个数分别是 2.4390 和 2.4150。它们不该被放在一起比较，在报告里必须分开写。

代码位置：打分脚本 cs336_systems/experiments/eval_checkpoint.py:72；训练内 eval 的窄切片性质在 artifacts/NIGHT_REPORT.md:15 的两条 caveat 里写着。

第二件不敢报的：general 这条流有 10% 的成分是被污染的。

mixed_valid.npy 的 instruction 区段是 build_mixed_corpus.build_instruction 把 2600 条 held-out Alpaca 循环 1.91 遍凑出来的——这个做法对 train split 是对的（用 epoch 数给小而精的域配比，正是 LLaMA 那套设计本身），但它被同样用在了 valid split 上。重复过的验证数据

是假的容易：一个 1024-token 窗口里可能装着同一条样本两次，第二遍靠 in-context 复制就能预测。完整的追查过程和那张“算术闭合”的切片表（表 3）属于数据那一章，见 §2.4；这里只取它落在本章上的两个后果。

第一，污染改变了 `delta` 的符号：在重复过的 instruction 区段上，`lr1e5_replay` 是 +0.0924（“变差”），而在干净的 `domain_valid_instruction.npy` 上是 -0.3263（大幅变好）。同一批 2600 条样本，两条流，符号相反。

第二，所以表 13 的 `general` 一列 绝对值和分解都不能引用，能引用的只有 arm 之间的排序——所有 arm 打的是同一批被同样污染的数据、配对比较，排序不受影响。本章里凡是用到 `general` 的地方（包括那个“效率”指标的分母）都受这一条约束。

第三件：这不是教科书意义上的 SFT。`cs336_basics` 的 `cross_entropy` 没有 loss masking, loss 是在整篇 `###Instruction:...###Response:... 文档` 上算的。论证见 §2.6 和那里的 Checkpoint 2.6。本章标题仍然叫 SFT，是因为改名会和代码里的目录名、config 名全部脱节；但每次引用表 13 都得带上这句话。

第四件：有几个数字这台机器上核不了。Alpaca 语料 4.31M token、三条分域验证流各 267K / 281K / 226K token——这些只记录在 `interview_prep/PROGRESS.md` 里，对应的 `data/` 目录和构建日志留在已经停掉的远端实例上，本地 `data/tokenized/` 只剩 TinyStories 的三个 `.npy`。本讲义里凡是用到这些数的地方（比如“900 步 $\approx$ 3.4 遍”）都是从它们推出来的，因此同样未经独立核实。

4.7 没有中途改实验

第一个 arm (`lr1e5_plain`) 跑完，九个点的曲线一出来，“900 步定得太长、合理值是 100–200 步”就已经很明显了。当时还有五个 arm 没跑。

Checkpoint 4.8 (dont_change_mid_experiment): 看到第一个 arm 就知道设错了，为什么不改

不看代码回答：

- (a) 把剩下五个 arm 改成 200 步，会失去什么？
- (b) 除了“可比性”之外，还有一个更强的理由不改。是什么？
- (c) 什么情况下应该中途停下来改？

参考答案：(a) 失去整张表。六个 arm 里只要有一个用了不同的步数预算，表 13 的每一行就不再是同一个实验的六个观测，而是两个实验各三行。`lr` 的效应和步数的效应会混在一起，“效率随 `lr` 单调下降”这条结论直接作废。

(b) 这条曲线本身就是结论。我们要的不只是“一个微调好的模型”，还有“微调的收益在哪一步耗尽”这个可以量化的事实。如果在 step 200 就停，表 14 只剩两列，“漂移随 `lr` 单调上升、回放把漂移压小 2.6 倍”（表 15）这一整块就没有了。换句话说，那个“设错了的”`max_iters=900`，恰好是让过拟合过程可观测的设置。

(c) 当发现的是正确性问题而不是设计不优的时候。这个区别在同一个项目里刚好有对照：预训练发散成 NaN 时立刻停了，因为那是一个 bug；微调步数偏长没有停，因为那只是一个不够优的选择，而且它产出的数据有独立价值。判据是“继续跑下去还会不会得到有效数据”——NaN 之后不会，过拟合之后会。

这一节和 §4.6 合起来是这一章的方法论：实验的价值不取决于参数选得多好，取决于它不能被逐行读、能不能被反驳。六个 arm 里有五个的最终 checkpoint 都不如它们自己的 step-100 版本；那不妨碍这张表说清楚了它要说的三件事。

5 KV cache 与 serving 集成：把两个仓库接起来

到这一步为止，两个仓库互不相识。LLM-system-project 训出了一个 0.19B 的 TransformerLM，它的公开接口就一个：forward(x) -> logits，输入 (batch, seq) 的 token id，输出 (batch, seq, vocab) 的 logits。llm-serving 那边的 LMEngine 是照着 HuggingFace 的形状写的：它调 model(input_ids=..., past_key_values=..., use_cache=True)，从返回值上取 .logits 和 .past_key_values，还会去读 model.config。两边一样都对不上。

摆在面前的有两条路：改 LMEngine 让它容纳两种模型，或者造一个 adapter 把 TransformerLM 包成 HF 的形状。这一章的第一件事，是说明“只补四样东西”这个约束是怎么把设计钉死的——一旦确认 LMEngine 依赖的 HF 约定恰好只有四样，adapter 就不再是“一个兼容层”这种含糊的东西，而是一个有明确边界、可以被逐条检查的对象。

第二件事是 KV cache 本身：它怎么做成一个对训练路径零风险的纯增量改动，怎么被测，以及它实测的收益和我预期的完全不一样 (§5.7，本章的重点)。

5.1 LMEngine 到底依赖 HF 的什么

LMEngine 是故意手动驱动模型的，不用 .generate()——这个决定和它的理由属于姊妹篇，见 [Assignment A §3.1 <为什么不用 generate\(\)>](#)。副作用是它把 HF 的调用约定摊开写在了自己的代码里，于是可以被逐行数出来。数的结果是四样，一样不多：

LMEngine 要的 HF 约定	本项目有	adapter 怎么补
model.eval()	有	不用补。TransformerLMAdapter 继承 nn.Module，.eval() 会递归到被包的模型。
model(input_ids, past_key_values, use_cache) 返回带 .logits 的对象	部分有	forward 接受这三个关键字参数 (use_cache 收下但不使用)，把结果装进 CausalLMOutput 这个 NamedTuple。
返回值还要有 .past_key_values，没有且能原样喂回下一步	没有	同一个 NamedTuple 的第二个字段。默认模式下它装的是 token 前缀，use_kv_cache=True 时装的是逐层 (K, V)。
model.config.n_positions	没有	AdapterConfig 这个 dataclass 的字段，构造时由 context_length 参数填。
model.config.eos_token_id	没有	同一个 dataclass 的字段，默认 50256 (GPT-2 的 < endoftext >)。

Table 16: LMEngine 依赖的四样 HF 约定（第二和第三行是同一次调用的两个返回字段，所以数成四样）× TransformerLM 有没有 × adapter 怎么补。出处：逐行读 llm-serving/inferenceLM/engine/lm_engine.py，四个依赖点分别在第 12 行 (eval())、第 36 和 60 行（模型调用）、第 38 和 40 行 (.logits / .past_key_values)、第 89 行 (n_positions)、第 119 行 (eos_token_id)。

这个“只有四样”的清单，以及为什么是这四样而不是别的，在姊妹篇里有完整的推导：[Assignment A §3.4 <四个方法逐行读>](#)。这一章只用它的结论。

Checkpoint 5.1 (four_hf_conventions): 为什么“数清楚是四样”比“写一个兼容层”重要

不看代码回答：

- 如果不先数清楚，最可能写出什么样的 adapter？
- use_cache 这个参数 adapter 收下了却不用，为什么不干脆删掉？
- 少补了其中一样，会在什么时候暴露？

Hint 想想 config 这个对象在 HF 里有多少个字段。

参考答案: (a) 大概率会写成“尽量像 HF”——把 PretrainedConfig 的一堆字段照抄一遍，返回值也做成一个字段齐全的 dataclass。那样 adapter 的正确性就没有边界了：你没法回答“它够不够”，只能回答“它像不像”。数清楚之后，adapter 的规格变成一个五行的清单，每一行都能对应到 lm_engine.py 的一个具体行号，也就能被逐条测。AdapterConfig 的 docstring 里那句“the subset of a HuggingFace PretrainedConfig that LMEngine actually reads”就是这个意思。

(b) 因为它是调用方的约定，不是被调方的选择。LMEngine.decode() 里写死了 use_cache=True；adapter 如果不接这个关键字，那一行就是 TypeError。收下但不使用，等于把“这个开关在我们这边没有意义”这件事写进签名里，而不是逼调用方为我们改代码。

(c) 在请求时，不是构造时。四样里三样是属性访问(config.n_positions、config.eos_token_id、outputs.past_key_values)，Python 不会在 TransformerLMAdapter(model,...) 这一步检查它们，只会在第一次真的处理请求时抛 AttributeError。这就是为什么 transformer_lm_adapter_test.py 里专门有一个只断言两个 config 字段的测试——它的 docstring 明说了这一点。

代码位置: llm-serving/inferenceLM/engine/lm_engine.py:89 与 llm-serving/inferenceLM/engine/lm_engine.py:119；测试在 llm-serving/tests/transformer_lm_adapter_test.py:86。

5.2 三个类，和一条不 import 的规矩

整个 adapter 是三个类，连同 39 行模块 docstring 一共 151 行。

```
@dataclass
class AdapterConfig:
    """The subset of a HuggingFace `PretrainedConfig` that `LMEngine` actually reads."""
    n_positions: int
    eos_token_id: int
    vocab_size: Optional[int] = None

class CausalLMOutput(NamedTuple):
    logits: torch.Tensor
    past_key_values: torch.Tensor
```

来源：llm-serving/inferenceLM/engine/transformer_lm_adapter.py:50–67，手抄（跨仓库文件，不能用 \textbackslashslstinputlisting）。CausalLMOutput 的 docstring 已省略，正文在 §5.3 引用。

第三个类 TransformerLMAdapter 继承 nn.Module，构造时收下已经建好的模型、一个 context_length、一个 eos_token_id，以及一个 use_kv_cache 开关。它的 forward 就是表 16 第二行那个签名。

真正值得停一下的是它没有做的一件事：transformer_lm_adapter.py 从头到尾没有 import cs336_basics。它的类型标注是 model: nn.Module，docstring 写着“Typed loosely on purpose”。这有三个后果，三个都是当初想要的：

- (1) inferenceLM 对训练项目零依赖。装了 llm-serving 的人不需要装 cs336_basics；两个仓库仍然可以各自独立 clone、独立测试。唯一需要两边都在的是 tests/trained_model_integration_test.py，它用 pytest.importorskip 在缺一半的环境里自己跳过 (llm-serving/tests/trained_model_integration_test.py:37)。
- (2) adapter 可以对着 stub 测。只要是 (batch, seq) -> logits 的 nn.Module 都能塞进去——§5.4 会看到塞进去的其实是 GPT-2。

- (3) 绕开了 `llm-serving/CLAUDE.md` 的 `Forbidden` 约束。那份文件写着“不许在 `inferenceLM/` 和 `tests/` 之外改文件”、“不许不加论证地引入新依赖”。一个 `import` 训练项目的 `adapter` 会同时违反两条；一个只收 `nn.Module` 的 `adapter` 一条都不违反。

Checkpoint 5.2 (`adapter_not_branch`): 为什么造 `adapter`，不是在 `LMEngine` 里加分支

不看代码回答：

- (a) 在 `LMEngine` 里加一个 `if isinstance(model, TransformerLM)` 分支，会带来什么具体后果？
- (b) `adapter` 这个方案的代价是什么？（它不是免费的）

参考答案：(a) 三个后果。第一，`LMEngine` 要 `import` 训练项目，两个仓库从此绑死，`llm-serving` 的 `uv sync` 得把 `einops/tiktoken` 一起拉进来。第二，`prefill()/decode()/inference()` 三个方法里每一处碰 `past_key_values` 或 `config` 的地方都要开分支，而这三个方法正是将来要改成 `continuous batching` 的地方——分支会被复制到每一个新特性里。第三，“`LMEngine` 支持哪些模型”这个问题从此没有静态答案，只能靠读分支。

(b) 代价是多了一层间接，以及一个字段名装了两种完全不同的东西——`past_key_values` 在默认模式下是 `token` 前缀，在 `use_kv_cache=True` 下是逐层 (K, V) 。这是真实的可读性代价，§5.3 的例子把两种模式并排放，就是为了让这一点无处可藏。它之所以可以接受，是因为 `LMEngine` 把这个字段当不透明对象处理：它只负责把上一步的返回值原样传回去，从不解释里面装的是什么。

Checkpoint 5.3 (`loose_typing_on_purpose`): 松类型标注是偷懒还是设计

不看代码回答：

- (a) `model: nn.Module` 而不是 `model: TransformerLM`，放弃了什么？换来了什么？
- (b) 有没有一个地方，`adapter` 还是做了运行时检查？

参考答案：(a) 放弃的是静态类型检查——`mypy` 不会告诉你塞进去的东西有没有正确的 `forward` 签名，签名不对要到第一次 `forward` 才炸。换来的是 §5.2 里那三条：零依赖、可对 `stub` 测、不违反仓库约束。这是一次明确的取舍，不是忘了写标注：`docstring` 里 “`Typed loosely on purpose`” 那半句就是在说这件事。

(b) 有，而且只有一处：`use_kv_cache=True` 时会 `hasattr(model, "forward_with_cache")`，不满足就在构造时 `raise ValueError`，错误信息里直接告诉你正确的签名，以及“留 `False` 就能用任何 `(batch, seq) -> logits` 模块”。选择在这里做检查而不是别处，是因为这是唯一一个“松类型会导致运行到一半才失败”的地方——其余的能力缺失在第一次 `forward` 就会暴露。

代码位置：`llm-serving/inferenceLM/engine/transformer_lm_adapter.py:97`。

5.3 默认路径的 `past_key_values` 不是 `cache`，是 `token` 前缀

Day 3–4 的 `adapter` 刻意不做真 `KV cache`。`past_key_values` 里装的是到目前为止的全部 `token id`，每一步 `decode` 都把这个前缀重新跑一遍完整前向。

Example 5.1 (adapter_call_trace): 同一个字段名，两种完全不同的东西

把 LMEngine 驱动 adapter 的三步调用并排写出来。prompt 长度 5，生成 2 个 token。左边是默认的 recompute 模式，右边是 use_kv_cache=True。

调用	参数	recompute 模式	cache 模式
prefill	input_ids	(1,5) 整个 prompt	(1,5) 整个 prompt
	past_key_values	None	None
	模型内实际算	5 个位置	5 个位置
	返回 .logits	(1,5,V)	(1,5,V)
	返回 .past_key_values	(1,5) token id	16 层 $\times$ (K,V)，各 (1,12,5,64)
decode 1	input_ids	(1,1) 新 token	(1,1) 新 token
	past_key_values	(1,5)	上一步那 16 层
	模型内实际算	6 个位置（全部重算）	1 个位置
	返回 .logits	(1,6,V)	(1,1,V)
	返回 .past_key_values	(1,6) token id	16 层，各 (1,12,6,64)
decode 2	模型内实际算	7 个位置	1 个位置
	返回 .logits	(1,7,V)	(1,1,V)

两件事一眼可见。第一，.logits 的形状在两种模式下不一样：recompute 返回整个前缀的 logits，cache 只返回新 token 的。这不影响 LMEngine，因为它自己切 [:, -1, :] (llm-serving/inferenceLM/engine/lm_engine.py:62)，两种模式下这一切都取到“最后一个位置的 logits”。第二，累计到第 n 步，recompute 一共算了 $\sum_i (L+i) = O(n^2)$ 个位置，cache 是 $O(n)$ 。形状是 16 层 $\times$ 12 头 $\times d_k=64$ ，来自 configs_sft/lr1e5_replay.json 的 model 段 (num_layers=16, num_heads=12, d_model=768)。

这个取舍是 PLAN.md 里就写好的，理由写进了模块 docstring。原话是：

Recompute keeps the two projects' seam small and honest: the serving layer is genuinely running the locally-trained weights, and the cost of not having a cache shows up as latency rather than as a correctness fudge.

来源：llm-serving/inferenceLM/engine/transformer_lm_adapter.py:31–33，逐字手抄。

”体现为延迟而不是正确性上的糊弄”这句话是有内容的，不是修辞。它说的是：recompute 解码和 cache 解码必须产生完全相同的 token 序列，差别只能出现在时间上。这条等式后来变成了 §5.6 里所有测试的共同断言，也变成了 §5.7 那张对比表能被解释的前提——如果两条路径的输出会分叉，比较它们的 tok/s 就没有意义。

Checkpoint 5.4 (recompute_is_not_a_cache): 一个叫 cache 的字段里没有 cache

不看代码回答：

- recompute 解码的复杂度是 $O(n^2)$ ，那 TTFT 会不会也受影响？
- 为什么 recompute 版本在 prefix.shape[1]>n_positions 时是从左边截断？
- 这个设计在什么情况下会从“慢”变成“错”？

参考答案：(a) 不会。TTFT 是 prefill 那一次前向，两种模式下算的位置数完全相同 (§5.3 的例子第一行)。实测也印证了：recompute 的 TTFT 是 61.3–65.7 ms，cache 是 59.3–63.0 ms，差别落在 run-to-run 波动里（表 18）。KV cache 从定义上就不改善 TTFT。

(b) 因为 LMEngine 已经在 inference() 里拒绝了超过 n_positions 的 max_length，所以这条路径只有在有人绕过 LMEngine 直接驱动 adapter 时才会走到。这种情况下”丢掉最老的

token”是最不意外的行为——注释里写的是“the least surprising behaviour”。

(c) 当 prefix 长到需要截断的时候。截断之后模型看到的位置从 0 重新开始编号，生成会和“完整上下文”的结果不同——但这不是 cache 与 recompute 的差别，是滑动窗口本身的语义。真 cache 路径在同样情况下的选择不一样：TransformerLM.forward_with_cache 直接 **raise** RuntimeError，理由是越界位置根本没有 RoPE 表项，静默回绕比报错糟糕得多（cs336-basics/cs336_basics/lm.py:203）。两条路径在越界这件事上行为不一致，这是一个真实的不一致，讲义里记下来，没有修。

代码位置：llm-serving/inferenceLM/engine/transformer_lm_adapter.py:146（左截断）、cs336-basics/cs336_basics/lm.py:203（越界报错）。

5.4 验证：把 GPT-2 自己剥成裸签名

recompute 解码”只会更慢、不会不同”是一个可以被证伪的断言。怎么证？

夜里用的办法是：拿 GPT2LMHeadModel 包一层，只暴露 (batch, seq)→(batch, seq, vocab)——把它的 config、它的 cache、它的输出 dataclass 全部剥掉，剥完之后它和我们自己的 TransformerLM 长得一模一样。然后把这个裸模型塞进 adapter，用 LMEngine 跑贪心解码，和 HuggingFace 自带的、带 KV cache 的 generate() 对拍。

```
class _LogitsOnly(nn.Module):
    """Reduce a HuggingFace causal LM to the bare (batch, seq) -> (batch, seq, vocab)
    signature that a from-scratch `TransformerLM` has."""

    def __init__(self, hf_model: GPT2LMHeadModel) -> None:
        super().__init__()
        self.hf_model = hf_model

    def forward(self, input_ids: torch.Tensor) -> torch.Tensor:
        return self.hf_model(input_ids=input_ids).logits
```

来源：llm-serving/tests/transformer_lm_adapter_test.py:15–30，手抄，docstring 只保留了第一句。

结果是 token 级完全一致，6 个测试全过。这个手法的正式名字和它的适用边界在姊妹篇里：Assignment A §5.1 <差分测试：让参照实现来定义“正确”>。

Checkpoint 5.5 (gpt2_as_stub): 为什么 stub 要用 GPT-2，而不是一个随机小模型

不看代码回答：

- (a) 用一个随机初始化的两层小模型当 stub，这个测试还成立吗？
- (b) 这个测试到底隔离出了什么？它不能证明什么？

Hint 参照物是“HuggingFace 自己带 cache 的 generate()”。

参考答案：(a) 部分成立但弱得多。随机小模型没有可信的参照实现——你只能拿它和它自己比，那测的是“recompute 循环写得对不对”，不是“recompute 语义对不对”。用 GPT-2 才能拿到一个独立实现的、带真 KV cache 的 generate() 当参照物：两边如果对上，说明 adapter 的 recompute 语义和工业界公认的 cached 语义等价。另外随机权重的 logits 很平，argmax 容易在数值噪声上翻转，会让测试变得又假阳又假阴。

(b) 隔离出来的是 adapter 的 recompute 逻辑——前缀怎么拼、[:, -1, :] 怎么切、past_key_values 怎么传回去。因为模型两边都是同一个 GPT-2 权重，模型本身的正确性被消掉了。它不能证明的是我们自己的 TransformerLM 是对的（那是训练那边的事），也不

能证明真 KV cache 是对的——那是另一套测试，见 §5.6。这两件事很容易混，但它们用的是完全不同的参照物：这里的参照物是 HF 的 `generate()`，那边的参照物是我们自己的 `forward`。

代码位置: `llm-serving/tests/transformer\lm\adapter\_test.py:62`。

5.5 真 KV cache：三个新方法，forward 一个字节没动

夜里第二轮才把真 cache 做出来，而且是刻意放在最后做的。实现方式是纯增量：

- `MultiHeadsAttention.forward_with_cache` (`cs336-basics/cs336\_basics/transformer/multiheads\_attention.py:127`)
- `PreNormTransformer.forward_with_cache` (`cs336-basics/cs336\_basics/transformer/transformer.py:77`)
- `TransformerLM.forward_with_cache` (`cs336-basics/cs336\_basics/lm.py:183`)

三个新方法。原来的 `forward` 一个字节没动，也就是训练路径零风险——改完之后在远端重跑 `init check`，输出和改之前逐位相同 (`W1 std 0.06533 / 初始 loss 7.0294 / global grad norm 2.151`，记录在 `interview_prep/PROGRESS.md`；本机没有对应日志可核对)。把这条做成硬约束的代价是有一点重复代码 (QKV 投影那几行在两个方法里各写了一遍)，换来的是“这个改动不可能影响到已经跑完的 7 小时训练”这个可以直接说出口的性质。

真 cache 为什么在数学上等价于重算——同一个位置的 K 、 V 只依赖于它自己那个 token 的隐状态，所以算过一次就可以存下来——这个论证属于姊妹篇，不在这里重复：Assignment A §3.2 <KV cache 为什么合法>。这一节只讲两个实现层面的不变量，它们不是数学问题，是“写错了会静默出错”的问题。

5.5.1 不变量 1: `is_causal` 不是手写开关，是从 `shape` 推出来的

```

154     # Append to the cache AFTER rotating: RoPE is applied once, at each token's own
155     # absolute position, so cached keys must already carry their rotation.
156     if past_kv is not None:
157         past_k, past_v = past_kv
158         K = torch.cat([past_k, K], dim=-2)
159         V = torch.cat([past_v, V], dim=-2)
160
161     Q_packed = rearrange(Q, "... h seq d_k -> (... h) seq d_k")
162     K_packed = rearrange(K, "... h seq d_k -> (... h) seq d_k")
163     V_packed = rearrange(V, "... h seq d_v -> (... h) seq d_v")
164
165     # Causal masking only applies when queries and keys are the same block (prefill).
166     # During decode the queries are the newest tokens and every cached key strictly
167     # precedes them, so the triangular mask is not just unnecessary but wrong: it is
168     # built as tril(seq_q, seq_k), which for a single query would expose only key 0.
169     is_causal = Q_packed.shape[-2] == K_packed.shape[-2]
170     out_packed = scaled_dot_product_attention(Q_packed, K_packed, V_packed, is_causal
= is_causal)

```

第 169 行是这一节的重点：

$$\text{is_causal} = \text{Q_packed.shape}[-2] == \text{K_packed.shape}[-2]$$

没有 `if is_prefill:`，没有多传一个 `mode` 参数，也没有让调用方负责设对。”什么时候该 mask”被表达成了一个关于 `shape` 的性质：query 和 key 数量相同 $\Leftrightarrow$ 它们是同一个 block $\Leftrightarrow$ 需要三角遮罩。prefill 时 $\text{seq}_q = \text{seq}_k = n$ ，自动为真；decode 时 $\text{seq}_q = 1$ 、 $\text{seq}_k = n+1$ ，自动为假。

Checkpoint 5.6 (is_causal_from_shape): decode 时为什么必须 is_causal=False

不看代码回答：

- (a) 现有的 causal mask 是怎么构造的？单 query 时它会露出哪些 key？
- (b) 如果 decode 时错误地传了 is_causal=True，模型会崩溃、还是会输出看起来正常但错误的结果？
- (c) 用 shape 推导代替一个显式开关，除了少写一个参数，还买到了什么？

Hint mask 的形状是 (seq_q, seq_k) ，不是 (seq_k, seq_k) 。

参考答案：(a) `causal_mask=torch.tril(torch.ones(seq_q, seq_k))` (`cs336-basics/cs336\_basics/transformer/scaled\_dot\_prod\_attention.py:49`)。这是一个左对齐的下三角：第 i 行露出 key $0 \dots i$ 。decode 时 $seq_q = 1$ ，只有第 0 行，于是只露出 key 0——整个缓存里除了第一个 token 全被 mask 掉。

(b) 不会崩溃。形状全部合法，softmax 正常，输出的 logits 形状也对。模型会安静地生成一串“只看得见第一个 token”的续写。这正是这一类 bug 最危险的地方：它不产生任何异常，只产生更差的文本，而一个 190M 的欠训模型本来输出就不怎么样，很容易被解释成“模型就是这个水平”。

(c) 买到的是这个 bug 不可能通过传错参数发生。显式开关意味着每一个调用点（三层 `forward_with_cache` 逐层往下传）都是一次犯错的机会，而且传错了没有任何检查。shape 推导把它降级成一个局部的、不依赖调用方的判断。代价是它对“ seq_q 恰好等于 seq_k 但语义上不是同一个 block”这种情形会判错——本项目里不存在这种调用，但这是这个技巧的真实边界。

代码位置： `cs336-basics/cs336\_basics/transformer/multiheads\_attention.py:169`（推导），`cs336-basics/cs336\_basics/transformer/multiheads\_attention.py:165` 起的三行注释写着为什么。mask 构造在 `cs336-basics/cs336\_basics/transformer/scaled\_dot\_prod\_attention.py:46-50`。

5.5.2 不变量 2：RoPE 必须按绝对位置旋转后再进 cache

上面那段代码的第 154–155 行注释说的就是这件事：“Append to the cache AFTER rotating: RoPE is applied once, at each token’s own absolute position, so cached keys must already carry their rotation.”

绝对位置是在最外层算的：

```

201     past_len = 0 if past_kv is None else past_kv[0][0].shape[-2]
202     seq_new = x.shape[1]
203     if past_len + seq_new > self.token_positions.shape[0]:
204         raise RuntimeError(
205             f"cached decode would reach position {past_len + seq_new}, beyond the "
206             f"model's context length {self.token_positions.shape[0]}"
207         )
208
209     x = self.in_embedding(x)
210     # Absolute positions continuing past the cache, so RoPE stays consistent with
211     # the cached keys were rotated by.
212     positions = self.token_positions[past_len:past_len + seq_new]
213     positions = positions.unsqueeze(0).expand(x.shape[0], -1).to(x.device)

```

`past_len` 从缓存的形状里读出来（第 201 行），`positions` 从 `past_len` 开始切（第 212 行）。所以第 n 步 decode 的那个 token 拿到的是位置 n ，不是位置 0。

Checkpoint 5.7 (rope_before_cache): 把旋转和入缓存的顺序颠倒，哪个测试会抓到

不看代码回答：

- (a) 如果 `positions` 写成 `self.token_positions[:seq_new]` (chunk 内相对位置)，`tests/test_kv_cache.py` 的 6 个测试里哪些会红？
- (b) 哪些不会红？为什么？
- (c) 这个 bug 在 prefill 阶段会不会暴露？

Hint prefill 时 `past_len` 是 0。

参考答案：这三问是写讲义时真的注入这个 bug 跑了一遍得到的，不是推理（做法见表 17 的 caption）。

(a) 两个会红：`test_greedy_decoding_is_token_identical` 和 `test_incremental_logits_match_full_forward_at_each_step`。

(b) 四个不会红。`test_prefill_matches_plain_forward` 不红是因为 prefill 时 `past_len=0`，`[:seq_new]` 和 `[0:seq_new]` 是同一个切片——bug 在 prefill 路径上根本不激活。`test_cache_grows_by_one_key_per_step` 和 `test_rejects_decoding_past_the_context_length` 只断言形状和边界，位置旋转对它们不可见。`test_batched_decoding_matches_per_sequence_decoding` 不红最值得注意的是：批量和逐条犯的是同一个错，两边一起错、错得一样，所以它们仍然相等。一个测试只能抓到破坏它所断言的那个对称性的 bug。

(c) 不会。这就是 (b) 的第一句。这个 bug 只在“缓存非空”时激活，所以任何只测 prefill 的检查都是安全通过的——包括 `kv_cache_gate.py` 的第一项 `check_logits_match`。门禁能挡住它，靠的是第二项 `check_greedy_match`。

代码位置：位置切片在 `cs336-basics/cs336\_basics/lm.py:212`；旋转-再入缓存的顺序在 `cs336-basics/cs336\_basics/transformer/multiheads\_attention.py:150–159`。

5.6 怎么测一个 cache

`tests/test_kv_cache.py` 有 6 个测试。它们不是六件事，是同一件事的六个角度：cache 是一个优化，所以唯一可接受的结果是它只改变速度。参照物是 `forward`，理由写在文件的 docstring 里：那是模型被训练时跑的那条代码，是唯一可信的基准。

写这份讲义时，我把这 6 个测试拿去做了一次变异测试：往 cache 实现里逐个注入五类真实的 bug，看哪些测试会红。结果如下。

注入的 bug	T1	T2	T3	T4	T5/T6
RoPE 按 chunk 内相对位置旋转	—	抓到	抓到	—	—/—
<code>is_causal</code> 硬编码 True	—	抓到	抓到	—	—/—
<code>is_causal</code> 硬编码 False	抓到	抓到	抓到	—	—/—
缓存 key 跨 batch 行串用	—	—	—	—	—/抓到
缓存 key 有 0.1% 数值偏差	—	—	抓到	—	—/—

Table 17: 五类注入 bug × 6 个测试。T1 = `test_prefill_matches_plain_forward`，T2 = `test_greedy_decoding_is_token_identical`，T3 = `test_incremental_logits_match_full_forward_at_each_step`，T4 = `test_cache_grows_by_one_key_per_step`，T5 = `test_rejects_decoding_past_the_context_length`，T6 = `test_batched_decoding_matches_per_sequence_decoding`。出处：写这份讲义时现做的实验，夜里没有做过——用 `monkeypatch` 替换 `MultiHeadsAttention`，`forward_with_cache` 和 `TransformerLM.forward_with_cache` 注入每一类 bug，再对未经修改的 `tests/test_kv_cache.py` 跑 `pytest`。本机 CPU，每轮约 0.15 秒。未注入时 6 passed。

从这张表能读出三件事，其中两件是设计时想到的，一件不是。

T4 和 T5 没有抓到任何一个注入的 bug。这不是它们没用——它们守的是形状和边界，而我注入的五类 bug 全是数值/语义的。一个把缓存拼错方向（dim=-1 而不是 dim=-2）的 bug 会立刻被 T4 抓到。但这张表确实说明：这套测试的实际防线是 T3 和 T6。

T6 是唯一能抓到批内串 key 的测试。其他五个测试全部用 batch=1，在那个维度上没有任何断言。

T3 比 T2 严格。最后一行是专门为验证这一点造的：给缓存里的 key 乘 1.001，logits 会动，但动得不足以翻转任何一个 argmax——于是 T2（比 token）全过，T3（比 logits）红。

Checkpoint 5.8 (stepwise_comparison): 为什么要逐步比对，而不是只看最终状态

不看代码回答：

- (a) T3 每一步都拿 `model(ids)` 重跑一遍完整前向来对照。这很慢，为什么值得？
- (b) 表 17 最后一行说明 T3 比 T2 敏感在哪？
- (c) 如果只留一个测试，留哪个？

Hint T3 的 docstring 自己给了答案的一半。

参考答案：(a) 因为“陈旧的 key”和“错位置的旋转”这两类 bug 的共同特点是要解码若干步之后才显形。只检查最终状态、而且是在一个短序列上检查，很可能整段都还没进入 bug 激活的区间。T3 的 docstring 原话是“a cache bug that only shows up deep into decoding (a stale key, a mis-rotated position) would pass an end-state check on a short run”。慢是真的慢（10 步就要跑 11 次完整前向），但这是一个 3 层 $d_{\text{model}}=64$ 的 CPU 模型，整份文件跑完不到 1 秒。
(b) 敏感在比较的对象是连续量而不是离散量。T2 比的是 argmax 之后的 token id，一个小到不足以翻转 argmax 的数值误差它看不见；T3 比的是 logits 本身，`atol=1e-4`。注入 0.1% 的 key 偏差时 T2 全过、T3 红，就是这个差别的直接证据。反过来说 T2 也不是多余的：它断言的是用户真正看得到的那个性质——两条路径生成的文本必须一模一样。T3 red 而 T2 green 时，你知道有数值漂移但输出还没变；两个都 red 时，问题已经跑到用户面前了。
(c) 留 T3。它是唯一一个既覆盖 prefill 又覆盖多步 decode、又比较连续量的测试。但留了 T3 之后就没有任何测试守着 batch 维度（表 17 第四行），所以实际上不能只留一个。

代码位置：tests/test_kv_cache.py:66。

```

66 @torch.no_grad()
67 def test_incremental_logits_match_full_forward_at_each_step(model):
68     """Step-by-step, not just at the end: a cache bug that only shows up deep into
69     decoding
70     (a stale key, a mis-rotated position) would pass an end-state check on a short run.
71     """
72     ids = torch.randint(0, 257, (1, 6))
73     logits, past = model.forward_with_cache(ids, past_kv=None)
74     for _ in range(10):
75         nxt = torch.argmax(logits[:, -1, :], dim=-1, keepdim=True)
76         ids = torch.cat([ids, nxt], dim=1)
77         logits, past = model.forward_with_cache(nxt, past_kv=past)
78         reference = model(ids)[:, -1, :]
79         assert torch.allclose(reference, logits[:, -1, :], atol=1e-4)

```

Checkpoint 5.9 (batched_decode_leak): 批内串 key 是什么样的 bug

不看代码回答：

- (a) T6 拿 batch=2 的解码结果和逐条解码的结果比。这能抓到什么？
- (b) 为什么其他五个测试抓不到？
- (c) 本项目的 LMEngine 目前一次只处理一个请求，为什么还要写这个测试？

参考答案：(a) 抓到任何“缓存记账不是逐序列的”错误：把一行的 K 广播给整批、reshape 时把 batch 和 head 两个维度搞混、cache 拼接用错了 dim。这一类 bug 的表现是 batch 里第 i 条请求看到了第 j 条请求的历史——在真实 serving 里这既是错误也是数据泄漏。变异实验里我注入的是最直白的一种 (`past_k[0:1].expand_as(past_k)`)，只有 T6 变红。
 (b) 因为其他五个都是 batch=1。batch=1 时“跨行串用”和“不串用”是同一件事，没有可以被破坏的对称性。这是表 17 想说的核心：测试能抓到的，只有它所断言的那个不变量被破坏的情形，再多的测试也不会自动覆盖一个没人断言过的维度。
 (c) 因为这是给还没写的那个调度器提前立的规矩。llm-serving 的路线图上 continuous batching 是明确的下一步（见姊妹篇 [Assignment A §4.2 <continuous batching 的定义，以及它对 scheduler 位置的裁决>](#) 和 [Assignment A §4.3 <backend 约束把设计逼回来：stage-homogeneous batching>](#)）。到那时候 batch 维度会突然变成热路径，而那时再补这个测试，就得先在一个已经变复杂的系统里定位 bug。现在写，成本是十几行。

代码位置：tests/test_kv_cache.py:104。

Downscaling Tip: 不用 GPU 也能验 cache

tests/test_kv_cache.py 整份文件用的是 CPU 上一个 vocab_size=257, context_length=64, num_layers=3, d_model=64 的模型。本机实测 6 passed in 0.96s（首次运行含 import；变异实验里热跑是 0.15 秒）。

这不是“降配将就”，这是这类测试的正确规模：cache 的正确性和模型规模无关。它断言的是一个代数等式——带缓存的前向必须和不带缓存的前向给出同样的数——而这个等式在 $d_{\text{model}}=64$ 和 $d_{\text{model}}=768$ 上是同一个等式。规模只影响性能结论（§5.7 那张表就必须在真 GPU、真权重上测），不影响正确性结论。

顺带说，这条性质是免费拿到的吗？不是。cs336_basics 里有 8 个文件无条件 import torch.cuda.nvtxasnvtx 并用 @nvtx.range(...) 标注 forward，在非 CUDA build 上导入正常、一调用就 RuntimeError。是先加了 cs336_basics/nvtx_compat.py（import 时探测一次，不可用就退化到 no-op），TransformerLM 才能在这台笔记本上跑起来。

Resource requirements: CPU 即可；uvrunpytesttests/test_kv_cache.py，约 1 秒。

除了进常规测试套件的这 6 个测试，还有一个一次性的门禁脚本 cs336_systems/experiments/kv_cache_gate.py：缓存解码必须和重算解码 logits 逐位相同 + 贪心 token 完全一致，否则 sys.exit(1)，调用方不许启用 cache。

Example 5.2 (kv_gate_output): 门禁的真实输出

夜间窗口 stage 7 的日志原文（run_logs/post_train_window.log:1123–1132）：

```
[kv] gating against checkpoints_pretrain_v1/checkpoint_epoch_0100.pt
=== gate 1: small random model ===
  prefill logits max|delta| = 0.000e+00 -> PASS
  greedy tokens identical    -> PASS
```

```
=== gate 2: real trained model ===
  prefill logits max|delta| = 0.000e+00 -> PASS
  greedy tokens identical    -> PASS
```

GATE PASSED: cached decode is token-identical to recompute.

0.000e+00 是逐位相同，不是“在容差内”。这比预期更强：门禁自己的容差是 $\text{tol}=2\text{e-}4$ ，脚本 docstring 里也写着“不同的 reduction order 产生微小 logit 差异是预期的”。实际拿到 0 是因为 prefill 路径上 forward_with_cache 和 forward 做的是同样顺序的同样运算（past_kv=None 时那两个分支的算子序列一致），所以浮点结果逐位一致。真正需要容差的是 decode 步——但那里比的是 token 不是 logits。

Checkpoint 5.10 (gate_with_final_weights): 为什么门禁要拿最终权重再跑一次

不看代码回答：

- (a) 小随机模型已经过了，为什么还要拿 190M 的真 checkpoint 再过一次？
- (b) 门禁脚本和 tests/test_kv_cache.py 有什么分工？为什么不合并？

参考答案: (a) 因为拿别的权重过的门禁不算这批权重的门禁。原则上 cache 的正确性和权重无关 (§5.6 的 tipbox 就是这么论证的)，但“原则上无关”和“这批要被报告数字的权重上确实无关”是两个命题，后者只要 30 秒就能验，没有理由不验。更实际的理由是：真 checkpoint 会额外走一遍 `k.removeprefix("_orig_mod.")` 的加载路径、真实的 `context_length=1024` 的 RoPE 表、真实的 16 层深度——这些都是小模型上不存在的东西。门禁通过之后才允许 benchmark 报缓存的数字，顺序在夜间脚本里是硬的：gate 失败则 `sys.exit(1)`，后面的 `bench_serving.py` 不会跑。

(b) 分工是“常驻回归”与“一次性放行”。`test_kv_cache.py` 进常规套件，每次 `uv run pytest` 都跑，用小模型、跑得起，防的是将来的改动打破 cache。`kv_cache_gate.py` 是一个带 `--checkpoint` 参数的脚本，用真权重、在 GPU 机器上跑，防的是这一次要报的数字建立在没验过的路径上。合并的话要么让常规套件依赖一个 2.3GB 的 checkpoint（跑不起来），要么让门禁失去“针对特定权重”的语义。

代码位置: `cs336/systems/experiments/kv_cache_gate.py:94` (gate 2 分支)、`cs336/systems/experiments/kv_cache_gate.py:118` (失败即退出)。

5.7 KV cache 的收益和预期完全不一样

门禁过了，可以报数字了。

五个加速比: 1.20/1.28/1.20/1.27/1.23。均值 $1.24\times$ 。prompt 长度涨了 32 倍，加速比几乎不动（末项比首项 = 1.02）。

这和事前的断言完全相反。事前的断言写在下面这个框里，原样保留：

撤回 (retracted): 加速比会随 prompt 长度增长，因为重算是 0(prefix) 而 cache 是 0(1)

这句话的前半段推理是对的——重算解码每步确实要跑完整个前缀，cache 每步确实只跑一个位置。错的是从这个渐进关系直接跳到“所以在这台机器、这个模型上，加速比会随长度增长”。渐进复杂度描述的是当 prefix 大到能主导总成本时的行为；它不告诉你在 `prefix = 512`、模型 190M、`batch = 1` 的时候，prefix 相关的那部分占总成本的多少。实测占比很小，所以加速比是平的。这条被 `artifacts/serving_benchmark_kv.json` 的五

prompt tokens	recompute			cache			加速比
	TTFT ms	p50 ms	tok/s	TTFT ms	p50 ms	tok/s	
16	61.3	63.03	15.9	62.3	52.45	19.2	1.20×
64	65.7	61.25	16.4	63.0	48.06	21.0	1.28×
128	61.8	62.79	15.7	59.3	52.78	18.9	1.20×
256	65.3	64.08	15.6	61.2	51.80	19.8	1.27×
512	64.5	64.82	15.6	61.5	52.12	19.2	1.23×

Table 18: recompute 与真 KV cache 的完整对照。出处：artifacts/serving_benchmark_kv.json，单卡 cuda:0、0.19B 模型、fp32、batch 1、new_tokens=64（即每格 63 个 decode 步）、两种模式用同一个 torch.Generator().manual_seed(0) 生成逐字节相同的 prompt、2 次 warmup。p50 是 63 个 step 时间的中位数。加速比 = cache tok/s ÷ recompute tok/s，取自 JSON 的 cache_speedup_by_prompt_len 字段。LMEngine 端到端往返（22 token）：recompute 1.36 s，cache 1.14 s。

个数直接否掉，撤回。

原因不难说清楚，但要说得准确需要一点算术。recompute 模式下，prompt 从 16 涨到 512（32 倍）时，每步 p50 从 63.03 ms 涨到 64.82 ms——只涨了 2.8%。也就是说连重算本身都几乎不随前缀长度增长。在这个规模上，每一步的成本几乎全部是“把 16 层权重从头到尾过一遍”这个固定开销，而 KV cache 消掉的那部分（前缀相关的 attention 计算和它的 QKV 投影）只是其中一小块。

这个固定开销到底是什么，值得记一笔没有查证到底的地方。夜报里的措辞是“dominated by the fixed forward pass over the weights”，暗示是访存。粗算一下：0.19B 参数 × 4 字节 = 760 MB，RTX 3090 的显存带宽约 936 GB/s，把权重读一遍的下界是约 0.8 ms；而实测每步 60 ms，是这个下界的约 70 倍。计算量更不是瓶颈（batch 1 单 token 约 0.38 GFLOP，在 fp32 的 3090 上是 0.01 ms 量级）。所以这 60 ms 里绝大部分既不是访存也不是算力，更像是每层十几次小 kernel launch、einops 的 rearrange、Python 层调度的累加。这只是一个数量级估算，没有做过 profiling，所以只能作为一个待查的开放问题写在这里。但它不影响本节的结论：不论那 60 ms 具体花在哪，它都是与前缀长度无关的固定成本，而这正是加速比平坦的直接原因。

还有一件必须说的：这张表跑的是朴素的 PyTorch attention，不是本项目的 Triton kernel。bench_serving.build_adapted_model 构造 TransformerLM 时没有传 attention_fn，走的是默认值 scaled_dot_product_attention（cs336-basics/cs336_basics/lm.py:131）；而 forward_with_cache 那条路径更是直接写死调用 scaled_dot_product_attention（cs336-basics/cs336_basics/transformer/multiheads_attention.py:170），连 self.attention_fn 都不看。两种模式因此是同一个 attention 实现，比较是公平的；但这张表和 Triton kernel 无关，不能拿它去说 kernel 的任何事。

Checkpoint 5.11 (flat_speedup): 加速比为什么是平的

不看代码回答：

- 在什么条件下 KV cache 才会给出教科书上那种数量级的加速？
- 表 18 里哪一列最能说明“重算不贵”？
- 1.24× 这个数字，能不能写进简历说“实现了 KV cache，decode 提速 24%”？

参考答案：(a) 当前缀相关的工作在每步成本里占比变大的时候，也就是：上下文更长（attention 是 $O(n)$ 每步，权重那一遍是 $O(1)$ ， n 大到一定程度必然反超）、batch 更大（权重那一遍在整个 batch 上摊薄，而 KV 的量随 batch 线性增长）、或者模型更小。这三条在这次

的配置里全是反的： $n \leq 576$ 、 $\text{batch} = 1$ 、190M 参数 fp32。KV cache 在这个规模上不划算——这是个比“cache 快 N 倍”更有信息量的结论。

(b) recompute 那一列的 p50: 63.03 $\rightarrow$ 61.25 $\rightarrow$ 62.79 $\rightarrow$ 64.08 $\rightarrow$ 64.82 ms。前缀涨了 32 倍，每步只贵了 2.8%。如果重算真的贵，这一列应该是明显上翘的。(JSON 里 `decode_cost_growth.latency_ratio` 字段记的就是这个比值: 1.028。)

(c) 可以，但必须带上后半句：“在 190M/fp32/batch=1/上下文 ≤ 576 的条件下测得 $1.24\times$ ，而且加速比不随 prompt 长度增长，因为这个规模下每步成本由固定的权重前向主导。”只报 $1.24\times$ 会让人以为这是个一般结论；只报“实现了 KV cache”则回避了最有意思的部分。真正值得讲的是“我预期它随长度增长，实测是平的，我去查了为什么”这个过程。

5.7.1 一个在读自己数据之前就写好结论的 benchmark

上面那句被撤回的话，不是我在报告里随手写的一句话。它是 `bench_serving.py` 里一个无条件 print 出来的硬编码结论，不看任何实际数字。`make_night_report.py` 里也有同一句的 markdown 版本。

后果在夜间日志里留着，原样引在这里 (`run_logs/post_train_window.log:1167-1175`):

```
=== recompute vs KV cache ===
prompt    recompute tok/s    cache tok/s    speedup
    16              15.9         19.2         1.20x
    64              16.4         21.0         1.28x
   128              15.7         18.9         1.20x
   256              15.6         19.8         1.27x
   512              15.6         19.2         1.23x
```

The speedup grows with prompt length because recompute is $O(\text{prefix})$ per step and the cache is $O(1)$ in the prefix — that widening gap IS the cost of not having a cache.

一张明显没有趋势的表，紧接着一行断言它在增长。两处都改成了按实测趋势再措辞： $> 1.25\times$ 说“增长”， $< 0.8\times$ 说“反常，需要查”，否则说“基本持平在 $\sim X\times$ ”。修复后的代码和它的注释在本仓库里，可以直接引：

```
181     # Describe what was measured. An earlier version printed "the speedup grows with
182     # prompt length" unconditionally -- which the first real run promptly
    contradicted
183     # (a flat  $\sim 1.2\times$  across a  $32\times$  range). A benchmark that states its conclusion
    before
184     # reading its own numbers is worse than no benchmark.
185     if len(speedups) >= 2:
186         ordered = [speedups[n] for n in sorted(speedups, key=int)]
187         trend = ordered[-1] / ordered[0] if ordered[0] else float("nan")
188         results["cache_speedup_trend_last_over_first"] = trend
189         if trend > 1.25:
190             print(f"\nCache speedup GROWS with prompt length ({ordered[0]:.2f}x -> "
191                   f"{ordered[-1]:.2f}x): the prefix-dependent recompute is a material
    "
192                   f"share of step cost at these lengths.")
193         elif trend < 0.8:
194             print(f"\nCache speedup SHRINKS with prompt length ({ordered[0]:.2f}x ->
    "
195                   f"{ordered[-1]:.2f}x), which is not the expected direction -- worth
    "
196                   f"investigating before quoting these numbers.")
197         else:
198             mean = sum(ordered) / len(ordered)
199             print(f"\nCache speedup is roughly FLAT at  $\sim$ {mean:.2f}x across "
```

```

200         f"{min(speedups, key=int)}--{max(speedups, key=int)} prompt tokens "
201         f"(not growing with length). At this model size the per-step cost
    is "
202         f"dominated by the fixed forward pass over the weights, so the "
203         f"prefix-dependent attention work the cache eliminates is only a "
204         f"small share of it. A cache pays off far more at longer contexts "
205         f"and/or larger batch, where that share is bigger.")

```

第 183–184 行那句“A benchmark that states its conclusion before reading its own numbers is worse than no benchmark”是这一整章里最该记住的一句。

Checkpoint 5.12 (benchmark_wrote_conclusion_first): 为什么这类 bug 比测错数字更严重

不看代码回答：

- (a) 一个 benchmark 把数字测错了，和一个 benchmark 数字测对了但结论是硬编码的，哪个更危险？
- (b) 修复为什么要写成三个分支（增长 / 反常 / 持平），而不是简单地把那句话删掉？
- (c) 这个 bug 是怎么被发现的？如果没被发现，会怎样？

参考答案：(a) 后者。测错数字的 benchmark 迟早会和别的证据对不上，而且它的失败是随机方向的。硬编码结论的 benchmark 每一次运行都朝同一个方向撒谎，而且它撒的谎恰好是你最想听的那一句——因为那句话正是你写它的时候相信的。它把 benchmark 从“检验假设的工具”降级成“把假设重复一遍的机器”。更糟的是它看起来在工作：表格是真的，数字是真的，只有那一行结论是假的。

(b) 因为删掉之后，读日志的人要自己从五个数里看出趋势，而这正是当初想省掉的那一步。三个分支保留了“脚本替你读表”这个便利，同时把“读的结果”和“数据”绑定起来。更重要的是 $<0.8\times$ 那个分支：它守的是一个我不希望看到但必须能看到的情形（cache 比重算还慢，说明实现有问题），措辞是“worth investigating before quoting these numbers”。一个只会说好消息的脚本，和一个硬编码结论的脚本，性质是一样的。

(c) 是在把表格和结论并排读的时候发现的——五个数明显没趋势，下一行说它在增长，荒唐到不用算。如果没发现，NIGHT_REPORT.md 里会留下一句和自己表格矛盾的结论，而这份报告是给别人看的。同一个晚上还在 bucket size 扫描的汇总代码里犯了同一类错（直接 `print "best: 50MB"`，见 §6），说明这不是一次手滑，是一种习惯：把“我预期会看到什么”写进了本该报告“我看到了什么”的地方。

代码位置：cs336_systems/experiments/bench_serving.py:181（注释与三分支）、cs336_systems/experiments/make_night_report.py:166（同一处修复的 markdown 版本）。

常见误解：artifacts/serving_benchmark_kv.json 是修复后的脚本产出的，所以它是干净的。

不是。那个 JSON 写于 2026-07-27 23:58（日志时间戳），修复是之后做的。证据：修复后的代码会往结果里写一个 `cache_speedup_trend_last_over_first` 字段（cs336_systems/experiments/bench_serving.py:188），而磁盘上那份 JSON 的顶层 key 只有 `device/new_tokens/modes/cache_speedup_by_prompt_len`，没有那个字段。

这不影响任何数字——五个加速比是同一次测量的产物，修复只动了措辞逻辑。但它意味着那份 artifact 是用有 bug 的脚本产出的，NIGHT_REPORT.md 里正确的“roughly flat at $1.24\times$ ”那句话来自 `make_night_report.py` 事后重新生成。这一条写在这里，是因为“结论对了”和“产出结论的代码对了”是两件事，而这一章刚好在讲这个区别。

5.8 端到端：链路里除 tokenizer 外没有 HuggingFace 模型

tests/trained_model_integration_test.py 走完整条链路: RequestReceiver → pending_queue → InferenceEngine → LMEngine → TransformerLMAdapter → 自训权重。2026-07-27 夜里在 GPU 机器上真跑通了, 2 passed。输出:

```
prompt:    The history of the internet began in the
continued: late 19th century, and the first wave of internet-connected websites
           emerged in the 20th century.
```

语法通顺、切题, 事实是错的 (还输出过 "Paris is the capital ... of the United States"), 并且开始复读。一个 190M 参数、只训了 2.8 token/param 的模型就该是这个样子, 不要把它说得比实际好。这段输出记录在 interview_prep/PROGRESS.md; 本机没有对应的测试日志, 属于核不到出处的一项。

链路里除了 tokenizer, 没有任何 HuggingFace 模型。而 tokenizer 之所以能直接用 HF 的, 是因为训练时用的 tiktoken.get_encoding("gpt2") 和 HF 的 gpt2 tokenizer 是 token-for-token 相同的。

Checkpoint 5.13 (tokenizer_compat): 凭什么说两个 tokenizer 是同一个

不看代码回答:

- (a) 训练用 tiktoken、serving 用 AutoTokenizer, 怎么保证 token id 对得上?
- (b) 如果不对得上, 会表现成什么样? 会不会报错?
- (c) 项目里为什么最后没有自己训 BPE?

参考答案: (a) 两者是同一份 GPT-2 BPE: 50257 个词表项, <|endoftext|> 都是 50256。本机核对过 tiktoken 这一半: tiktoken.get_encoding("gpt2") 的 n_vocab=50257、eot_token=50256, 和 adapter 里那个 GPT2_EOS_TOKEN_ID = 50256 常量一致 (llm-serving/inferenceLM/engine/transformer_lm_adapter.py:47)。HF 那一半本机没有 transformers 可以核对, 所以 "token-for-token 相同" 这个说法目前的依据是 adapter docstring 第 35–38 行和集成测试第 75 行的注释, 加上两边都声称是 GPT-2 的原始 BPE。真正把它验成事实的是集成测试本身: 如果 id 不一致, 用 HF tokenizer 编码的 prompt 喂进按 tiktoken 训的模型, 输出会是乱码而不是通顺英文。

(b) 不会报错, 因为两边 vocab size 相同, 任何 id 都在合法范围内。表现是模型输出彻底的胡言乱语——但一个欠训的 190M 模型输出本来就不太好, 这两种 "不好" 很容易混淆。这正是为什么集成测试要 print 出续写文本让人眼看, 而不只是断言 len(generated) > 0。

(c) 完整的理由属于预训练那一章 (§3.2): 自训 BPE 在真实混合语料上慢到会整个时间预算吃光, 于是换成 tiktoken 的预训练 gpt2 编码, 780MB + 20MB 语料 92 秒编码完 (run_logs/tiktoken_encode.log: 86.0 s + 5.9 s)。这里只记它顺带的收益: serving 端因此不需要分发一份自定义 tokenizer, RequestReceiver 的 AutoTokenizer 直接指向 "gpt2" 就行, 一行都不用改。

代码位置: llm-serving/tests/trained_model_integration_test.py:75。

5.9 环境这一坑

最后记一件纯工程的事, 因为它花掉的时间不比上面任何一节少。

要在 GPU 机器上同时跑两个仓库的代码, 第一反应是给 llm-serving 单独 uv sync 一份 venv。查了之后发现两边 venv 各缺一半:

- llm-serving 没有 einops 和 tiktoken, 而这两个是 cs336_basics 的必需依赖;

- 本项目没有 transformers 和 pytest-asyncio，而 llm-serving 的测试全靠它们。

而且 llm-serving 的 pyproject.toml pin 的是 torch>=2.11，单独装会再拉一份约 3GB 的 torch（本项目跑的是已经验证过的 2.6），而那台实例的根分区只剩约 10GB。

最后的解法是不建第二个 venv：复用本项目的 venv，用 `uv run --with transformers --with pytest-asyncio` 临时补上缺的两个包，再用 `PYTHONPATH=~/.llm-serving` 让 inferenceLM 可导入。不改任何 pyproject.toml，torch 版本不变。

```
PYTHONPATH=~/.llm-serving uv run --with transformers --with pytest-asyncio \
python cs336_systems/experiments/bench_serving.py \
    --config cs336_systems/experiments/real_pretrain_config.json \
    --checkpoint checkpoints_pretrain_v1/checkpoint_epoch_0100.pt \
    --modes recompute cache
```

这里有一个反直觉的细节，踩过一次才知道：`uv pip install` 装的包活不过下一次 `uv run`。`uv run` 会把 venv 重新同步回 lockfile，把手动装的包清掉。所以必须用 `--with`（它每次都重新解析这几个额外依赖），而不是先 `uv pip install` 再 `uv run`。

这一坑之所以值得写进讲义，是因为它是“两个仓库解耦”这个设计决定的账单。§5.2 说 adapter 不 import cs336_basics 换来了三个好处；代价就在这里——总有一个地方要把两边拼起来，那个地方是运行环境，而运行环境比 import 语句难 debug 得多。这个账单是划算的，但把它记下来比假装它不存在诚实。

6 系统 benchmark 与测量噪声

这一章要讲的，是整个冲刺里最想写进简历的一句话：“自研 bucketed overlapped DDP 比 naive DDP 快 X%”这句话我们先写错了一次，然后用了一整轮复测才搞清楚自己有没有资格说它。所以本章的主线不是“我们的 DDP 有多快”，而是：一个百分比的差异，要满足什么条件才配叫结论。

前四章有一个共同的便利：它们比的是 loss，而 loss 是配对打分的——同一批 batch、同一个固定 seed，两个 checkpoint 之间 0.005 的差都能分辨 (§4)。这一章比的是 wall-clock。同一套代码、同一个配置，换个观测量，可信度整个塌一层。

6.1 第一版矩阵表

想回答的问题有四个，一张表就能同时覆盖：朴素 $O(N^2)$ attention 在真实 190M 模型上还跑不跑得动、Triton FlashAttention 比 vectorized PyTorch 快多少、自研 bucketed DDP 相对 naive DDP 和 PyTorch 官方 DDP 什么位置、以及 bucket 的额外显存要多少。驱动脚本是 benchmark_lm_matrix.py，它对 kernels 与 wrappers 做笛卡尔积，每格跑 3 个不计时的 warmup epoch、再跑 3 个计时 epoch，每 epoch 固定 51 步 (cs336_systems/experiments/benchmark_lm_matrix.py:66、...:67、...:232)。计时用的 wall-clock 在 rank 之间取 MAX 而不是平均，所以 tok/s 是按最慢的那张卡算的，不会被快卡拉高 (...:288)。

Kernel	Local (1 GPU)	Naive DDP	Bucketed (自研)	PyTorch DDP
scaled_dot_prod_attention	OOM	OOM	OOM	OOM
vectorized_torch	5213.3	7344.7	7706.3	OOM
flash_attention_triton	5309.2	8188.7	8398.0	8532.3

Table 19: 第一版矩阵，单位 tok/s，真实 0.19B 模型 / ctx 1024 / batch 8 每卡 / 2×RTX 3090。每格只跑了一次。flash_attention_triton 那一行的 per-GPU 峰值显存依次是 20003.9 / 20004.7 / 20745.0 / 20731.4 MB，即 bucket 的额外显存是 +741.1 MB (+3.7%)。数据源 artifacts/bench_matrix/lm_matrix_results.csv。

6.2 撤回：每格只跑了一次

这张表当晚就被当成结论写出去了。第二天做 bucket size 复测时才第一次量到自己的测量噪声——同一个配置连测三次是 9590 / 9125 / 8553 tok/s，围绕均值上下约 $\pm 6\%$ 。更难看的是隔夜重跑：同一个 Naive DDP 配置，前一晚 8188.7，第二天 8629，差 5.4%，比表里报的任何一个“结论”都大。

撤回 (retracted): Bucketed 比 Naive 快 2.6%；PyTorch DDP 比 Bucketed 快 1.6%

这两句话来自上表里 8398.0 / 8188.7 / 8532.3 三个数，每个都是单次测量。已量到的单次 run-to-run 噪声是 $\pm 6\%$ (bucket 复测) 到 CV 9.5% (5-rep 复测的非配对散布)。两个独立单次测量作商，相对标准差还要再乘 $\sqrt{2}$ ，也就是 8.5%–13%。2.6% 大约是 0.3σ ，1.6% 更小。两条都撤回。vectorized_torch 那行的“Bucketed 快 4.9%”同样撤回。

6.3 哪些格子仍然成立

撤回不等于全废。同一张表里，结论的类型不同，抗噪声的能力差好几个数量级。

完全不受影响的是二值结论。scaled_dot_prod_attention 那一行四格全部 OOM，错误信息在 CSV 里逐条留着 (artifacts/bench_matrix/lm_matrix_results.csv:2)，“跑不跑得动”不是一个带误差棒的量。同理，vectorized_torch 那格 PyTorch DDP 也 OOM，而 Triton 那格能跑——

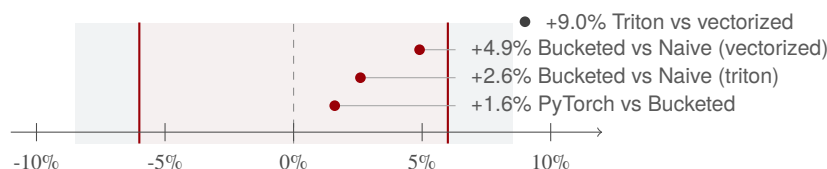


Figure 1: 把想报的效应放到量到的噪声带上。红带是单次测量本身的 $\pm 6\%$ ，灰带是两次独立单次测量作商的 1σ ($\pm 8.5\%$)。三个 DDP 差异全部落在红带内，跨 kernel 的 +9.0% 只是刚探出灰带。作为对照，TF32 on/off 是 +43%（远在画面右侧），那种量级单跑一对就够。

只有 Triton 省下的显存余量够 PyTorch DDP 起来。这类结论和 [Assignment A §5.1 <差分测试：让参照实现来定义“正确”>](#) 是同一种形状：答案是 0 或 1，噪声进不来。

被我们高估了的是跨 kernel 的百分比。当时的说法是“跨 kernel 的差距远大于噪声”。重新算一遍：Triton 相对 vectorized 的领先，在三种 wrapper 下分别是 +1.8% (Local)、+11.5% (Naive)、+9.0% (Bucketed)。三个观测同向，但幅度差了 6 倍，而且中位数 +9.0% 恰好压在噪声带边缘。更糟的是这三对根本不是干净的配对——脚本的循环是 `for kernel: for wrapper:` (`cs336/_systems/experiments/benchmark/_lm/_matrix.py:710`)，两个 kernel 是按块跑的，中间隔了整整四格。这正是配对分析最怕的情形：条件和时间漂移被绑在一起。所以这一条应该降级成“方向可能对，幅度不可信”，而不是像当初那样当成安全区。

Checkpoint 6.1 (know_your_noise): 在报一个 2% 的差异之前，你需要先知道什么

不看代码回答：

- 一张 3×4 的 benchmark 矩阵，每格跑一次。这张表能支撑的最小可报差异是多大？要先做什么实验才知道？
- 已知单次测量的相对标准差是 σ 。两个独立单次测量作商，这个比值的相对标准差是多少？

Hint 噪声不是从表里读出来的，它只能另外测——固定配置重复跑，看散布。

参考答案: (a) 无法从表本身判断，必须先做重复测量。我们后来量到单次噪声约 $\pm 6\%$ ，于是这张表能支撑的只有二值结论 (OOM/不 OOM) 和远大于 10% 的差异。(b) 约 $\sqrt{2}\sigma$ 。 $\sigma = 6\%$ 时是 8.5%，所以两个数字差 2.6% 时几乎没有信息量。这就是为什么撤回的门槛看起来比直觉高得多。

代码位置: `artifacts/bench/_bucket/_50mb/lm/_matrix/_results.csv:2`、`interview/_prep/PROGRESS.md:142`

Checkpoint 6.2 (which_comparisons_survive): 同一张表里，哪些格子撤回、哪些不撤回

不看代码回答：

- 为什么“朴素 attention 全部 OOM”不需要重复测量就能报？
- 脚本的外层循环是 kernel-major 的。这对“Triton 比 vectorized 快 9%”这条结论有什么影响？

Hint 一个是分类问题，一个是估计问题。

参考答案: (a) 它是二值的: 进程要么完成要么抛 `torch.OutOfMemoryError`, 没有分布可言。时间噪声不会把一次 OOM 变成不 OOM。(b) 两个 kernel 按块跑, kernel 和时间漂移完全混淆, 所以那三个百分比不是配对观测。方向上 3/3 同向 (+1.8% / +11.5% / +9.0%), 但 $n = 3$ 的双侧 sign test 最小 p 就是 0.25, 形式上依然不显著。

代码位置: `cs336\_systems/experiments/benchmark\_lm\_matrix.py:710`

6.4 复测设计: 5 轮 × 3 wrapper, 按 rep 交替

复测只改了一件事: 执行顺序。不是把 Naive 连跑 5 次再把 Bucketed 连跑 5 次 (wrapper-major), 而是每一轮里三种 wrapper 各跑一次、跑完再进下一轮 (rep-major), 结果按 `rep<N>_.../` 分目录落盘, 分析脚本按目录名解析轮次 (`cs336\_systems/experiments/analyze\_ddp\_replicate.py:29`)。

顺序为什么是设计的一部分: 机器在几十分钟的尺度上会漂 (温度、显存碎片、page cache 冷热)。wrapper-major 的排法会让“策略”和“时刻”完全共线, 第 1 个策略拿到的是冷机器、第 3 个拿到的是热机器, 差多少都说不清是谁的。rep-major 让同一轮内的三次运行在时间上相邻, 共享同一时刻的漂移, 轮内作差就能把这部分消掉。脚本的 docstring 把这个前提写死了: 如果按 wrapper 分块跑, 这套分析直接失效 (`cs336\_systems/experiments/analyze\_ddp\_replicate.py:14`)。

Downscaling Tip: 没有 GPU 也能复现这一章的后半段

本章从 §6.5 起用到的只有 15 个 tok/s 数字。把表 20 的逐轮百分比抄成一个 Python list, 照 `cs336\_systems/experiments/analyze\_ddp\_replicate.py:88` 那 12 行把 sign test 敲一遍, 能原样复现 $p = 0.062$; 把 $2 * se$ 那行敲一遍能复现 $2SE=10.2\%$ 。统计部分和 GPU 没有任何关系, 这也正是它值得单独写成脚本的原因。

Resource requirements: CPU 即可; 不到 1 分钟。

6.5 三种检验, 三个结论

复测跑完, 同一批数据被三种方法各分析了一遍, 得到的结论不一样。这是本章最容易被拿去作坏事的地方, 所以三个都列出来。

	Bucketed vs Naive	PyTorch vs Bucketed	PyTorch vs Naive
逐轮领先	+1.1 / +6.5 / +4.3 / +29.9 / +9.3%	—	—
胜场	5/5	4/5 (偏向 Bucketed)	3/5
中位数	+6.5%	-2.75%	—
均值 vs 2SE	+10.3% vs 10.2% (勉强过)	—	+10.4% (过)
sign test 双侧 p	0.062	0.375	0.375
结论	可以报	分辨不出	两种方法打架, 不报

Table 20: 5 reps × 3 wrapper 复测的三种检验对照。“—”表示这一格没有被记录下来: 复测的原始 CSV (`artifacts/ddp_replicate/`) 没有同步回本地, 我们无法重算, 所以宁可留空也不填一个推出来的数。逐轮百分比与胜场、中位数、 p 值出自 `interview_prep/PROGRESS.md`。

三种检验各自在回答不同的问题:

非配对 Welch 问的是“两个 wrapper 的总体均值差多少”。它把 rep 配对结构丢掉了, 于是 9.5% 的 run-to-run 方差直接淹掉信号。脚本里旧版给出的结论是“ $-11.1\% \pm 11.7\%$, within noise, 分辨不出”。这个结论没有错, 只是问错了问题。

配对 t 问的是“轮内差值的均值是否显著非零”。判据写得很克制: 只判断均值有没有越过 2 个标准误, 不叫它 significance (`cs336\_systems/experiments/analyze\_ddp\_replicate.py:77`)。

Bucketed vs Naive 的均值 10.3% 对 2SE 10.2%——刚好压线，而且这条线完全靠 r4 那个 +29.9% 撑着。

sign test 问的是“B 是不是一贯比 A 快”。它只看符号，不看幅度，所以离群值动不了它。实现是精确的二项尾部，不是正态近似：

```

88     pos = sum(1 for d in diffs if d > 0)
89     n = len(diffs)
90
91     def _comb(n, k):
92         num = den = 1
93         for i in range(k):
94             num *= n - i
95             den *= i + 1
96         return num // den
97
98     tail = sum(_comb(n, k) for k in range(min(pos, n - pos) + 1))
99     p = min(1.0, 2 * tail / (2 ** n))

```

Checkpoint 6.3 (paired_vs_unpaired): 同一批数字，配对和非配对为什么给出相反的结论

不看代码回答：

- (a) 非配对检验丢掉了什么信息？在什么前提下丢掉它是合理的？
- (b) 三种检验结论不一致时，为什么“挑一个自己喜欢的报”比只做一种检验更糟？

Hint 配对的前提是执行顺序，不是统计选择。

参考答案：(a) 丢掉了“哪几次运行在时间上相邻”这个信息。只有当运行顺序本身是随机化的、没有共享漂移时，丢掉它才不亏。我们的顺序是刻意 rep-major 排的，丢掉就白排了。(b) 只做一种检验是无知；做了三种再挑一种报，是在已知它们不一致的情况下隐瞒。正确做法是三种都列，并说明为什么采信其中一种——这里采信 sign test，因为它不依赖 r4 那个 +29.9% 的离群值，而配对 t 完全靠它才勉强过线。

代码位置：cs336_systems/experiments/analyze_ddp_replicate.py:71、...:88

6.6 样本量的天花板：这个实验从设计上就到不了 $p < 0.05$

这是全章最重要的一条。双侧 sign test 在 n 轮全胜时的精确 p 值是

$$p = 2 \cdot \frac{\binom{n}{0}}{2^n} = 2^{1-n}.$$

$n = 5$ 代入得 $p = 2/32 = 0.0625$ 。也就是说：哪怕 5/5 全胜，这个实验能拿到的最小 p 也是 0.0625，永远越不过 0.05。要越过至少需要 $n = 6$ ($2^{-5} = 0.031$)。

这件事的实际含义是： $p = 0.062$ 不是“接近显著、再努力一点就能过”，而是已经到顶了。样本量的天花板应该在设计实验的时候算，不是在看到 p 值之后再算——后者的顺序会诱导人去补跑第 6 轮，而补跑的动机来自已经看到的结果，那是标准的 p -hacking。

Checkpoint 6.4 (n5_ceiling): 为什么 $n=5$ 时 $p < 0.05$ 是不可能的

不看代码回答：

- (a) 写出双侧 sign test 在 n 轮全胜下的 p 值表达式，并算出 $n = 4, 5, 6$ 的值。

(b) 如果实验做完看到 $p = 0.062$ ，然后决定“再补一轮凑 $n = 6$ ”，问题出在哪？

Hint 在零假设下，每一轮的符号是独立的公平硬币。

参考答案: (a) $p = 2^{1-n}$: $n = 4$ 给 0.125, $n = 5$ 给 0.0625, $n = 6$ 给 0.031。所以 $n \leq 5$ 的双侧 sign test 无论如何达不到 0.05。 (b) 停止规则依赖于已观测的结果，实际的第一类错误率会高于名义值。补轮次本身没错，错的是看到 p 之后才决定补。样本量该在跑之前定。

代码位置: cs336_systems/experiments/analyze_ddp_replicate.py:98

6.7 唯一站得住的一条

在上面两个前提之下，能报的只有一条：Bucketed Overlapping DDP 快于 Naive DDP，逐轮 +1.1 / +6.5 / +4.3 / +29.9 / +9.3%，5/5 全胜，sign test 双侧 $p = 0.062$ （该样本量下的最小值），配对 t 也过线。

报的时候用中位数 +6.5%，不用均值 +10.3%。原因是 r4 那个 +29.9% 是离群值：五个值里其余四个落在 1–10%，它一个把标准差抬到 11.3%。均值被它拉高了近 4 个百分点，而中位数不动。脚本一次把 mean / median / sd / 2SE 四个都打出来，就是为了让这个选择必须当面做出，而不是默认取均值（cs336_systems/experiments/analyze_ddp_replicate.py:103）。

常见误解: 跑的次数越多，噪声就被平均掉了，所以 5 次的均值比单次可信

既然单次测量有 $\pm 6\%$ 的噪声，那跑 5 次取平均就能把噪声压到 $6\% / \sqrt{5} \approx 2.7\%$ ，于是 +10.3% 这个均值已经安全地大于噪声了，可以直接报。

这段话有两处错。第一， $\sqrt{n}$ 只压均值的标准误，压不掉分布的重尾——这里 sd 是 11.3%，2SE 是 10.2%，均值 10.3% 只领先 0.1 个百分点，全靠一个离群值。第二， n 同时也是 sign test 的天花板 (§6.6)，多跑几次并不能无限换到显著性。

Checkpoint 6.5 (median_not_mean): 为什么报中位数 +6.5% 而不是均值 +10.3%

不看代码回答：

(a) 五个逐轮值是 +1.1 / +6.5 / +4.3 / +29.9 / +9.3%。中位数和均值各是多少？差别从哪来？

(b) 如果只能报一个数给面试官，报哪个？为什么另一个不算撒谎但仍然不该报？

Hint 先问“去掉最大的那个值，结论还在不在”。

参考答案: (a) 中位数 6.5%，均值 10.2–10.3%。差别全部来自 r4 的 +29.9%：去掉它之后剩下四个是 1.1/4.3/6.5/9.3，均值降到 5.3%。(b) 报中位数。均值不是假数，但它对单个观测过度敏感，在 $n = 5$ 、sd 11.3% 的样本上把它当作效应量，等于把一次运行的偶然值写进结论。

代码位置: cs336_systems/experiments/analyze_ddp_replicate.py:103

6.8 让脚本去否定实验设计本身

rep-major 交替是为了降低方差。问题是：它真的降了吗？这句话本身是可测的，所以分析脚本里专门留了一段去测它，而不是在报告里断言它成功了。代码注释直接写着“更早的版本只是宣布它有效”：

```

127     if unpaired_cvs and paired_sds:
128         u, p = st.mean(unpaired_cvs), st.mean(paired_sds)
129         n_pairs = max(len(sorted(set(data[a]) & set(data[b]))) for a in data for b in
data if a != b)
130         print(f"\nPairing check: mean unpaired CV {u:.1f}% vs mean paired sd {p:.1f}%.")
131         if n_pairs < 4:
132             print(f" With only {n_pairs} paired reps both estimates are themselves very
"
133                   f"noisy -- this comparison is not yet meaningful either way.")
134         elif p < 0.6 * u:
135             print(" Paired spread is materially smaller: the rep-major interleaving is "
136                   "cancelling shared drift, as intended.")
137         else:
138             print(" Paired spread is NOT materially smaller than unpaired. The variance
is "
139                   "then mostly within-run (each measurement is only ~78s), not shared
drift "
140                   "-- longer measurements per run would help more than more reps.")

```

实测结果是 paired sd 11.3% vs unpaired CV 9.5%——配对没有降低方差，走的是上面的 `else` 分支。脚本 docstring 里预期的“配对标准差通常远小于非配对”(cs336_systems/experiments/analyze_ddp_replicate.py:13) 被自己的输出否掉了。

这个否定结论比它想验证的东西更有用。配对能消掉的只有运行之间共享的漂移；配对没帮上忙，说明方差主要来自单次运行内部。于是改进方向整个反了过来：不是“多跑几轮”，而是“每轮跑更久”多跑轮次只能把已经很大的单次方差除以 $\sqrt{n}$ ，还要付 §6.6 那个天花板的代价；跑更久是直接把单次方差本身降下来。

按脚本 docstring 的说法，复测里一次运行只有约 78 秒 (cs336_systems/experiments/analyze_ddp_replicate.py:6)。这个数字我们没能核实——复测的 CSV 没同步回本地，而本地留下的 bucket 扫描每次是 2 个计时 epoch、wall 174–208 秒。所以“78 秒”按脚本自述记录，具体是哪种配置存疑。

Checkpoint 6.6 (pairing_did_not_help): 配对没有降低方差，说明了什么

不看代码回答：

- (a) 配对（轮内作差）能消掉哪一类方差？消不掉哪一类？
- (b) 已知 paired sd $\approx$ unpaired CV。下一步该增加重复轮数还是延长单次时长？为什么？

Hint 把总方差拆成“运行间共享的”和“运行内自有的”两项。

参考答案：(a) 消掉运行之间共享的漂移（温度、机器负载的慢变化）；消不掉每次运行内部自有的抖动（随机窗口带来的 page cache 冷热、allocator 状态等）。(b) 延长单次时长。配对无效说明共享项很小，总方差几乎全是运行内的；延长测量时间是在源头上摊薄它，而增加轮数只能除以 $\sqrt{n}$ ，还撞 n 的显著性天花板。

代码位置：cs336_systems/experiments/analyze_ddp_replicate.py:130、...:137

6.9 两个诊断错误

噪声这么大，很自然会去找原因。我们找错了两次，两次的错法都值得单独记。

无效的 throttle 检查。第一次怀疑是过热降频，于是去读 GPU 的 throttle 状态——但那次查询发生在 torch.compile 阶段，GPU 当时是空闲的。降频只在负载下才会发生，在空闲时查“有没有 throttle”，得到的“没有”不包含任何信息，等于什么都没测。后来在负载下重查，真实情况

是两张卡都钉在约 278 W 的 280 W power cap (`clocks_event_reasons.active=0x4`) ——是功耗墙，不是过热墙。

把 3 个点叫趋势。中途看到 Naive 连续三轮下降 ($8629 \rightarrow 7800 \rightarrow 7141$)，当时说“这是趋势不是噪声”。否掉它的不是统计，是同期的对照：PyTorch 在同样这几轮里是上升的 ($8450 \rightarrow 9661$)，全局衰减的解释直接不成立。事后补算概率：3 个可交换的观测排成指定方向的严格单调序列，概率是 $1/3! = 1/6 \approx 17\%$ （任一方向则是 $1/3$ ）。六分之一的东西不需要解释。

Checkpoint 6.7 (`invalid_throttle_check`): 什么样的检查是“无效检查”

不看代码回答：

- (a) 在 GPU 空闲时查询 throttle 状态，为什么这个“没有 throttle”的结果不能用？
- (b) 真实原因是 280 W power cap 而不是温度。这两者在现象上有什么区别？

Hint 一个检查只有在“如果假设为真，它会给出不同结果”的条件下才有信息。

参考答案：(a) 空闲时无论有没有降频问题，查询都会返回“无”。这个检查在两种世界里输出相同，所以它的结果不更新任何信念。要在负载中查才有效。(b) power cap 在负载一上来就立刻生效、水平稳定（两卡都钉在 ~ 278 W），温度墙则表现为随时间逐渐下滑。二者的时间形状不同，这也是为什么“三轮递减”看起来像温度墙。

代码位置： `interview\_prep/PROGRESS.md:169`

Checkpoint 6.8 (`three_point_trend`): 3 个点的单调序列意味着什么

不看代码回答：

- (a) 3 个独立同分布的观测，按时间恰好严格单调递减的概率是多少？
- (b) 最终否掉“全局衰减”这个解释的是什么证据？

Hint 只需要数 $3!$ 种排列里有几种符合。

参考答案：(a) 指定方向 $1/6 \approx 17\%$ ，任一方向 $1/3$ 。所以三连降完全在随机范围内。(b) 同期的对照条件：PyTorch DDP 在同样这几轮里是上升的。如果是机器整体在衰减，所有条件应该一起下降。这也是 rep-major 交替顺带买到的好处——每一轮都有对照，才能做这种反驳。

代码位置： `interview\_prep/PROGRESS.md:169`

6.10 bucket size 扫描：只报被重复验证过的那一条

单次扫描给出 1 MB $\rightarrow$ 8242.4、10 MB $\rightarrow$ 8032.1、25 MB $\rightarrow$ 8080.2、50 MB $\rightarrow$ 9589.5、100 MB $\rightarrow$ 8104.4 tok/s。50 MB 比其余四个的均值高 18.2%，而其余四个彼此散布在 $\pm 1.3\%$ 以内。一个孤峰配一片平地，更像噪声而不是曲线形状——而且当时那段汇总代码直接打印了“best: 50MB”，正是同一晚刚在别的脚本里修掉的毛病：先写结论再看数据 (§5)。所以只对 50 MB 和 25 MB 各补了 3 次重复。

能报的只有一句：50 MB 显著快于 25 MB，约 +16%，依据是三次重复区间不重叠——这是个不依赖任何分布假设的判据。不报“最优 bucket size 是多少”，也不报曲线形状：1 / 10 / 100 MB 各只有一次样本。同一批数据里，50 MB 三次跨 8553–9590 ($\pm 6\%$)，这正说明单点既不足以断言“50 MB 最优”，也不足以断言“那是噪声”——两个方向都得重复才知道。

bucket	第 1 次	第 2 次	第 3 次	均值	min	max
50 MB	9589.5	9125	8553	9089	8553	9590
25 MB	8080.2	7659	7794	7844	7659	8080

Table 21: 50 MB 与 25 MB 各三次重复, 单位 tok/s. $\min(50) = 8553 > \max(25) = 8080$, 两个区间完全不重叠。第 1 次的两个数来自本地 `artifacts/bench_bucket_50mb/` 与 `bench_bucket_25mb/`; 第 2、3 次只记录在 `interview_prep/PROGRESS.md` 里, 重复运行覆盖了同名 CSV, 本地已无原始文件。

Checkpoint 6.9 (bucket_50mb): 三次重复不重叠, 能报什么、不能报什么

不看代码回答:

- (a) “ $\min(A) > \max(B)$ ”作为判据, 比“均值差 16%”强在哪?
- (b) 已知 50 MB 快于 25 MB。为什么仍然不能说“最优 bucket size 是 50 MB”?

Hint 区间不重叠是一个不需要假设分布的陈述。

参考答案: (a) 它不依赖正态性、不依赖方差估计, 也不受重尾影响; 在 $n = 3$ 这种样本量下, 均值差 16% 的标准误本身就很不可靠, 而“三次里最差的仍然优于对方最好的”是可以直接看的。(b) 曲线上另外三个点 (1 / 10 / 100 MB) 都只测了一次, 落在 8032–8242, 既没有重复也没有排除 50 MB 那种量级的单点抖动。我们只验证了一对比较, 就只能报这一对。

代码位置: `artifacts/bench\_bucket\_25mb/lm\_matrix\_results.csv:2`、`interview\_prep/PROGRESS.md:186`

6.11 README 里的旧数字: 并列, 不是覆盖

仓库 README 上写着另一组 DDP 数字: 85.9% scaling efficiency、比 Naive DDP 快 9.4% (25,336.7 → 27,719.1 tok/s)、峰值显存 +3.3% (+498.8 MB)。这次的真实 190M 模型上, 对应的数是 79.1% scaling efficiency (8398.0/(2 × 5309.2))、比 Naive 快 2.6% (已撤回)、显存 +3.7% (+741.1 MB)。

新数不应该覆盖旧数, 因为两者不是同一个场景。README 的命令指向 `default_pipeline_config.json`: vocab 10000、12 层、 d_{ff} 3072、TinyStories 语料; 这次是 vocab 50257、16 层、 d_{ff} 2048、真实混合语料。模型越小、通信占 step 时间的比例越大, 重叠能省下的相对收益就越明显——旧配置下 naive 只有 25,336.7 tok/s 却仍能被拉高 9.4%, 和新配置下 8188.7 tok/s 上只剩 2.6%, 方向是自洽的。正确的写法是并列成“在 TinyStories/12 层配置下……/ 在 190M 真实模型下……”两组。

磁盘上其实已经是并列状态: README 引用的图还停在 `artifacts/` 根目录 (7 月 15 日), 新一轮写进了 `artifacts/bench_matrix/` (7 月 28 日), 没有互相覆盖。顺带一个核实到的不一致: README 里那条命令的注释写“bucket size 20 MB” (README.md:96), 而它 `--config` 指向的文件里 `bucket_size_mb` 是 25 (`cs336\_systems/experiments/default\_pipeline\_config.json:61`)。两个数字对不上, 所以那次实验用的 bucket size 从仓库里已经无法确定——这本身就是“旧数字不能直接和新数字比”的又一个理由。

Example 6.1 (retracting_a_result): 撤回一条结论的完整流程

1. 发现。做别的实验 (bucket 复测) 时顺手量到同配置三次是 9590 / 9125 / 8553, 第一次意识到自己从没测过噪声。回头一看, 已经报出去的差异是 2.6% 和 1.6%。
2. 量化。把噪声当成一个要测的量而不是一句免责声明: 单次 ±6%, 隔夜同配置 5.4%, 5-rep

复测的非配对 CV 9.5%。两个独立单次之商再乘 $\sqrt{2}$ 。

3. 分类。逐条判断原表里哪些结论受影响：OOM 那一行不受影响（二值），跨 kernel 的百分比降级（kernel-major 执行顺序造成时间混淆），DDP 三策略之间的百分比全部撤回。

4. 重新设计。5 reps $\times$ 3 wrapper, rep-major 交替；并且在分析脚本里显式测量“配对到底有没有帮上忙”，而不是假定它帮上了。

5. 只保留站得住的。三种检验全部列出；先说清 $n = 5$ 的 p 值天花板；只报 Bucketed > Naive 一条，用中位数 +6.5%；另外两条明确写“分辨不出”和“方法打架，不报”。

6. 旧结论保留原样。§6.2 的红框不删。删掉它，读者就看不出这条结论曾经存在过，也就学不到它为什么会被写出来。

这一章没有产出一个更快的 kernel，也没有把 DDP 改快。它产出的是一个门槛：在报一个百分比之前，先花一次实验的成本去测自己的噪声。按本章的账算，那次复测花掉的 GPU 时间比原来那张矩阵表还多，换回来的是把三条结论删掉两条、第三条从 +2.6% 改成 +6.5%。这笔交易在简历上看起来是亏的，在面试里不是。

Epilogue

这份讲义覆盖的是一次时间盒住的冲刺：从零建语料、训一个 0.19B 模型、做 6 个 arm 的微调 ablation、把权重接进 llm-serving 的请求链路、再补一轮系统 benchmark 和它的复测。收尾这一节记三件事：东西现在在哪、下一步该做什么、以及这次冲刺里哪些结论是被自己撤掉的。

交付状态：什么在本地，什么留在了远端

vast.ai 实例已经 stop（状态 exited），算力停止计费，磁盘保留但磁盘费仍在走。下表的“本地”一列是写这份讲义时在本机磁盘上逐个核对过的，不是凭记忆列的。

产物	本地状态
checkpoints_sft_lr1e5_replay/（6 个 arm 里的 Pareto 最优模型）	完整，2,285,748,962 B = 2.29 GB；已 torch.load 验证：0.190B 参数、global_step=900、含 optimizer state
artifacts/（NIGHT_REPORT.md + benchmark CSV/PNG + 13 个 eval JSON）	完整，2.5 MB
run_logs/（29 个日志）、configs_sft/（6 个）、全部代码	完整
checkpoints_pretrain_v1/	只下到 671,519,672 B，即完整版的 29.4%
checkpoints_sft_lr1e5_plain/（对照组）	65.9%
data/mixed_corpus/	空目录（0%），重建需 2–3 小时
artifacts/ddp_replicate/	目录不存在——5 reps 复测的原始 CSV 一行都没回来
三个编排脚本（continue_pretrain_pipeline.sh、post_train_window.sh、stage8_leftover_benchmarks.sh）	不在本地，只在那个实例上

Table 22: 交付状态。上半区是完整的，下半区是被 SSH 限流截断的。

最后两行的代价是具体的，不是抽象的“数据没下全”。artifacts/ddp_replicate/ 缺失，直接就是 §6 表 20 里那几个“—”的来源：逐轮百分比、中位数、 p 值都还在 PROGRESS.md 的正文里，但原始数字没了，重算不了。三个编排脚本是这次冲刺工程含量最高的一块（吞吐标定 → NaN 门禁 → 反推 max_iters → 无人值守窗口），丢了就要重想一遍。

真正不可替代的那一个——Pareto 最优的 SFT 权重——在手上。缺的两个里，plain 只是对照组（结论已经量化记录在 §4），pretrain_v1 只在要继续训练时才需要，而继续训练本来就要重新租 GPU。所以按 ROADMAP.md 第 3 条的判断：没有什么是必须抢救的。

想让模型变好，瓶颈是数据不是步数

最容易走错的下一步是“再多训几万步”。数字不支持这个方向：语料只有约 174M unique token，而这次已经训了 492M token——整整 2.8 遍。再加步数是在重复同样的字节。换算成另一个角度，492M token 对 0.19B 参数是 2.6 token/param，离 Chinchilla 的 20× 差了近一个数量级，但这个缺口不是靠步数补的。

正确的动作是把 build_mixed_corpus.py 的字符预算从 780 MB 提到约 3 GB（C4 和 Wikipedia 都是流式的，取之不尽）→ 约 700M token，再训到约 2B token（10.5 token/param）。成本是建语料 2–3 小时加训练约 12 小时。

有一个前置条件必须先做：先修验证集污染，再重新建语料。build_instruction() 会 cycle 小语料池来凑字符预算——这对 train split 是对的（那正是按 epoch 数给小而精的域配比的做法），但同一段逻辑也被用在了 valid split 上，把 held-out 的 Alpaca 重复了 1.91 遍 (§2)。不先修它，新一轮训练出来的验证数字仍然不可信，12 小时就白烧了。

完整的优先级、成本估算和收尾项在仓库根的 ROADMAP.md（Part I 共 7 条，上面两条就是它的第 1、2 条）。这里不重复，只强调它的排序依据是依赖关系而不是价值大小。

这次冲刺撤回或修正的四条结论

按 `interview_prep/PROGRESS.md` 自己的记账，这次冲刺一共撤回或修正了四条自己先报错的结论。四条的共同点是：都不是别人指出来的，都是继续往下测的时候被自己的下一个数据点否掉的。

1. **DDP 三种策略之间的性能对比 (§6)**。“Bucketed 比 Naive 快 2.6%”“PyTorch DDP 比 Bucketed 快 1.6%”都来自每格只跑一次的矩阵表，而后来量到单次 run-to-run 噪声是 $\pm 6\%$ 、隔夜同配置差 5.4%。全部撤回。复测后只剩一条站得住：Bucketed 快于 Naive，5/5 全胜，中位数 +6.5%，sign test $p = 0.062$ 。
2. **KV cache 的加速比随 prompt 长度增长 (§5)**。这句话是被 `bench_serving.py` 和 `make_night_report.py` 无条件 print 出来的硬编码结论，不看实际数字。实测 16→512 token 的加速比是 1.20 / 1.28 / 1.20 / 1.27 / 1.23 $\times$ ，平的，没有趋势。于是报告里出现了“表格显示没趋势 + 下一行断言它在增长”的荒唐组合。正确性那一侧倒是干净的：缓存解码与重算解码 logits 逐位相同 (Assignment A §3.2 <KV cache 为什么合法>)，被推翻的只是那句性能断言。
3. **训练内 val 曲线的代表性 (§3)**。4.4579 → 4.0057 这条曲线不是混合验证集上的 loss。三件事叠在一起 (`valid.txt` 按域顺序写、`val_sampler` 是 `shuffle=False`、`_run_eval` 取前 50 个 batch)，每次 eval 打的都是 `valid.txt` 开头约 1800 个 token，也就是同样那几篇 web 文档。曲线本身仍然有效 (天然配对，下降是真的)，但不能叫“混合验证集 val loss”——那个数是跑完之后另测的 3.5866。
4. **把 3 个点的单调序列叫“趋势” (§6)**。看到 Naive 连续三轮下降就断言是趋势不是噪声；否掉它的是同期对照——PyTorch 在同样这几轮里是上升的。事后补算：3 个可交换观测排成指定方向的严格单调序列，概率就有 $1/6 \approx 17\%$ 。

这个“四条”是日志自己的计数，不是全部。至少还有三处中途修正没算进去：NaN 的根因从“缺 LR warmup”改判为 SwiGLU FFN 的初始化方差用了默认的 1；TF32 的加速从顺手观察到的 1.5 $\times$ 改成对照实验测出的 1.43 $\times$ ；以及“继承 optimizer state 会和 LR schedule 打架”这句话被限定为只对微调成立，对同任务续训不成立。

操作层教训：一个主题，三个实例

这次踩的操作坑里，最贵的三个属于同一类错误——包装层掩盖了真实的退出码或状态。分开看像三件无关的事，合起来看是同一个反模式：在命令和它的真实结果之间加了一层，而那一层有自己的成功语义。

这张表和 §6 讲的其实是同一件事，只是换了个层面：一个 benchmark 在读自己的数据之前就把结论 print 出来，和一条命令链把失败的退出码 pipe 掉，是同一种错误——中间隔了一层，而那一层总是报成功。前者毁掉的是结论，后者毁掉的是数据。两者都不会报错，都要靠人主动去掀开那一层看。

最后

这次冲刺真正打通的是一条线：公开语料 → 按配比混合 → 2 卡约 6.5 小时预训练 → 6 个 arm 的指令微调 ablation → 一条真实请求走完 RequestReceiver 到自训权重的全链路。线是通的，模型是欠训的，几个性能数字是被自己撤回过一轮的。

三样东西按“值得讲”排序，恰好和“看起来漂亮”排序相反：最值得讲的是 25% 预训练数据回放在指令微调中是严格的 Pareto 改进 (§4，指令能力更好、遗忘代价低 3 倍)；其次是一个加权平均对不上，顺着追下去发现是自己在验证集上误用了训练集的正确做法 (§2)；最后才是那些吞吐百分比——它们大部分没活过复测。

现象	被掩盖的是什么	规则
主题：包装层掩盖了真实的退出码或状态		
<code>pkill -f</code> 按脚本名杀进程时自匹配（踩了三次）	自己那条 <code>ssh/bash</code> 命令行里就含有这个关键词，于是先把自己杀掉；同样地， <code>pgrep -f</code> 判活会永远返回“还在跑”	凡是要杀远程进程，先 <code>ps</code> 拿 PID 再 <code>kill <pid></code> ；判活用 <code>kill -0 <pid></code> ，不用字符串匹配。要么把脚本写成文件再 <code>bash file.sh</code> ，命令行里就不再出现关键词
<code>rsync ... tail</code>	<code>&&</code> 拿到的是 <code>tail</code> 的退出码， <code>rsync</code> 的失败被吞掉，链条继续往下走（还叠了一层：macOS 自带 <code>rsync 2.6.9</code> 不支持 <code>--info=progress2</code> ）	要判成败的命令不要 <code>pipe</code> 给别的命令；非 <code>pipe</code> 不可就用 <code>set -o pipefail</code> 或读 <code>PIPESTATUS</code>
SSH 被限流	<code>vastai show instances</code> 显示实例 <code>running</code> ，于是把连接被拒当成网络抖动；第一反应是 20 秒一次加密重试，每敲一次刷新一次封禁计时	所有 <code>rsync</code> 复用一条连接： <code>ControlMaster=auto + ControlPath + ControlPersist=30m</code> ，只认证一次。被拒立刻指数退避，绝不加密
其余		
<code>lscheckpoint_step_*.pt tail -1</code>	默认是字典序，挑出 <code>step_7500</code> 而不是 <code>step_22500</code> ；偏偏只在“训练早死、 <code>fallback</code> 生效”时触发，也就是这个选择最要紧的时候	<code>sort -V</code>
“校验不过就不停实例”的自动闸	本意是防数据丢失，但 <code>vastai stop</code> 保留磁盘、根本不丢数据，而停机再开会重建容器和 <code>sshd</code> 、大概率清掉封禁	写防护闸时要先确认它挡住的是不是唯一的解法

Table 23: 操作层教训。上半区三行是同一个反模式的三个实例。